



Guía de referencia SQL de SAP HANA Cloud, base de datos de SAP HANA

Generado el: 2026-06-06 23:19:41 GMT+0000

SAP HANA Cloud, SAP HANA Database | QRC 1/2026

Público

Contenido original: https://help.sap.com/docs/HANA_CLOUD_DATABASE/c1d3f60099654ecfb3fe36ac93c121bb?locale=en-US&state=PRODUCTION&version=2026_1_QRC

Exención de responsabilidades sobre la traducción automática

Para su comodidad, este documento PDF se ha traducido de forma automática. SAP no ofrece ninguna garantía respecto a la precisión o integridad de la traducción automática. La versión original en inglés tiene prioridad.

Si desea hacer algún comentario al respecto, utilice la opción de feedback del SAP Help Portal disponible en la versión HTML de este documento.

Advertencia

Este documento lo ha generado el SAP Help Portal y es una versión incompleta de la documentación oficial del producto de SAP. La información incluida en la documentación personalizada puede no reflejar la disposición de los temas en SAP Help Portal y puede carecer de aspectos y/o correlaciones importantes con otros temas. Por este motivo, no se recomienda para uso productivo.

Para obtener más información, visite <https://help.sap.com/docs/disclaimer>.

Funciones SQL

Documenta las funciones SQL integradas que se proporcionan con SAP HANA.

Las funciones analíticas se posicionan en esta guía como funciones agregadas y funciones agregadas de ventana.

[Lista alfabética de funciones](#)

[Agregar funciones](#)

Las funciones de agregación son funciones analíticas que calculan un valor agregado basado en un grupo de filas.

[Funciones de matriz](#)

Las funciones de matriz toman matrices como entrada.

[Funciones de conversión de tipo de datos](#)

Las funciones de conversión de tipo de datos convierten datos de un tipo de datos a otro.

[Funciones de fecha y hora](#)

Las funciones de fecha y hora realizan operaciones en tipos de datos de fecha y hora o información de fecha u hora de devolución.

[Funciones de jerarquía](#)

Las funciones de jerarquía le permiten trabajar con datos jerárquicos, como tablas con filas dispuestas en un árbol o gráfico dirigido.

[Funciones JSON](#)

Las funciones JSON son funciones que devuelven u operan en datos JSON.

[Otras funciones](#)

SAP HANA admite muchas funciones que devuelven valores del sistema y realizan varias operaciones en valores, expresiones y valores de retorno de otras funciones.

[Funciones numéricas](#)

Las funciones numéricas realizan operaciones matemáticas en tipos de datos numéricos o información numérica de retorno.

[Funciones de seguridad](#)

Las funciones de seguridad proporcionan una funcionalidad especial para fines de seguridad.

[Funciones de datos de serie](#)

Las funciones de datos de serie proporcionan una funcionalidad especial para los datos de serie y las tablas de serie.

[Funciones de cadena](#)

Las funciones de cadena realizan la extracción y manipulación de cadenas, o devuelven información sobre cadenas.

[Funciones vectoriales](#)

Las funciones vectoriales realizan cálculos en vectores.

[Funciones de ventana y la especificación de ventana](#)

Las funciones de ventana le permiten realizar operaciones analíticas en un conjunto de filas de entrada.

Lista alfabética de funciones

Función ABAP_ALPHANUM (cadena)

Convierte una cadena a lo que resultaría si la cadena se transformara en un tipo ALPHANUM y luego se convierta de nuevo en una cadena.

Sintaxis

```
ABAP_ALPHANUM( <string>, <chars> )
```

Elementos de sintaxis

<string>

Especifica la cadena de entrada que se va a convertir.

<chars>

Especifica la longitud para rellenar a la izquierda <string> con (usando 0s) si <string> es más corto que la longitud especificada por <chars>.

Descripción

El valor de entrada, <string>, se verifica para averiguar si es numérico o una mezcla de numéricos y no numéricos. Si <string> es numérico, se rellena a la izquierda con ceros hasta la longitud especificada por <chars>, si es necesario. Si <string> es una mezcla o numérico y no numérico, o está vacío, se devuelve sin modificar.

Las cadenas que empiezan con espacios y que solo contienen dígitos después del prefijo de espacio en blanco se consideran numéricas.

Ejemplo

El siguiente ejemplo devuelve 012. Dado que <chars> está fijado en 3, el dígito más a la izquierda se rellena con un 0.

```
SELECT ABAP_ALPHANUM( '12' , 3) FROM DUMMY;
```

El siguiente ejemplo devuelve 1234. El valor de entrada era numérico y no era más corto que <chars>, por lo que no se requería relleno a la izquierda.

```
SELECT ABAP_ALPHANUM( '1234' , 3) FROM DUMMY;
```

El siguiente ejemplo devuelve 12x. Dado que el valor de entrada contenía una mezcla de numéricos y no numéricos, se devuelve sin modificar.

```
SELECT ABAP_ALPHANUM( '12x' , 13) FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función ABAP_DF16RAW_TO_SMALLDECIMAL (string)

Convierte del tipo *DECFLOAT* ABAP *DF16_RAW* al tipo *DECFLOAT* HANA *SMALLDECIMAL*.

Sintaxis

```
ABAP_DF16RAW_TO_SMALLDECIMAL(<binary_data_in_df16raw_format>)
```

Elementos de sintaxis

<binary_data_in_df16raw_format>

El formato a convertir: Punto flotante decimal, 16 dígitos, RAW en la base de datos

Descripción

Convierte del tipo *DECFLOAT* ABAP *DF16_RAW* al tipo *DECFLOAT* HANA *SMALLDECIMAL*.

Tenga en cuenta que solo está disponible la conversión unidireccional del tipo ABAP a HANA. La conversión inversa debe realizarse en la capa ABAP si es necesario. Para obtener más información, [consulte Números de coma flotante decimales](#) en la documentación de palabras clave ABAP.

Ejemplo

En el siguiente ejemplo, se convierte el valor binario al tipo decimal.

```
CREATE TABLE t1(b binary(16))
INSERT INTO t1 values(X'BF2C000000000000')
INSERT INTO t1 values(X'BF54000000000001')
INSERT INTO t1 values(X'3DE3E7F9FE7F9FE7')

SELECT ABAP_DF16RAW_TO_SMALLDECIMAL(b) FROM t1
/* returns
[ Decimal('0.1')
, Decimal('1.0000000000000001')
, Decimal('-0.1')]
*/
```

Función ABAP_DF34RAW_TO_DECIMAL (string)

Convierte del tipo *DECFLOAT* ABAP *DF34_RAW* al tipo *DECFLOAT* HANA *DECIMAL*.

Sintaxis

ABAP_DF34RAW_TO_DECIMAL (<binary_data_in_df34raw_format>)

Elementos de sintaxis

<binary_data_in_df34raw_format>

El formato a convertir: Punto flotante decimal, 34 dígitos, RAW en la base de datos

Descripción

Convierte del tipo *DECFLOAT* ABAP *DF34_RAW* al tipo *DECFLOAT* HANA *DECIMAL*.

Tenga en cuenta que solo está disponible la conversión unidireccional del tipo ABAP a HANA. La conversión inversa debe realizarse en la capa ABAP si es necesario. Para obtener más información, [consulte Números de coma flotante decimales](#) en la documentación de palabras clave ABAP.

Ejemplo

El siguiente ejemplo convierte el valor binario al tipo *DECIMAL*.

```
CREATE TABLE t2(b binary(64));
INSERT INTO t2 values(X'BDBBC00000000000000000000000000');
INSERT INTO t2 values(X'BDBE4000000000000000000000000001');
INSERT INTO t2 values(X'BDBEC8D9428D937193B9CEA0D7F45DF7');

SELECT ABAP_DF34RAW_TO_DECIMAL(b) FROM t2
/* returns
[ Decimal('0.1')
, Decimal('1.0000000000000000000000000000000001')
, Decimal('3.141592653589793238462643383279503')]
*/
```

Función ABAP_LOWER (cadena)

Convierte en minúsculas todos los caracteres de una cadena especificada.

Sintaxis

ABAP_LOWER(<string>)

Elementos de sintaxis

<string>

La cadena que se convertirá a minúsculas.

Descripción

Convierte en minúsculas todos los caracteres del parámetro *<string>*.

Ejemplo

El siguiente ejemplo convierte la cadena indicada en minúsculas y devuelve el valor ant.

```
SELECT ABAP_LOWER ( 'AnT' ) "lower" FROM DUMMY;
```

Información relacionada

Tipos de datos de cadena de caracteres

Función ABAP_NUMC (cadena)

Convierte una cadena de entrada en una cadena de una longitud especificada que solo contiene dígitos.

Sintaxis

ABAP_NUMC(<string>, <chars>)

Elementos de sintaxis

<string>

Especifica la cadena de entrada que se va a convertir.

<chars>

Especifica la longitud que debe tener la cadena devuelta.

Descripción

Los valores numéricos y de cadena se rellenan a la izquierda con 0s si *<string>* es más corto que *<chars>*. Si *<string>* es más largo que *<chars>*, los caracteres se truncan desde la izquierda.

Los valores numéricos se redondean a su parte entera y el signo se descarta.

Los valores TIMESTAMP, SECONDDATE, DATE y TIME se rellenan y truncan desde el lado derecho, según sea necesario.

Las entradas DECIMAL y SMALLDECIMAL nunca se truncan, sino que crean un error de desbordamiento si se detecta que son demasiado largas.

Ejemplo

El siguiente ejemplo devuelve 012.

```
SELECT ABAP_NUMC(12, 3) FROM DUMMY;
```

El siguiente ejemplo devuelve 234.

```
SELECT ABAP_NUMC(1234, 3) FROM DUMMY;
```

El siguiente ejemplo devuelve 01230000000000000000.

```
SELECT ABAP_NUMC(1.23e18, 20) FROM DUMMY;
```

El siguiente ejemplo devuelve 001235.

```
SELECT ABAP_NUMC(-1234.5, 6) FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función ABAP_UPPER (cadena)

Convierte en mayúsculas todos los caracteres de la cadena especificada.

Sintaxis

```
ABAP_UPPER(<string>)
```

Elementos de sintaxis

<string>

La cadena que se va a convertir a mayúsculas.

Descripción

Convierte en mayúsculas todos los caracteres del parámetro **<string>**.

Ejemplo

El siguiente ejemplo convierte la cadena indicada en mayúsculas y devuelve el valor ANT:

```
SELECT ABAP_UPPER ( 'Ant' ) "uppercase" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función ABS (numérica)

Devuelve el valor absoluto de un argumento numérico.

Sintaxis

ABS (**<num>**)

Elementos de sintaxis

<num>

Especifica el argumento numérico.

Descripción

Devuelve el valor absoluto del argumento numérico **<num>**.

Ejemplo

El siguiente ejemplo devuelve el valor 1 para "absoluto":

```
SELECT ABS (-1) "absolute" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función ACOS (numérica)

Devuelve el arco-coseno, en radianes, de un argumento numérico entre -1 y 1.

Sintaxis

`ACOS(<num>)`

Elementos de sintaxis

<num>

Especifica un argumento numérico entre -1 y 1.

Descripción

Devuelve el arco-coseno, en radianes, del argumento numérico <n> entre -1 y 1.

Ejemplo

El siguiente ejemplo devuelve el valor 1.0471975511965979:

```
SELECT ACOS (0.5) "acos" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función ADD_DAYS (fecha y hora)

Calcula la fecha especificada, más o menos el número de días especificado.

Sintaxis

`ADD_DAYS(<date>, <num_days>)`

Elementos de sintaxis

<date>

Especifica la fecha para el incremento.

<num_days>

Especifica el número de días (entero). Un entero positivo incrementa la fecha por el importe especificado; un entero negativo disminuye la fecha.

Descripción

Calcula la fecha especificada más o menos el número de días especificado.

Ejemplo

El siguiente ejemplo incrementa el valor de fecha 2009-12-05 en 30 días y devuelve el valor 2010-01-04:

```
SELECT ADD_DAYS (TO_DATE ( '2009-12-05' , 'YYYY-MM-DD' ), 30) "add days" FROM DUMMY;
```

El siguiente ejemplo reduce el valor de fecha 2009-12-05 en 30 días y devuelve el valor 2009-11-05:

```
SELECT ADD_DAYS (TO_DATE ( '2009-12-05' , 'YYYY-MM-DD' ), -30) "subtract days" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función ADD_MONTHS (fecha y hora)

Calcula la fecha especificada más el número de meses especificado.

Sintaxis

```
ADD_MONTHS(<date>, <num_months>)
```

Elementos de sintaxis

<date>

La fecha que se incrementa.

<num_months>

El número de meses en los que se incrementa la fecha. La especificación de un número negativo disminuye la fecha.

Descripción

Calcula la fecha especificada más el número de meses especificado.

Para calcular la fecha de modo que la fecha de salida se fije en el último día del mes cuando la fecha de entrada sea el último día del mes, utilice la función ADD_MONTHS_LAST.

El parámetro **<date>** debe convertirse implícita o explícitamente a uno de los siguientes tipos de datos SQL:

- FECHA
- CRONOMARCADOR
- SECONDDATE

El tipo de datos SQL de los parámetros de salida es el mismo que el tipo de datos SQL de los parámetros de entrada. Por ejemplo, ADD_MONTHS_LAST(DATE) devuelve una fecha, mientras que ADD_MONTHS_LAST(TIMESTAMP) devuelve un cronomarcador.

Ejemplo

El siguiente ejemplo incrementa la fecha 2009-12-05 en un mes y devuelve el valor 05/01/2010:

```
SELECT ADD_MONTHS (TO_DATE ( '2009-12-05' , 'YYYY-MM-DD' ), 1) "add months" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función ADD_MONTHS_LAST (fecha y hora)

Calcula la fecha especificada más el número de meses especificado, siendo la fecha de salida el último día del mes si la fecha de entrada es el último día del mes, incluso si esas dos fechas difieren.

Sintaxis

```
ADD_MONTHS_LAST(<date>, <num_months>)
```

Elementos de sintaxis

<date>

La fecha que se incrementa.

<num_months>

El número de meses en los que se incrementa la fecha.

Descripción

Calcula la fecha especificada más el número de meses especificado. Si la fecha de entrada es el último día del mes de entrada, la fecha de salida se fija en el último día del mes de salida.

Para calcular la fecha más los meses sin utilizar la funcionalidad del último día del mes, utilice la función ADD_MONTHS.

El parámetro **<date>** debe convertirse implícita o explícitamente a uno de los siguientes tipos de datos SQL:

- FECHA
- CRONOMARCADOR
- SECONDDATE

El tipo de datos SQL de los parámetros de salida es el mismo que el tipo de datos SQL de los parámetros de entrada. Por ejemplo, ADD_MONTHS_LAST(DATE) devuelve una fecha, mientras que ADD_MONTHS_LAST(TIMESTAMP) devuelve un cronomarcador.

Ejemplo

El siguiente ejemplo incrementa la fecha 28/02/2009 (el último día de febrero) en un mes y devuelve el valor 31/03/2009 (el último día de marzo).

```
SELECT ADD_MONTHS_LAST (TO_DATE ( '2009-02-28' , 'YYYY-MM-DD' ), 1) "add months last" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función ADD_NANO100 (fecha y hora)

Añade el número especificado de 10^{-7} unidades de segundos al valor TIMESTAMP especificado.

Sintaxis

`ADD_NANO100( <time>, <num> )`

Elementos de sintaxis

<time>

El valor TIMESTAMP al que añadir las unidades de segundos.

<num>

El número de 10^{-7} unidades de segundos en las que se incrementa el valor TIMESTAMP.

Descripción

Calcula el valor TIMESTAMP especificado más el número especificado de microsegundos.

Ejemplo

El siguiente ejemplo incrementa el valor TIMESTAMP en 864000000000 microsegundos y devuelve 1000-01-02 10:00:00.0000000:

```
SELECT ADD_NANO100(TO_TIMESTAMP('1000-01-01 10:00:00.0000000'), 864000000000) FROM DUMMY;
```

El siguiente ejemplo incrementa el valor TIMESTAMP en 1 microsegundo y devuelve 1000-01-01 10:00:00.00000001:

```
SELECT ADD_NANO100(TO_TIMESTAMP('1000-01-01 10:00:00.0000000'), 1) FROM DUMMY;
```

Información relacionada

[Función ADD_SECONDS \(fecha y hora\).](#)

Función ADD_SECONDS (fecha y hora)

Calcula la hora especificada más los segundos especificados.

Sintaxis

`ADD_SECONDS(<time>, <num_seconds>)`

Elementos de sintaxis

<time>

El tiempo que se incrementa.

<num_seconds>

El número de segundos en los que se incrementará el tiempo. Los valores fraccionados se admiten para milisegundos y microsegundos:

- 1/1000 – milisegundos
- 1/1000000 - microsegundos

Descripción

Calcula la hora especificada más el número de segundos especificados.

Ejemplo

El ejemplo incrementa el TIMESTAMP 2012-01-01 23:30:45 en 60*30 segundos y devuelve el valor 2012-01-02 00:00:45.0:

```
SELECT ADD_SECONDS (TO_TIMESTAMP ('2012-01-01 23:30:45'), 60*30) "add seconds" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

[Función ADD_NANO100 \(fecha y hora\)](#)

Función ADD_WORKDAYS (fecha y hora)

Calcula una fecha añadiendo un número de días laborables a una fecha de inicio.

Sintaxis

```
ADD_WORKDAYS(
  <factory_calendar_id>,
  <start_date>,
  <workdays>
  [, <source_schema>
  [, <table_name_suffix>,
  [, <client> ] ] ] )
```

Elementos de sintaxis

<factory_calendar_id>

Especifica el ID del calendario de fábrica.

<factory_calendar_id> ::= <string_literal>

<start_date>

Especifica la fecha de inicio a la que se añadirán los días laborables. Puede utilizar un tipo DATE o una cadena de formato de fecha (por ejemplo, '20140101' o '2014-01-01') para este parámetro.

<start_date> ::= <string_literal> | <date>

<workdays>

Especifica el número de días laborables que se deben añadir a la fecha de inicio.

<workdays> ::= <signed_integer>

Cuando *<días laborables>* es positivo, la fecha calculada resultante es el siguiente día laborable después del período definido por el número de *<días laborables>*. Cuando *<días laborables>* es negativo, la fecha calculada resultante es el día laborable anterior al período definido por la cantidad de *<días laborables>*.

<source_schema>

Especifica el esquema que contiene la tabla Calendario de fábrica o las tablas Calendario de fábrica y calendario de días festivos.

<source_schema> ::= <string_literal>

Este parámetro se puede omitir si el esquema es el mismo que el esquema actual.

<table_name_suffix>

Especifica el sufijo añadido al nombre de tabla del calendario de fábrica estándar o a los nombres de tabla del calendario de fábrica y festivos estándar si se utiliza un conjunto alternativo de tablas. Por ejemplo, con el sufijo "_XYZ", la función accede a las tablas alternativas TFACS_XYZ o FHC_C_FCAL_XYZ en lugar de las tablas estándar TFACS o FHC_C_FCAL. Si se omite o está vacío, se utilizan las tablas estándar.

<table_name_suffix> ::= <string_literal>

<client>

Especifica el mandante utilizado para la implementación del calendario de fábrica y festivos dependiente del mandante. Si se omite o está vacía, la función intenta recuperar el cliente del contexto (parámetro de usuario), que se puede fijar en el contexto mediante ALTER USER *<user_name>* SET PARAMETER CLIENT = *<client>*. El mandante se ignora si la implementación del calendario de fábrica independiente del mandante está activa.

<client> ::= <string_literal>

Tipo de devolución

FECHA

Descripción

Existen dos implementaciones diferentes para el cálculo de días laborables en sistemas basados en ABAP como SAP S/4HANA:

- El calendario de fábrica que almacena sus datos en una única tabla TFACS.
- El calendario de fábrica y festivos que almacena sus datos en varias tablas (FHC*).

Por defecto, la función ADD_WORKDAYS asume que el calendario de fábrica está activo. Una vez que la tabla FHC_CONFIG (o FHC_CONFIG *<table_suffix>* donde se especifica *<table_suffix>*) esté disponible en el sistema y contenga una entrada con KEY_FIELD = 'MIGRATION_COMPLETED' y VALUE = 'X' para el cliente especificado, la función ADD_WORKDAYS asume que el calendario de fábrica y festivos más reciente está activo.

Las tablas correspondientes deben estar disponibles en la base de datos SAP HANA para utilizar la función ADD_WORKDAYS. En SAP BW, SAP CRM y SAP ERP que se ejecutan en una base de datos SAP HANA, estas tablas se encuentran en el esquema ABAP

SAP<SID>. Para otros sistemas de base de datos SAP HANA, estas tablas se pueden replicar desde un sistema SAP Business Suite.

Cuando el calendario de fábrica está activo, ADD_WORKDAYS accede a la tabla TFACS (o TFACS<table_suffix> si se especifica <table_suffix>). Cuando el calendario de fábrica y festivos está activo, ADD_WORKDAYS accede a las siguientes tablas, que deben estar disponibles en el sistema con <table_suffix> añadido a sus respectivos nombres si se especifica <table_suffix>:

- FHC_C_FCAL
- FHC_C_FCAL_EXC
- FHC_C_HCAL
- FHC_C_HCAL_ASSGN
- FHC_C_HOL
- FHC_C_HOL_EASTER
- FHC_C_HOL_FIXED
- FHC_C_HOL_FLOAT
- FHC_C_HOL_FLDATE
- FHC_C_HOL_WKDAY
- FHC_CONFIG

Ejemplos

Los siguientes ejemplos utilizan exclusivamente la tabla Calendario de fábrica para el mes de enero de 2014. Para obtener ejemplos de cómo utilizar el calendario de fábrica y de días festivos, consulte [Calendario de fábrica](#).

Campo de bits de TFACS	Día del mes	Motivo por el que no funciona
0	1	Día festivo
1	2	
1	3	
0	4	Fin de semana
0	5	Fin de semana
0	6	Día festivo
1	7	
1	8	
1	9	
1	10	
0	11	Fin de semana
0	12	Fin de semana
1	13	

Campo de bits de TFACS	Día del mes	Motivo por el que no funciona
1	14	
1	15	
1	16	
1	17	
0	18	Fin de semana
0	19	Fin de semana
1	20	
1	21	
1	22	
1	23	
1	24	
0	25	Fin de semana
0	26	Fin de semana
1	27	
1	28	
1	29	
1	30	
1	31	

El siguiente ejemplo devuelve el valor 10.01.2014. A partir de este resultado puede ver que se ha añadido un solo día laborable a la fecha de inicio de la novena, siendo el resultado la décima.

```
SELECT ADD_WORKDAYS('01', '2014-01-09', 1, 'FCTEST') "result date" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 09.01.2014. A partir de este resultado se puede ver que la décima fue considerada como una jornada laboral, produciendo un resultado final de la novena.

```
SELECT ADD_WORKDAYS('01', '2014-01-10', -1, 'FCTEST') "result date" FROM DUMMY;
```

El siguiente ejemplo toma una entrada de día laborable positiva y devuelve el valor 20.01.2014, un resultado después de un período de fin de semana no laborable. A partir de este resultado puede ver que se ha añadido un solo día laborable a la fecha de inicio del día 17. Esto produjo el día resultante del 20, que permitió el período no laborable del fin de semana.

```
SELECT ADD_WORKDAYS('01', '2014-01-17', 1, 'FCTEST') "result date" FROM DUMMY;
```

El siguiente ejemplo toma una entrada de día laborable negativa y devuelve el valor 17.01.2014, que muestra un resultado después de un período de fin de semana no laborable. A partir de este resultado se puede ver que el día 20 se consideró como el día laborable, produciendo un resultado del día 17. Este resultado tiene en cuenta el horario no laborable del fin de semana.

```
SELECT ADD_WORKDAYS('01', '2014-01-20', -1, 'FCTEST') "result date" FROM DUMMY;
```

El siguiente ejemplo complejo devuelve el valor 27.01.2014. La sentencia anterior añade 16 días laborables a la fecha de inicio de la primera. El sistema tiene en cuenta los fines de semana (4, 5, 11, 12, 18 y 19) y los días festivos (1 y 6) durante el período laborable, lo que da el último día laborable como el 24. A continuación, el sistema devuelve el siguiente día laborable posible, que es el 27.

```
SELECT ADD_WORKDAYS('01', '2014-01-01', 16, 'FCTEST') "result date" FROM DUMMY;
```

El siguiente ejemplo utiliza una tabla para la entrada y devuelve los valores 4.02.2014, 14.05.2014, 05.08.2014 y 30.10.2014.

```
CREATE SCHEMA SAPABC;  
SET SCHEMA "SAPABC";  
CREATE ROW TABLE MY_DATES (FCID NVARCHAR(2), STARTDATE DATE, DURATION INTEGER);  
INSERT INTO MY_DATES VALUES ('01', '2014-01-01', 30);  
INSERT INTO MY_DATES VALUES ('01', '2014-04-01', 28);  
INSERT INTO MY_DATES VALUES ('01', '2014-07-01', 25);  
INSERT INTO MY_DATES VALUES ('01', '2014-10-01', 20);  
  
SELECT ADD_WORKDAYS(FCID, STARTDATE, DURATION) "shipment date" FROM MY_DATES;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

[Función WORKDAYS BETWEEN \(fecha y hora\)](#)

[Declaración ALTER USER \(control de acceso\)](#)

[Nota SAP 3396583](#) 

Función ADD_YEARS (fecha y hora)

Calcula la fecha especificada más los años especificados.

Sintaxis

```
ADD_YEARS(<date>, <num_years>)
```

Elementos de sintaxis

<date>

La fecha que se incrementa.

<num_years>

El número de años en los que se incrementa la fecha.

Descripción

Calcula la fecha especificada más el número de años especificado.

Ejemplo

El siguiente ejemplo incrementa el valor de fecha especificado 2009-12-05 en 1 año y devuelve el valor 2010-12-05:

```
SELECT ADD_YEARS (TO_DATE ( '2009-12-05', 'YYYY-MM-DD'), 1) "add years" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función ALLOW_PRECISION_LOSS (varios)

Permite la pérdida de precisión al agregar valores decimales mediante una expresión agregada.

Sintaxis

```
ALLOW_PRECISION_LOSS( <aggregate_expression> )
```

Elementos de sintaxis

<aggregate_expression>

La expresión agregada para la que se permite la pérdida de precisión. Actualmente, SUM es la única expresión agregada admitida.

```
<aggregate_expression> ::= SUM ( <expression> )
```

Descripción

Utilice esta función para mejorar el rendimiento de una expresión de agregación en valores DECIMAL cuando la pérdida de precisión es aceptable y para mejorar el rendimiento de la agregación.

Ejemplo

El siguiente ejemplo muestra la diferencia en los valores devueltos al utilizar la función ALLOW_PRECISION_LOSS:

```
CREATE TABLE TESTTABLE (COL1 decimal(10,5), COL2 decimal);  
INSERT INTO TESTTABLE VALUES(1.139999, 1.138888888);  
INSERT INTO TESTTABLE VALUES(2.119999, 2.118888888);  
INSERT INTO TESTTABLE VALUES(2.119999, 2.118888888);  
INSERT INTO TESTTABLE VALUES(2.669999, 2.668888888);  
  
-- The following query, which does not allow precision loss, returns 8.01, 8.01  
SELECT SUM(TO_DECIMAL(COL1,10,2)), SUM(TO_DECIMAL(COL2,10,2)) FROM TESTTABLE;  
  
-- The following query, which uses the ALLOW_PRECISION_LOSS function to allow precision loss, returns 8.01, 8.01  
SELECT ALLOW_PRECISION_LOSS(SUM(TO_DECIMAL(COL1,10,2))), ALLOW_PRECISION_LOSS(SUM(TO_DECIMAL(COL2,10,2))) FROM TESTTABLE;
```

Información relacionada

[Expresiones](#)

Función ASCII (cadena)

Devuelve el valor ASCII entero del primer carácter de una cadena especificada.

Sintaxis

`ASCII(<string>)`

Elementos de sintaxis

<string>

Especifica la cadena desde la que se debe devolver el valor entero ASCII.

Descripción

Devuelve el valor ASCII entero del primer carácter de una cadena *<string>*.

Ejemplo

Este ejemplo convierte el primer carácter de la cadena Ant en un valor ASCII numérico y devuelve el valor 65:

```
SELECT ASCII('Ant') "ascii" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función ASIN (numérica)

Devuelve el arco-seno, en radianes, de un argumento numérico.

Sintaxis

`ASIN(<number>)`

Elementos de sintaxis

<number>

Especifica un argumento numérico entre -1 y 1.

Descripción

Devuelve el arco-seno, en radianes, del argumento numérico *<número>* entre -1 y 1.

Ejemplo

El siguiente ejemplo devuelve el valor 0,5235987755982989 para "asin":

```
SELECT ASIN (0.5) "asin" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función ATAN (numérica)

Devuelve el arco-tangente, en radianes, de un argumento numérico.

Sintaxis

ATAN(<number>)

Elementos de sintaxis

<number>

Especifica un argumento numérico.

Descripción

Devuelve el arco-tangente, en radianes, del argumento numérico <número>. El rango de <número> es ilimitado.

Ejemplo

El siguiente ejemplo devuelve el valor 0,4636476090008061 para "atan":

```
SELECT ATAN (0.5) "atan" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función ATAN2 (numérica)

Devuelve el arco tangente, en radianes, de la relación de dos números.

Sintaxis

ATAN2(<number1>, <number2>)

Elementos de sintaxis

<number1>

Especifica el primer argumento numérico.

<number2>

Especifica el segundo argumento numérico.

Descripción

Devuelve el arco tangente, en radianes, de la relación de dos números *<number1>* y *<number2>*.

Ejemplo

El siguiente ejemplo devuelve el valor 0,4636476090008061 para "atan2":

```
SELECT ATAN2 (1.0, 2.0) "atan2" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función AUTO_CORR (agregar)

Calcula todos los coeficientes de autocorrelación para una expresión de entrada determinada y devuelve una matriz de valores.

Sintaxis

```
AUTO_CORR( <expression>, <maxTimeLag> { <series_order_by_clause> | <order_by_clause> } )
```

Elementos de sintaxis

<expression>

Especifica la expresión de entrada. Los valores *<expresión>* pueden ser de cualquier tipo numérico.

<maxTimeLag>

El tamaño del período de tiempo está limitado por el parámetro maxTimeLag. Este parámetro debe ser un número entero positivo. El tamaño del resultado es el mínimo de maxTimeLag y el tamaño de columna - 2 para los datos de serie densos.

<series_orderby>

<series_orderby> solo se puede utilizar con una serie equidistante.

```
<series_orderby> ::= SERIES( <series_period> <series_equidistant_definition> )
```

No se admite el uso de variables SQLScript en esta cláusula. La serie * no debe contener elementos que faltan. Para obtener más información sobre esta cláusula, consulte la función SERIES_GENERATE.

<order_by_clause>

Especifica el orden de clasificación de las filas de entrada.

```
<order_by_clause> ::= ORDER BY <order_by_expression> [, <order_by_expression> [...]]
```

```
<order_by_expression> ::=
  <column_name> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
  | <column_position> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
```

```
<collate_clause> ::= COLLATE <collation_name>
```

<clausula colateral> especifica la intercalación que se va a utilizar para ordenar los valores en los resultados. *<collate_cláusula>* solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un sinónimo de NVARCHAR).

<collation_name> es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

Descripción

Calcula todos los coeficientes de autocorrelación para una expresión de entrada determinada y devuelve una matriz de valores.

Se eliminan los pares que contienen al menos un valor nulo. Aunque AUTO_CORR puede gestionar valores de entrada nulos, se recomienda encarecidamente sustituir primero los valores nulos (por ejemplo, utilizando LINEAR_APPROX), lo que permite un procesamiento mucho más rápido.

La salida está vacía si hay menos de dos filas.

Ejemplos

El siguiente ejemplo muestra la autocorrelación de datos de series densas y devuelve

[0,285714, -0,351351, -0,5625, -0,25,1,1,1,1].

```
CREATE COLUMN TABLE correlationTable (TS_ID NVARCHAR(10), DATE DAYDATE, VALUE DOUBLE);
INSERT INTO correlationTable VALUES ('A', '2014-10-01', 1);
INSERT INTO correlationTable VALUES ('A', '2014-10-02', 2);
INSERT INTO correlationTable VALUES ('A', '2014-10-03', 3);
INSERT INTO correlationTable VALUES ('A', '2014-10-04', 4);
INSERT INTO correlationTable VALUES ('A', '2014-10-05', 5);
INSERT INTO correlationTable VALUES ('A', '2014-10-06', 1);
INSERT INTO correlationTable VALUES ('A', '2014-10-07', 2);
INSERT INTO correlationTable VALUES ('A', '2014-10-08', 3);
INSERT INTO correlationTable VALUES ('A', '2014-10-09', 4);
INSERT INTO correlationTable VALUES ('A', '2014-10-10', 5);

SELECT TS_ID, AUTO_CORR(VALUE, 8 SERIES (PERIOD FOR SERIES(DATE)
    EQUIDISTANT INCREMENT BY INTERVAL 1 DAY MISSING ELEMENTS NOT ALLOWED))
FROM correlationTable
GROUP BY TS_ID ORDER BY TS_ID;
```

El siguiente ejemplo muestra la autocorrelación de datos de series escasas sin tener en cuenta las entradas que faltan y devuelve

[1,1,1,1,1].

```
CREATE COLUMN TABLE correlationTable (ts_id NVARCHAR(20), date DAYDATE, val DOUBLE);
INSERT INTO correlationTable VALUES ('A', '2014-10-01', 1);
INSERT INTO correlationTable VALUES ('A', '2014-10-02', 2);
INSERT INTO correlationTable VALUES ('A', '2014-10-04', 3);
INSERT INTO correlationTable VALUES ('A', '2014-10-07', 4);
INSERT INTO correlationTable VALUES ('A', '2014-10-11', 5);
INSERT INTO correlationTable VALUES ('A', '2014-10-21', 6);
INSERT INTO correlationTable VALUES ('A', '2014-10-22', 7);

SELECT ts_id, AUTO_CORR(val, 999 SERIES (PERIOD FOR SERIES(DATE)
    EQUIDISTANT INCREMENT BY INTERVAL 1 DAY MISSING ELEMENTS NOT ALLOWED))
FROM correlationTable
GROUP BY ts_id ORDER BY ts_id;
```

A la correlationTable le faltan entradas, como '2014-10-03', pero la cláusula WITHIN GROUP considera que la serie es equidistante con intervalos de un día, donde los elementos que faltan no están permitidos.

Dado que se supone que los datos de la serie son densos, se calcula la autocorrelación del conjunto de datos [1..7].

El siguiente ejemplo muestra la autocorrelación de datos de series dispersas teniendo en cuenta las entradas que faltan y devuelve

```
[1.0, null, 1.0, null, null, null, null, null, * 1.0, null, null, null, null, null, null, null, 1.0]
```

```
.
```

```
CREATE COLUMN TABLE correlationTable (ts_id NVARCHAR(20), date DAYDATE, val DOUBLE);
INSERT INTO correlationTable VALUES ('A', '2014-10-01', 1);
INSERT INTO correlationTable VALUES ('A', '2014-10-02', 2);
INSERT INTO correlationTable VALUES ('A', '2014-10-04', 3);
INSERT INTO correlationTable VALUES ('A', '2014-10-07', 4);
INSERT INTO correlationTable VALUES ('A', '2014-10-11', 5);
INSERT INTO correlationTable VALUES ('A', '2014-10-21', 6);
INSERT INTO correlationTable VALUES ('A', '2014-10-22', 7);

SELECT ts_id, AUTO_CORR(val, 999 SERIES (PERIOD FOR SERIES(DATE)
    EQUIDISTANT INCREMENT BY INTERVAL 1 DAY MISSING ELEMENTS ALLOWED))
FROM correlationTable
GROUP BY ts_id ORDER BY ts_id;
```

La autocorrelación funciona como si hubiera valores nulos en lugar de los elementos que faltan en la columna.

Información relacionada

[Funciones de ventana y la especificación de ventana](#)

[Funciones agregadas de ventana](#)

[Agregar funciones](#)

[Función CORR \(agregar\)](#)

[Función CORR_SPEARMAN \(agregado\)](#)

[Función CROSS_CORR \(agregar\)](#)

[Expresiones](#)

Función AVG (agregar)

Devuelve la media aritmética de la expresión. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
AVG( [ ALL | DISTINCT ] <expression> )
```

Función de ventana:

```
AVG( <expression> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica los datos de entrada para la función.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Tipo de resultado basado en entrada

TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL (p, s)	DECIMAL	REAL	DOUBLE
DECIMAL (9, 6)	DECIMAL (11, 6)	DECIMAL (16, 6)	DECIMAL (25, 6)	DECIMAL (p, s)	DECIMAL	REAL	DOUBLE

Ejemplo

Las siguientes sentencias crean una tabla con datos, por ejemplo:

```
DROP TABLE "MyProducts";
CREATE COLUMN TABLE "MyProducts" (
  "Product_ID" NVARCHAR(10),
  "Product_Name" NVARCHAR(100),
  "Category" NVARCHAR(100),
  "Quantity" INTEGER,
  "Price" DECIMAL(10,2),
  PRIMARY KEY ("Product_ID") );

INSERT INTO "MyProducts" VALUES('P1','Shirts', 'Clothes', 32,20.99);
INSERT INTO "MyProducts" VALUES('P2','Jackets', 'Clothes', 16, 99.49);
INSERT INTO "MyProducts" VALUES('P3','Trousers', 'Clothes', 30, 32.99);
INSERT INTO "MyProducts" VALUES('P4','Coats', 'Clothes', 5, 129.99);
INSERT INTO "MyProducts" VALUES('P5','Purse', 'Accessories', 3, 89.49);
```

El siguiente ejemplo promedia los precios de todos los productos en la tabla MyProducts y devuelve el valor 74,59.:

```
SELECT AVG("Price") FROM "MyProducts";
```

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Agregar funciones](#)
- [Expresiones](#)

Función BINNING (ventana)

Divide un conjunto de entrada en subconjuntos separados asignando un número de ubicación a cada fila.

Sintaxis

```
BINNING( <binning_param> => <expression> [ {, <binning_parameter> => <expression> } ... ] ) <window>
<binning_param> ::= VALUE | BIN_COUNT | BIN_WIDTH | TILE_COUNT | STDDEV_COUNT
```

Elementos de sintaxis

<binning_param>

VALUE siempre es obligatorio. Especifica la columna a la que se aplica la clasificación. Cuando se utiliza BIN_WIDTH, la columna de entrada debe tener un tipo de datos numérico. BIN_COUNT especifica el número de ubicaciones de igual ancho. BIN_WIDTH especifica el ancho de las ubicaciones. TILE_COUNT especifica el número de ubicaciones con el mismo número de registros. STDDEV_COUNT especifica el número de desviaciones estándar a la izquierda y a la derecha de la media.

El método de clasificación adecuado se selecciona en función del parámetro especificado; exactamente uno de los últimos cuatro parámetros no debe ser NULL. El valor asignado al parámetro de método de agrupación debe ser una expresión de entero.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La cláusula OVER no debe especificar una <window_order_by_class> ni ningún marco de ventana, ya que la función de agrupamiento funciona en una partición completa.

Ejemplos

El siguiente ejemplo distribuye el valor del conjunto de entradas en cuatro ubicaciones que tienen el mismo ancho.

```
DROP TABLE weather;
CREATE ROW TABLE weather (station INT, ts DATE, temperature FLOAT);
INSERT INTO weather VALUES(1, '2014-01-01', 0);
INSERT INTO weather VALUES(1, '2014-01-02', 3);
INSERT INTO weather VALUES(1, '2014-01-03', 4.5);
INSERT INTO weather VALUES(1, '2014-01-04', 6);
INSERT INTO weather VALUES(1, '2014-01-05', 6.3);
INSERT INTO weather VALUES(1, '2014-01-06', 5.9);
INSERT INTO weather VALUES(1, '2015-01-01', 1);
INSERT INTO weather VALUES(1, '2015-01-02', 3.4);
INSERT INTO weather VALUES(1, '2015-01-03', 5);
INSERT INTO weather VALUES(1, '2015-01-04', 6.7);
INSERT INTO weather VALUES(1, '2015-01-05', 4.6);
INSERT INTO weather VALUES(1, '2015-01-06', 6.9);

SELECT *, BINNING(VALUE => temperature, BIN_COUNT => 4) OVER () AS bin_num FROM weather;
```

ESTACIÓN	SC	TEMPERATURA	BIN_NUM
1	2014-01-01	0	1
1	02-01-2014	3	2
1	2014-01-03	4.5	3
1	2014-01-04	6	4
1	2014-01-05	6.3	4
1	2014-01-06	5.9	4
1	01/01/2015	1	1
1	2015-01-02	3.4	2
1	2015-01-03	5	3
1	2015-01-04	6.7	4

ESTACIÓN	SC	TEMPERATURA	BIN_NUM
1	05/01/2015	4.6	3
1	2015-01-06	6.9	4

Información relacionada

[Expresiones](#)
[Funciones de ventana y la especificación de ventana](#)

Función BINTOHEX (cadena)

Convierte una cadena binaria en un valor hexadecimal NVARCHAR.

Sintaxis

```
BINTOHEX(<expression>)
```

Elementos de sintaxis

<expression>
El valor que se va a convertir en un valor hexadecimal.

Descripción

Convierte un valor binario en un valor hexadecimal como un tipo de datos NVARCHAR. Si el valor de entrada no es un valor binario, primero se convierte en un valor binario.

Ejemplo

El siguiente ejemplo convierte el valor binario AB en un valor NVARCHAR hexadecimal 4142:

```
SELECT BINTOHEX('AB') "bintohehex" FROM DUMMY;
```

Información relacionada

[Expresiones](#)
[Tipos de datos de cadena de caracteres](#)

Función BINTONHEX (cadena)

Convierte un valor binario en un valor hexadecimal como un tipo de datos NVARCHAR.

Sintaxis

```
BINTONHEX(<expression>)
```

Elementos de sintaxis

<expression>

El valor que se va a convertir en un valor hexadecimal.

Descripción

Convierte un valor binario en un valor hexadecimal como un tipo de datos NVARCHAR. El valor de entrada se convierte primero en un valor binario si no es un valor binario.

Ejemplo

El siguiente ejemplo convierte el valor binario AB en un valor NVARCHAR hexadecimal 4142.

```
SELECT BINTONHEX('AB') "bintonhex" FROM DUMMY;
```

Información relacionada

[Expresiones](#)

[Tipos de datos de cadena de caracteres](#)

Función BINTOSTR (cadena)

Convierte una cadena VARBINARY en una cadena de caracteres.

Sintaxis

BINTOSTR(<varbinary_string>)

Elementos de sintaxis

<varbinary_string>

La cadena VARBINARY que se va a convertir en una cadena de caracteres.

Descripción

Convierte una cadena VARBINARY <varbinary_string> en una cadena de caracteres con codificación CESU-8.

Ejemplo

Este ejemplo convierte la cadena VARBINARY 416E74 en una cadena de caracteres codificada CESU-8 y devuelve el valor Ant:

```
SELECT BINTOSTR ('416E74') "bintostr" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función BITAND (numérica)

Realiza una operación AND en los bits de dos argumentos.

Sintaxis

BITAND(<value1>, <value2>)

Elementos de sintaxis

<value1>

El argumento debe ser un valor INTEGER o VARBINARY.

<value2>

El argumento debe ser un valor INTEGER o VARBINARY.

Descripción

Realiza una operación AND en los bits de los argumentos <value1> y <value2>.

La función BITAND funciona un poco diferente de las funciones BITOR, BITXOR y BITNOT. La función BITAND convierte argumentos de cadena en BIGINT, mientras que las otras funciones convierten argumentos de cadena en INT.

La función BITAND devuelve un resultado junto con el tipo del argumento.

Ejemplo

El siguiente ejemplo devuelve el valor 123:

```
SELECT BITAND (255, 123) "bitand" FROM DUMMY;
```

Información relacionada

[Expresiones](#)

[Tipos de datos numéricos](#)

Función BITCOUNT (numérica)

Cuenta el número de bits establecidos de una expresión.

Sintaxis

BITCOUNT(<expression>)

Elementos de sintaxis

<expression>

Especifica la expresión de la que cuenta la función el número de bits establecidos de. *<expresión>* debe ser un valor INTEGER o VARBINARY.

Descripción

Cuenta el número de bits establecidos del argumento *<expresión>*.

La función BITCOUNT devuelve un valor INTEGER.

Ejemplo

El siguiente ejemplo cuenta los bits para 255 y devuelve el valor 8:

```
SELECT BITCOUNT (255) "bitcount" FROM DUMMY;
```

Información relacionada

[Expresiones](#)

[Tipos de datos numéricos](#)

Función BITNOT (numérica)

Realiza una operación NOT por bits en los bits de una expresión.

Sintaxis

BITNOT(*<expression>*)

Elementos de sintaxis

<expression>

Especifica la expresión en la que la función realiza una operación NOT por bits. *<expresión>* debe ser un valor INTEGER.

Descripción

Realiza una operación NOT por bits en los bits del argumento *<expresión>*.

Las funciones BITNOT funcionan un poco diferente a la función BITAND. La función BITAND convierte argumentos de cadena en BIGINT, mientras que la función BITNOT convierte argumentos de cadena en INT.

La función BITNOT devuelve un resultado junto con el tipo de argumento.

Ejemplo

El siguiente ejemplo realiza una operación BITNOT en 255 y devuelve el valor -256 para "bitnot":

```
SELECT BITNOT (255) "bitnot" FROM DUMMY;
```

Información relacionada

[Expresiones](#)

[Tipos de datos numéricos](#)

Función BITOR (numérica)

Esta función realiza una operación OR en los bits de dos argumentos.

Sintaxis

BITOR(<expression1>, <expression2>)

Elementos de sintaxis

<expression1>

Un argumento para la operación OR por bits que debe ser un valor INTEGER o VARBINARY no negativo.

<expression2>

Un argumento para la operación OR por bits que debe ser un valor INTEGER o VARBINARY no negativo.

Descripción

Esta función realiza una operación OR en los bits de los argumentos <expresión1> y <expresión2>.

Las funciones BITOR funcionan un poco diferente que la función BITAND. La función BITAND convierte argumentos de cadena en BIGINT, mientras que la función BITOR convierte argumentos de cadena en INT.

La función BITOR devuelve un resultado junto con el tipo del argumento.

Ejemplo

El siguiente ejemplo realiza una operación OR por bits para los argumentos 255 y 123, y devuelve el valor 255 para "bitor":

```
SELECT BITOR (255, 123) "bitor" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función BITSET (numérica)

Establece un número específico de bits en 1 en un número de destino a partir de una posición de índice basada en 1 especificada.

Sintaxis

BITSET(<target_num>, <start_bit>, <num_to_set>)

Elementos de sintaxis

<target_num>

El número VARBINARY donde se deben establecer los bits.

<target_num> ::= <string_literal>

<start_bit>

Una posición de índice basada en 1 en la que se va a establecer el primer bit.

<start_bit> ::= <unsigned_integer>

<num_to_set>

El número de bits a establecer en el número objetivo.

<num_to_set> ::= <unsigned_integer>

Descripción

Establece <num_to_set> bits en 1 en <target_num> desde la posición <start_bit>.

Ejemplo

El siguiente ejemplo devuelve el valor E000 para "conjunto de bits":

```
SELECT BITSET ('0000', 1, 3) "bitset" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función BITUNSET (numérica)

Establece un número especificado de bits en 0 en un número objetivo a partir de una posición de índice basada en 1 especificada.

Sintaxis

BITUNSET(<target_num>, <start_bit>, <num_to_unset>)

Elementos de sintaxis

<target_num>

El número VARBINARY en el que se van a desestablecer los bits.

<target_num> ::= <string_literal>

<start_bit>

Una posición de índice basada en 1 en la que se va a desestablecer el primer bit.

<start_bit> ::= <unsigned_integer>

<num_to_set>

El número de bits a desestablecer en el número objetivo.

<num_to_set> ::= <unsigned_integer>

Descripción

Establece <num_to_unset> bits en 0 en <target_num> desde la posición <start_bit>.

Ejemplo

El siguiente ejemplo devuelve el valor 1FFF para "bitunset":

```
SELECT BITUNSET ('ffff', 1, 3) "bitunset" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función BITXOR (numérica)

Realiza una operación XOR en los bits de dos argumentos.

Sintaxis

BITXOR(<expression1>, <expression2>)

Elementos de sintaxis

<expression1>

Un argumento que debe ser un valor INTEGER o VARBINARY.

<expression2>

Un argumento para la operación OR por bits que debe ser un valor INTEGER o VARBINARY.

Descripción

Realiza una operación XOR en los bits de los argumentos <expresión1> y <expresión2>.

Las funciones BITXOR funcionan un poco diferente que la función BITAND. La función BITAND convierte argumentos de cadena en BIGINT, mientras que la función BITXOR convierte argumentos de cadena en INT.

La función BITXOR devuelve un resultado junto con el tipo del argumento.

Ejemplo

El siguiente ejemplo realiza una operación XOR por bits para los argumentos 255 y 123, y devuelve el valor 132 para "bitxor":

```
SELECT BITXOR (255, 123) "bitxor" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función CARDINALITY (matriz)

Devuelve el número de elementos en una matriz especificada.

Sintaxis

```
CARDINALITY(<array_value_expression>)
```

Elementos de sintaxis

<array_value_expression>

Especifica la matriz para la que la función devuelve el número de elementos.

Descripción

Devuelve el número de elementos en *<array_value_expression>*.

Ejemplo

El siguiente ejemplo devuelve el número de elementos (3 y 4, respectivamente) contenidos en dos matrices.

```
CREATE COLUMN TABLE ARRAY_TEST (IDX INT, VAL INT ARRAY);  
INSERT INTO ARRAY_TEST VALUES (1, ARRAY(1, 2, 3));  
INSERT INTO ARRAY_TEST VALUES (2, ARRAY(10, 20, 30, 40));  
  
SELECT CARDINALITY(VAL) "cardinality" FROM ARRAY_TEST;
```

Información relacionada

[Expresiones](#)

Función CARDINALITY (vector)

Devuelve la dimensión de un vector.

Sintaxis

```
CARDINALITY(<vector>)
```

Elementos de sintaxis

<vector>

Especifica el vector para el que la función devuelve la dimensión.

Descripción

Devuelve la dimensión de <vector>.

Ejemplos

Los siguientes ejemplos devuelven 3.

```
SELECT CARDINALITY(TO_REAL_VECTOR('[2,3,5]')) "cardinality" FROM DUMMY;
SELECT CARDINALITY(TO_HALF_VECTOR('[2,3,5]')) "cardinality" FROM DUMMY;
```

Función CAST (conversión de tipo de datos)

Devuelve el valor de una expresión convertida en un tipo de datos proporcionado.

Sintaxis

```
CAST( <expression> AS <data_type>[ ( <length> ) ] )
```

Elementos de sintaxis

<expression>

Especifica la expresión que se va a convertir. Si utiliza un parámetro, el parámetro está vinculado como <data_type> o como una cadena si <data_type> es LOB o GEOMETRY.

<data_type>

Especifica el tipo de datos de destino.

```
<data_type >:::=
BOOLEAN
| TINYINT
| SMALLINT
| INTEGER
| BIGINT
| DECIMAL
| SMALLDECIMAL
| REAL
| DOUBLE
| NVARCHAR
| DAYDATE
| DATE
| TIME
| SECONDDATE
| TIMESTAMP
| LOB
| BINARY
| GEOMETRY
| REAL_VECTOR
```

<length>

Si el tipo de destino es una cadena, **<length>** especifica la longitud máxima de la cadena en caracteres. Si el tipo de destino es un vector, **<longitud>** especifica la dimensión del vector.

Descripción

Devuelve el valor de una expresión convertida en un tipo de datos proporcionado.

Ejemplos

Convierta el valor 7 en el valor NVARCHAR 7:

```
SELECT CAST (7 AS NVARCHAR) "cast" FROM DUMMY;
```

Convierte el valor 10,5 en el valor INTEGER 10, que trunca la mantisa.

```
SELECT CAST (10.5 AS INTEGER) "cast" FROM DUMMY;
```

Vincule un parámetro como BIGINT. Sin la función CAST, el tipo de datos del parámetro es ambiguo.

```
CREATE COLLECTION C1;  
INSERT INTO C1 VALUES({A:1,B:2});  
INSERT INTO C1 VALUES({A:'ABC'});  
SELECT * FROM C1 WHERE A = CAST (? AS BIGINT);
```

Información relacionada

[Expresiones](#)

[Conversión de tipo de datos](#)

Función CEIL (numérica)

Devuelve el número decimal más pequeño igual o mayor que el valor especificado.

Sintaxis

CEIL(**<number>**)

Elementos de sintaxis

<number>

Especifica el número para el que la función devuelve el valor tope.

Descripción

Devuelve el número decimal más pequeño mayor o igual que el **<número>** especificado.

Ejemplo

El siguiente ejemplo devuelve el valor 15.0 para "techo":

```
SELECT CEIL (14.5) "ceiling" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función CHAR (cadena)

Devuelve el carácter que tiene el valor ASCII del número especificado.

Sintaxis

```
CHAR(<number>)
```

Elementos de sintaxis

<number>

Indica el número con el valor ASCII que la función convierte en un carácter.

Descripción

Devuelve el carácter que tiene el valor ASCII del número especificado.

Ejemplo

Este ejemplo convierte tres valores ASCII en caracteres y concatena los resultados, devolviendo la cadena Ant:

```
SELECT CHAR (65) || CHAR (110) || CHAR (116) "character" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función COALESCE (varios)

Devuelve la primera expresión no NULL de una lista especificada.

Sintaxis

```
COALESCE(<expression_list>)
```

Elementos de sintaxis

<expression_list>

Descripción

Devuelve la primera expresión no NULL de una lista. Al menos dos expresiones deben estar incluidas en *<expression_list>* y todas las expresiones deben ser comparables. El resultado es NULL si todas las expresiones son NULL.

Ejemplo

```
CREATE ROW TABLE coalesce_example (ID INT PRIMARY KEY, A REAL, B REAL);
INSERT INTO coalesce_example VALUES(1, 100, 80);
INSERT INTO coalesce_example VALUES(2, NULL, 63);
INSERT INTO coalesce_example VALUES(3, NULL, NULL);

SELECT id, a, b, COALESCE (a, b*1.1, 50.0) "coalesce" FROM coalesce_example;
```

El ejemplo anterior devuelve los resultados siguientes:

ID	A	B	fusionar
1	100.0	80,0	100.0
2	NULO	63.0	69.30000305175781
3	NULO	NULO	50,0

Información relacionada

[Expresiones](#)

Función CONCAT (string)

Devuelve una cadena combinada que consta de dos cadenas especificadas.

Sintaxis

```
CONCAT(<string1>, <string2>)
```

Elementos de sintaxis

<string1>

Especifica la primera cadena que se va a combinar.

<string2>

Especifica la segunda cadena que se va a combinar.

Descripción

Devuelve una cadena combinada que consta de *<cadena1>* seguida de *<cadena2>*. El operador de concatenación (||) es idéntico a esta función.

La longitud máxima de la cadena concatenada es 8.388.607. Si la longitud de la cadena es mayor que la longitud máxima, se genera una excepción. Excepcionalmente, se realiza un truncamiento implícito al convertir un valor tipificado NCLOB con un tamaño mayor que la longitud máxima de un valor NVARCHAR.

Si uno o ambos argumentos son NULL, la función devuelve NULL.

Ejemplo

Este ejemplo concatena los argumentos de cadena especificados y devuelve el valor Cat:

```
SELECT CONCAT ( 'C' , 'at' ) "concat" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función CONCAT_NAZ (cadena)

Devuelve una cadena combinada de valor no nulo que consta de dos cadenas especificadas.

Sintaxis

```
CONCAT_NAZ(<string1>, <string2>)
```

Elementos de sintaxis

<string1>

Especifica la primera cadena que se va a combinar.

<string2>

Especifica la segunda cadena que se va a combinar.

Descripción

Devuelve una cadena de valor combinado que consta de <cadena1> seguido de <cadena2>. Si un valor es NULO, se devuelve el otro valor. Si ambos valores son NULL, se devuelve NULL.

La longitud máxima de la cadena concatenada es 8.388.607. Si la longitud de la cadena es mayor que la longitud máxima, se genera una excepción. Excepcionalmente, se realiza un truncamiento implícito al convertir un valor tipificado NCLOB con un tamaño mayor que la longitud máxima de un valor tipificado NVARCHAR.

Ejemplo

El siguiente ejemplo devuelve AB:

```
SELECT CONCAT_NAZ ( 'A' , 'B' ) "concat" FROM DUMMY;
```

concat
AB

El siguiente ejemplo devuelve C:

```
SELECT CONCAT_NAZ ('C', null) "concat" FROM DUMMY;
```

concat
C

El siguiente ejemplo devuelve el valor NULL:

```
SELECT CONCAT_NAZ (null, null) "concat" FROM DUMMY;
```

concat
NULO

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función CONVERT_CURRENCY (varios)

Calcula valores en diferentes monedas.

Sintaxis

```
CONVERT_CURRENCY( <named_parameter_value>[{, <named_parameter_value>}...])
```

Elementos de sintaxis

<named_parameter_value>

Especifica los parámetros para la conversión.

```
<named_parameter_value> ::=
"<field_reference_parameter>" => <expression>
| "<const_string_parameter>" => <const_string>
```

```
<field_reference_parameter> ::=
AMOUNT
| SOURCE_UNIT
| TARGET_UNIT
| REFERENCE_DATE
| CLIENT
```

Los parámetros de referencia de campo se utilizan para hacer referencia a columnas de tabla para la conversión.

Nombre de parámetro de referencia de campo	Descripción	Obligatorio	Valor por defecto
IMPORTE	El identificador de columna que contiene los valores que se van a convertir.	Sí	Ninguno
CLIENTE	Define una cadena de tres caracteres que se utiliza para separar arrendatarios dentro de las tablas del sistema ERP. Esto se utiliza en las tablas de conversión para seleccionar las filas correctas para cada usuario. Un identificador de columna también se acepta mediante comillas dobles. Este parámetro es obligatorio, ya que este comando no utiliza la variable de contexto de sesión CLIENT.	Sí	Ninguno
SOURCE_UNIT	El identificador de columna que describe la unidad de entrada. Una cadena constante también se acepta mediante comillas simples.	Sí	Ninguno
TARGET_UNIT	El identificador de columna que describe la unidad de destino. Una cadena constante también se acepta mediante comillas simples.	Sí	Ninguno
REFERENCE_DATE	El identificador de columna que describe la fecha de referencia de moneda. Una cadena constante también se acepta mediante comillas simples.	Sí	Ninguno

```
<const_string_parameter> ::=
  SCHEMA
| DATABASE
| CONVERSION_TYPE
| LOOKUP
| ERROR_HANDLING
| ACCURACY
| DATE_FORMAT
| STEPS
| CONFIGURATION_TABLE
| PRECISIONS_TABLE
| NOTATIONS_TABLE
| RATES_TABLE
| PREFACTORS_TABLE
```

<const_string_parameter> define los valores de parámetro constantes utilizados en la conversión.

Nombre de parámetro constante	Descripción	Obligatorio	Valor por defecto
ESQUEMA	Define el esquema que contiene las tablas de conversión utilizadas para la conversión.	Sí	Ninguno
BASE DE DATOS	Define el arrendatario en el que se encuentran las tablas de conversión (por ejemplo, las tablas TCUR* de la conversión de moneda ERP). Si no se especifica, la conversión utiliza la base de datos actual.	No	Ninguno
MÉTODO	Valores posibles: ERP, Banca	No	ERP
BID_ASK_TYPE	Valores posibles: puja, pregunta, medio	No	(cadena vacía)
MARKET_DATA_AREA	Define el área de datos de mercado tal y como se almacena en las tablas. Esto es obligatorio para la conversión de moneda bancaria.	No	(cadena_vacía)
SYSTEM_TIME	Define el cronomarcador del sistema para la funcionalidad de viajes en el tiempo.	No	Cronomarcador de sistema actual en GMT
CONVERSION_TYPE	Define el tipo de conversión como almacenado en las tablas de conversión. Los tipos de conversión disponibles en su sistema varían según la configuración de su sistema ERP. En general, son M o EURX. Póngase en contacto con el administrador del sistema para obtener los detalles de su configuración de tabla específica.	No	M
LOOKUP	El tipo de búsqueda que se debe realizar. Valores posibles: Normal Se realiza una conversión regular. Anular Realiza una conversión inversa con las unidades de entrada intercambiadas.	No	Normal
ERROR_HANDLING	Define cómo trata el sistema una situación en la que no se ha	No	Fallo en caso de error

Nombre de parámetro constante	Descripción	Obligatorio	Valor por defecto
	podido convertir una fila. Valores posibles: fallo en caso de error La conversión falla con un error. definir como nulo La salida de la fila que ha causado el error se fija en NULL. mantener sin convertir Se devuelve el valor de entrada.		
PRECISIÓN	Define el comportamiento de redondeo del sistema. Valores posibles: compatibilidad Comportamiento de ERP de Mimics redondeando el resultado. más alto Mantiene tantos dígitos como sea posible en el resultado.	No	Compatibilidad
DATE_FORMAT	Solo para uso interno	n/d	n/d
PASOS	Define los pasos que se deben incluir en el proceso de conversión. Proporcione una lista delimitada por comas de los pasos que se incluirán y el orden de los pasos es irrelevante. Mayús Habilita un desplazamiento decimal según la moneda de origen seleccionada. Por ejemplo, si la moneda de origen tiene 0 dígitos válidos según PRECISIONS_TABLE, cada valor debe multiplicarse por 100 porque en los sistemas SAP ERP, los valores se almacenan con dos dígitos. Esto debe hacerse para convertir los valores ERP a su representación numérica correcta. convertir	No	cambiar, convertir

Nombre de parámetro constante	Descripción	Obligatorio	Valor por defecto
	Desencadena la conversión real de la moneda de origen a la moneda de destino. redondear Redondea el valor convertido al número de dígitos de la moneda de destino. Utilice este paso con cuidado si se realizan agregaciones posteriores en el número, ya que se podrían acumular errores de redondeo. shift_back Mientras que el desplazamiento modifica los decimales de dos a la precisión configurada de la moneda de origen, shift_back los modifica de nuevo a dos, pero de la moneda de destino. Si el tratamiento de errores está fijado para mantener sin convertir, la moneda de salida puede ser la moneda de origen en lugar de la moneda de destino. En caso de error, el redondeo y el shift_back se realizan con respecto a la moneda de origen y la conversión se elimina. Esto hace que todos los pasos sean redundantes y devuelve el valor de entrada de nuevo.		
CONFIGURATION_TABLE	El identificador de tabla de la configuración del tipo de conversión. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	No	TCURV
PRECISIONS_TABLE	El identificador de tabla de la tabla de precisión. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	No	TCURX
NOTATIONS_TABLE	El identificador de tabla de la tabla que almacena notaciones. Si la tabla reside en un arrendatario diferente, proporcione un nombre	No	TCURN

Nombre de parámetro constante	Descripción	Obligatorio	Valor por defecto
	completamente cualificado para la tabla o utilice el parámetro DATABASE.		

Parámetros y valores predeterminados adicionales para el método ERP

Nombre de parámetro constante	Descripción	Obligatorio	Valor por defecto
CONFIGURATION_TABLE	El identificador de tabla de la configuración del tipo de conversión. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	Sin TCURV	TCURV
RATES_TABLE	El identificador de tabla de la tabla de tasas de conversión. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	Sin TCURR	TCURR
PREFACTORS_TABLE	El identificador de tabla de la tabla de factores previos. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	Sin TCURF	TCURF

Parámetros y valores por defecto adicionales para el método Banking

Nombre de parámetro constante	Descripción	Obligatorio	Valor por defecto
MARKET_DATA_AREA	Define el área de datos de mercado tal y como se almacena en las tablas. Esto es obligatorio para la conversión de moneda bancaria.	Sí	(cadena_vacía)
SYSTEM_TIME	Define el cronomarcador del sistema para la funcionalidad de viajes en el tiempo. T	No	Cronomarcador sistema actual en MM
CONFIGURATION_TABLE	El identificador de tabla de la configuración del tipo de conversión. Si la tabla reside en un arrendatario diferente, proporcione un nombre	No	/BA1/TF4_FXRTTYP

Nombre de parámetro constante	Descripción	Obligatorio	Valor por defecto
	completamente cualificado para la tabla o utilice el parámetro DATABASE.		
RATES_TABLE	El identificador de tabla de la tabla de tasas de conversión. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	No	/BA1/F4_FXRATES
PREFACTORS_TABLE	El identificador de tabla de la tabla de factores previos. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	No	/BA1/TF4_FXCVFCT

Descripción

La función CONVERT_CURRENCY proporciona un método eficiente para calcular valores en diferentes monedas.

La conversión de moneda HANA proporciona dos métodos diferentes. El método ERP, que almacena los tipos cruzados en la tabla TCURR con 9 dígitos, y el método Banking, que almacena los tipos de cambio en la tabla /BA1/F4_FXRATES con 15 dígitos. En ambos casos, también puede utilizar tablas que almacenen los tipos cruzados en un tipo de datos con más dígitos, como DECIMAL(25,21).

Para utilizar la función CONVERT_CURRENCY, las tablas de conversión de moneda TCURV, TCURX, TCURN, TCURR y TCURF deben estar disponibles en la base de datos SAP HANA para el método ERP. Como alternativa, las tablas de conversión de moneda /BA1/TF4_FXRTTYP, TCURX, TCURN, /BA1/F4_FXRATES y /BA1/TF4_FXCVFCT deben estar disponibles para el método Banking. Si ha almacenado la información de tasas en tablas con una disposición diferente, puede utilizar vistas para asignar el contenido de la tabla al formato utilizado en las tablas ERP o Banking.

La función CONVERT_CURRENCY no admite algunas funciones de conversión. Solo admite las columnas XINVR, BWAER, XBWRL y XEURO de la tabla Customizing TCURV. Aparece un mensaje de error cuando uno de los campos no admitidos BKUZU, GKUZU o XBWRL se actualiza en la tabla TCURV. Consulte la nota SAP 2792149 - Error de conversión de moneda/unidad: el tipo de conversión "<tipo de conversión>" tiene un conjunto de tipos "BKUZU" o "GKUZU" no admitidos.

Ejemplo

Cree una tabla y complétela con dos importes de moneda de ejemplo.

```
CREATE ROW TABLE sample_input (price DECIMAL(15,2),
                                source_unit NVARCHAR(4),
                                target_unit NVARCHAR(4),
                                ref_date NVARCHAR(10)
                                );

INSERT INTO sample_input VALUES (1.0, 'SRC', 'TRG', '2011-01-01');
INSERT INTO sample_input VALUES (1.0, 'SRC', 'TRG', '2011-02-01');
```

Convierta los valores en la tabla de monedas utilizando las tablas de conversión contenidas en el esquema SYSTEM en el arrendatario NDB.

```
SELECT CONVERT_CURRENCY(amount=>price,  
    "SOURCE_UNIT" =>source_unit,  
    "SCHEMA" => 'SYSTEM',  
    "DATABASE" => 'NDB',  
    "TARGET_UNIT" => target_unit,  
    "REFERENCE_DATE" =>'2013-09-23',  
    "ERROR_HANDLING"=>'set to null',  
    "CLIENT" => '000') as converted  
FROM sample_input;
```

Convierta los valores mediante Banking *method*.

```
SELECT *, CONVERT_CURRENCY(method=>'Banking', -- new  
  
    market_data_area=>'S000', -- new (and mandatory for Banking)  
  
    amount=>ext_limit,  
    source_unit =>ext_limit_curr,  
  
    schema => 'SAPCOB',  
  
    target_unit=> 'EUR',  
  
    reference_date => business_day,  
  
    error_handling=> 'set to null',  
  
    client => '150') as converted from v_fx_input
```

Convierta los valores utilizando Banking *method* y *system_time*.

```
SELECT *, CONVERT_CURRENCY(method=>'Banking', -- new  
  
    market_data_area=>'S000', -- mandatory (for Banking)  
  
    bid_ask_type=>'MID', -- optional  
  
    system_time=>to_timestamp('2011-05-11 12:59.999','YYYY-MM-DD HH:SS.FF3')  
  
    amount=>ext_limit,  
  
    source_unit =>ext_limit_curr,  
  
    schema => 'SAPCOB',  
  
    target_unit=> 'EUR',  
  
    reference_date => business_day,  
  
    error_handling=> 'set to null',  
  
    client => '150') as converted from v_fx_input
```

Información relacionada

[Nota SAP 2792149](#) 

[Expresiones](#)

Función CONVERT_UNIT (varios)

Convierte las unidades de origen especificadas en unidades de destino especificadas.

Sintaxis

```
CONVERT_UNIT( <named_parameter_value>, ... )
```

Elementos de sintaxis

<named_parameter_value>

Valor con nombre de los parámetros de referencia de campo o cadena constante.

```
<named_parameter_value> := "<field_reference_parameter>" =>  
  <expression> | "<const_string_parameter>" => <const_string>
```

```
<field_reference_parameter> ::=  
  QUANTITY  
  | SOURCE_UNIT  
  | TARGET_UNIT  
  | CLIENT
```

```
<const_string_parameter> ::=  
  SCHEMA  
  | DATABASE  
  | ERROR_HANDLING  
  | RATES_TABLE  
  | DIMENSION_TABLE
```

Nombre de parámetro de referencia de campo	Descripción	Obligatorio	Valor por defecto
CANTIDAD	La columna que se va a convertir.	Sí	Ninguno
SOURCE_UNIT	El identificador de columna que describe la unidad de entrada. Una cadena constante también se acepta mediante comillas simples.	Sí	Ninguno
TARGET_UNIT	El identificador de columna que describe la unidad de destino. Una cadena constante también se acepta mediante comillas simples.	Sí	Ninguno
CLIENTE	Define una cadena de tres caracteres que se utiliza para separar arrendatarios dentro de las tablas del sistema ERP. Esto se utiliza en las tablas de	Sí	Ninguno

Nombre de parámetro de referencia de campo	Descripción	Obligatorio	Valor por defecto								
	conversión para seleccionar las filas correctas para cada usuario. Un identificador de columna también se acepta mediante comillas dobles. Este parámetro es obligatorio, ya que este comando no utiliza la variable de contexto de sesión CLIENT.										
ESQUEMA	Define el esquema que contiene las tablas de conversión utilizadas para la conversión.	Sí	Ninguno								
BASE DE DATOS	Define el arrendatario en el que se encuentran las tablas de conversión. Si no se especifica, la conversión utiliza la base de datos actual.	No	Ninguno								
ERROR_HANDLING	Define cómo trata el sistema una situación en la que no se puede convertir una fila. <table><tr><th>Valor</th><th>Objetivo</th></tr><tr><td>fallo en caso de error</td><td>La conversión falla con un error.</td></tr><tr><td>definir como nulo</td><td>La salida de la fila que ha causado el error se fija en NULL.</td></tr><tr><td>mantener sin convertir</td><td>Se devuelve el valor de entrada.</td></tr></table>	Valor	Objetivo	fallo en caso de error	La conversión falla con un error.	definir como nulo	La salida de la fila que ha causado el error se fija en NULL.	mantener sin convertir	Se devuelve el valor de entrada.	No	fallo en caso de error
Valor	Objetivo										
fallo en caso de error	La conversión falla con un error.										
definir como nulo	La salida de la fila que ha causado el error se fija en NULL.										
mantener sin convertir	Se devuelve el valor de entrada.										
RATES_TABLE	Define una tabla que almacena las tasas de conversión. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	No	T006								
DIMENSION_TABLE	Define una tabla que almacena las dimensiones de conversión. Si la tabla reside en un arrendatario diferente, proporcione un nombre completamente cualificado para la tabla o utilice el parámetro DATABASE.	no	T006D								

Descripción

La función CONVERT_UNIT es una representación SQL de la función integrada de script SQL CE_CONVERSION y utiliza internamente CE_CONVERSION para el cálculo.

Para utilizar la función CONVERT_UNIT, las tablas de conversión de unidades T006 y T006D deben estar disponibles en la base de datos SAP HANA. Para otras bases de datos SAP HANA, replique las tablas T006 y T006D desde un sistema SAP ERP.

Ejemplo

```
CREATE ROW TABLE sample_input (  
    quant DECIMAL(15,2),  
    source_unit NVARCHAR(4),  
    target_unit NVARCHAR(4)  
);  
  
INSERT INTO sample_input VALUES (1.0, 'SRC', 'TRG');  
INSERT INTO sample_input VALUES (1.0, 'SRC', 'TRG');  
  
SELECT CONVERT_UNIT("QUANTITY"=>quant  
    , "SOURCE_UNIT" =>source_unit  
    , "SCHEMA" => 'SYSTEM'  
    , "DATABASE" => 'NDB'  
    , "TARGET_UNIT" => target_unit  
    , "ERROR_HANDLING"=>'set to null'  
    , "CLIENT" => '000') AS converted  
FROM sample_input;
```

Información relacionada

[Expresiones](#)

Función CORR (agregar)

Calcula el coeficiente de correlación de momento del producto Pearson entre dos columnas. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
CORR(<column1>, <column2>)
```

Función de ventana:

```
CORR(<column1>, <column2>) <window_specification>
```

Elementos de sintaxis

<column1> and <column2>

Especifica las columnas que proporcionan los datos de entrada para la correlación.

Los valores de <column1> y <column2> pueden ser de cualquier tipo numérico.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para `<window_Specification>`, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Calcula el coeficiente de correlación de momento del producto Pearson entre dos columnas.

El resultado oscila entre -1 y 1, dependiendo de la correlación, o NULL si no se puede calcular una correlación.

El resultado puede devolver NULL por uno de los siguientes motivos:

- Se correlacionan menos de dos pares de valores después de eliminar los valores NULL
- Hay una desviación cero en al menos una de las dos columnas

Ejemplos

Los siguientes ejemplos suponen que se ha creado una tabla de correlación con los siguientes valores:

```
CREATE COLUMN TABLE correlationTable (  
    ts_id NVARCHAR(20),  
    DATE DAYDATE,  
    value1 DOUBLE,  
    value2 DOUBLE);  
  
INSERT INTO correlationTable VALUES ('A', '2014-10-01', 1, 1);  
INSERT INTO correlationTable VALUES ('A', '2014-10-02', 2, 2);  
INSERT INTO correlationTable VALUES ('A', '2014-10-04', 3, 3);  
INSERT INTO correlationTable VALUES ('B', '2014-10-07', 1, 3);  
INSERT INTO correlationTable VALUES ('B', '2014-10-11', 2, 2);  
INSERT INTO correlationTable VALUES ('B', '2014-10-21', 3, 1);
```

1. El siguiente ejemplo de función de agregación devuelve la correlación entre la columna `ts_id` y las columnas `value1` y `value2`.

```
SELECT ts_id, CORR(value1, value2) FROM correlationTable GROUP BY ts_id;
```

Los resultados son los siguientes:

TS_ID	CORR(VALUE1, VALUE2)
A	1
B	-1

2. El siguiente ejemplo de cláusula `WHERE` devuelve la correlación entre la columna `ts_id` y las columnas `value1` y `value2` solo para filas en las que `ts_id` es igual a A.

```
SELECT ts_id, CORR(value1, value2) FROM correlationTable  
WHERE ts_id='A' GROUP BY ts_id;
```

Los resultados son los siguientes:

TS_ID	CORR(VALUE1, VALUE2)
A	1

3. El siguiente ejemplo utiliza una función de ventana.

```
SELECT ts_id, CORR(value1, value2) OVER (PARTITION BY ts_id) FROM correlationTable;
```

Los resultados son los siguientes:

TS_ID	CORR(VALUE1, VALUE2)OVER(PARTITIONBYTS_ID)
A	1

4. El siguiente ejemplo utiliza una función de ventana deslizante.

```
SELECT ts_id, CORR(value1, value2) OVER (PARTITION BY ts_id ORDER BY date)
FROM correlationTable ORDER BY ts_id;
```

Los resultados son los siguientes:

TS_ID	CORR(VALUE1, VALUE2)OVER(PARTITIONBYTS_IDORDERBYDATE)
A	?
A	0,9999999999999998
A	0,9999999999999998
B	?
B	-0.9999999999999998
B	-0.9999999999999998

5. El siguiente ejemplo utiliza una cláusula ROWS BETWEEN.

```
SELECT ts_id, CORR(value1, value2) OVER (PARTITION BY ts_id ORDER BY date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) from correlationTable;
```

Los resultados son los siguientes:

TS_ID	CORR(VALUE1, VALUE2)OVER(PARTITIONBYTS_IDORDERBYDATEROWS)
A	?
A	0,9999999999999998
A	0,9999999999999998
B	?
B	-0.9999999999999998
B	-0.9999999999999998

Información relacionada

[Funciones de ventana y la especificación de ventana](#)[Funciones agregadas de ventana](#)[Agregar funciones](#)[Función AUTO_CORR \(agregar\)](#)[Función CORR_SPEARMAN \(agregado\)](#)[Función CROSS_CORR \(agregar\)](#)

Función CORR_SPEARMAN (agregado)

Devuelve el coeficiente de correlación de clasificación de Spearman de los valores encontrados en las filas correspondientes de dos columnas. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

`CORR_SPEARMAN(<column1>, <column2>)`

Función de ventana:

`CORR_SPEARMAN(<column1>, <column2>) <window_specification>`

Elementos de sintaxis

<column1> and <column2>

Especifica las columnas que proporcionan los datos de entrada para la correlación.

Los valores de <column1> y <column2> pueden contener tipos de números o caracteres.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Devuelve el coeficiente de correlación de clasificación de Spearman de los valores encontrados en las filas correspondientes de <column1> y <column2>.

Aunque esta función está agrupada con las funciones agregadas, su cláusula OVER opcional también la posiciona como una función de Windows.

El resultado oscila entre -1 y 1, dependiendo de la correlación, o NULL si no se ha podido calcular una correlación.

El resultado puede devolver NULL por uno de los siguientes motivos:

- Se correlacionan menos de dos pares de valores después de eliminar los valores NULL
- Hay una desviación cero en al menos una de las dos columnas

Siempre que se encuentre un valor NULO, se ignorarán tanto el valor NULL como el valor correspondiente de la otra columna de entrada.

Ejemplos

El siguiente ejemplo devuelve -1.

```
CREATE COLUMN TABLE A (date DAYDATE, val INT);
INSERT INTO A VALUES ('2014-10-01', 100);
INSERT INTO A VALUES ('2014-10-02', 200);
INSERT INTO A VALUES ('2014-10-03', 300);

CREATE COLUMN TABLE B (date DAYDATE, val INT);
INSERT INTO B VALUES ('2014-10-01', 300);
INSERT INTO B VALUES ('2014-10-02', 200);
INSERT INTO B VALUES ('2014-10-03', 100);

SELECT CORR_SPEARMAN(A.val, B.val) "corr" FROM A, B WHERE A.date = B.date;
```

Los siguientes ejemplos suponen que la tabla de correlación se ha creado con los siguientes valores:

```
CREATE COLUMN TABLE correlationSpearmanTable (
    ts_id NVARCHAR(20),
    date DAYDATE,
    value1 DOUBLE,
    value2 DOUBLE);

INSERT INTO correlationSpearmanTable VALUES ('A', '2014-10-01', 34.345, 45.345);
INSERT INTO correlationSpearmanTable VALUES ('A', '2014-10-02', 27.145, 28.893);
INSERT INTO correlationSpearmanTable VALUES ('A', '2014-10-02', 48.312, 28.865);
INSERT INTO correlationSpearmanTable VALUES ('A', '2014-10-03', 94.213, 58.854);
INSERT INTO correlationSpearmanTable VALUES ('A', '2014-10-03', 16.567, 28.231);
INSERT INTO correlationSpearmanTable VALUES ('A', '2014-10-03', 38.894, 94.378);
INSERT INTO correlationSpearmanTable VALUES ('B', '2014-10-04', 45.643, 76.987);
INSERT INTO correlationSpearmanTable VALUES ('B', '2014-10-04', 53.345, 50.893);
INSERT INTO correlationSpearmanTable VALUES ('B', '2014-10-04', 66.342, 48.342);
INSERT INTO correlationSpearmanTable VALUES ('B', '2014-10-04', 76.432, 37.234);
INSERT INTO correlationSpearmanTable VALUES ('B', '2014-10-05', 88.432, 23.242);
INSERT INTO correlationSpearmanTable VALUES ('B', '2014-10-05', 93.234, 13.132);
```

1. Ejecute el siguiente ejemplo de función de agregación:

```
SELECT ts_id, CORR_spearman(value1, value2) FROM correlationSpearmanTable GROUP BY ts_id;
```

Los resultados son los siguientes:

TS_ID	CORR_SPEARMAN(VALUE1,VALUE2)
A	0,542857
B	-1

2. Ejecute la función de ventana siguiente:

```
SELECT ts_id, CORR_spearman(value1, value2) OVER (PARTITION BY ts_id) FROM correlationSpearma
```

Los resultados son los siguientes:

TS_ID	CORR_SPEARMAN(VALUE1,VALUE2)OVER(PARTITIONBYTS_ID)
A	0,542857
A	0,542857

TS_ID	CORR_SPEARMAN(VALUE1,VALUE2)OVER(PARTITIONBYTS_ID)
A	0,542857
A	0,542857
A	0,542857
A	0,542857
B	-1
B	-1
B	-1
B	-1
B	-1
B	-1

3. Ejecute el siguiente ejemplo de ventana deslizante:

```
SELECT ts_id, CORR_spearman(value1, value2) OVER (PARTITION BY ts_id ORDER BY date) FROM corr
```

Los resultados son los siguientes:

TS_ID	CORR_SPEARMAN(VALUE1,VALUE2)OVER(PARTITIONBYTS_IDORDERBYDATE)
A	?
A	-0,5
A	-0,5
A	0.5428571428571428
A	0.5428571428571428
A	0.5428571428571428
B	-1
B	-1
B	-1
B	-1
B	-1
B	-1

4. Ejecute el siguiente ejemplo ROWS BETWEEN:

```
SELECT ts_id, CORR_spearman(value1, value2) OVER (PARTITION BY ts_id ORDER BY date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) from correlationSpearmanTable;
```

Los resultados son los siguientes:

TS_ID	CORR_SPEARMAN(VALUE1,VALUE2)OVER(PARTITIONBYTS_IDORDERBYDATEROWS)
A	?
A	1
A	-0,5
A	0,4
A	0,7
A	0.5428571428571428
B	?
B	-1
B	-1
B	-1
B	-1
B	-1

5. Ejecute el siguiente ejemplo de grupo:

```
SELECT ts_id, CORR_spearman(value1, value2) OVER (PARTITION BY ts_id ORDER BY date GROUPS BET
```

Los resultados son los siguientes:

TS_ID	CORR_SPEARMAN(VALUE1,VALUE2)OVER(PARTITIONBYTS_IDORDERBYDATEGROUPS)
A	?
A	-0,5
A	-0,5
A	0.5428571428571428
A	0.5428571428571428
A	0.5428571428571428
B	-1
B	-1
B	-1
B	-1
B	-1
B	-1

[Funciones de ventana y la especificación de ventana](#)[Funciones agregadas de ventana](#)[Agregar funciones](#)[Función AUTO_CORR \(agregar\)](#)[Función CORR \(agregar\)](#)[Función CROSS_CORR \(agregar\)](#)

Función COS (numérica)

Devuelve el coseno del ángulo, en radianes, para el argumento especificado.

Sintaxis

`COS(<number>)`

Descripción

Devuelve el coseno del ángulo <número>, en radianes.

Ejemplo

El siguiente ejemplo devuelve el valor 1.0 para "cos":

```
SELECT COS (0.0) "cos" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función COSH (numérica)

Calcula el coseno hiperbólico del argumento especificado.

Sintaxis

`COSH(<number>)`

Descripción

Calcula el coseno hiperbólico del argumento numérico <número>.

Ejemplo

El siguiente ejemplo devuelve el valor 1.1276259652063807 para "cosh":

```
SELECT COSH (0.5) "cosh" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función COSINE_SIMILARITY (vector)

Calcula la similitud coseno de dos vectores.

Sintaxis

`COSINE_SIMILARITY(<vector1>, <vector2>)`

Elementos de sintaxis

<vector1>

Especifica el primer vector.

<vector2>

Especifica el segundo vector.

Descripción

Calcula el coseno del ángulo entre <vector1> y <vector2>, comparando la dirección de los vectores. Ambos vectores deben tener la misma dimensión.

El resultado es un número DOUBLE entre -1 y 1. Cuanto mayor sea el resultado, más similares serán los vectores.

Para dos vectores **u** y **v**, la similitud de coseno se define matemáticamente como sigue:

$$S_C(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2 \cdot \sum_{i=1}^n v_i^2}}$$

Ejemplos

El siguiente ejemplo devuelve 0.5:

```
SELECT COSINE_SIMILARITY(
  TO_REAL_VECTOR(' [1, 0, 0] '),
  TO_REAL_VECTOR(' [0.5, 0.8660254, 0] '))
FROM DUMMY;
```

El siguiente ejemplo devuelve 0.499919672300608:

```
SELECT COSINE_SIMILARITY(
  TO_HALF_VECTOR(' [1, 0, 0] '),
  TO_HALF_VECTOR(' [0.5, 0.8660254, 0] '))
FROM DUMMY;
```

Información relacionada

[Tipos de datos vectoriales](#)

Función COT (numérica)

Calcula la cotangente de un número especificado.

Sintaxis

`COT(<number>)`

Descripción

Calcula la cotangente de un número <número>, donde <número> es un ángulo expresado en radianes.

Ejemplo

El siguiente ejemplo devuelve el valor -0.8950829176379128 para "cuna":

```
SELECT COT (40) "cot" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función COUNT (agregar)

Cuenta el número de filas devueltas por una consulta. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
COUNT (*)  
| COUNT ( [ ALL ] <expression> )  
| COUNT ( DISTINCT <expression_list>
```

Función de ventana:

```
COUNT (*) [ <window_specification> ]  
| COUNT ( [ ALL ] <expression> ) [ <window_specification> ]  
| COUNT ( DISTINCT <expression> ) [ <window_specification> ]
```

Elementos de sintaxis

*

Devuelve el número de filas devueltas por la consulta, independientemente del valor de esas filas e incluyendo los valores duplicados.

ALL

Palabra clave opcional que refleja el comportamiento predeterminado. Es decir, COUNT (ALL <expresión>) y COUNT (<expresión>) devuelven el mismo resultado.

<expression>

Especifica los datos de entrada para la función. La función devuelve el número de valores no NULL para la expresión especificada devuelta por la consulta, incluidos los valores duplicados (a menos que también se especifique DISTINCT).

<expression_list>

Especifica los datos de entrada para la función como una lista separada por comas de <expresión>. <expression_list> solo se admite para su uso con la cláusula DISTINCT.

<expression_list>::= <expression> [, <expression> [,...]

DISTINCT

Devuelve el número de valores distintos devueltos por la consulta, excluyendo las filas con todos los valores NULL para esa expresión.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para obtener sintaxis, consulte [Funciones de ventana y Especificación de ventana](#).

Descripción

Tipo de resultado basado en entrada							
TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL(p,s)	DECIMAL	REAL	DOUBLE
BIGINT	BIGINT	BIGINT	BIGINT	BIGINT	BIGINT	BIGINT	BIGINT

Ejemplo

Las siguientes sentencias crean una tabla con datos, por ejemplo:

```
DROP TABLE "MyProducts";
CREATE COLUMN TABLE "MyProducts"(
  "Product_ID" NVARCHAR(10),
  "Product_Name" NVARCHAR(100),
  "Category" NVARCHAR(100),
  "Quantity" INTEGER,
  "Price" DECIMAL(10,2),
  PRIMARY KEY ("Product_ID") );

INSERT INTO "MyProducts" VALUES('P1','Shirts', 'Clothes', 32, 20.99);
INSERT INTO "MyProducts" VALUES('P2','Jackets', 'Clothes', 16, 99.49);
INSERT INTO "MyProducts" VALUES('P3','Trousers', 'Clothes', 30, 32.99);
INSERT INTO "MyProducts" VALUES('P4','Coats', 'Clothes', 5, 129.99);
INSERT INTO "MyProducts" VALUES('P5','Purse', 'Accessories', 3, 89.49);
```

El siguiente ejemplo devuelve 5, el número de filas en la tabla MyProducts:

```
SELECT COUNT(*) FROM "MyProducts";
```

El siguiente ejemplo devuelve 2, el número de filas con valores distintos en la columna Categoría:

```
SELECT COUNT(DISTINCT "Category") FROM "MyProducts";
```

Información relacionada

[Funciones de ventana y la especificación de ventana](#)

[Funciones agregadas de ventana](#)

[Agregar funciones](#)

Función CROSS_CORR (agregar)

Calcula todos los coeficientes de correlación cruzada entre dos expresiones dadas.

Sintaxis

```
CROSS_CORR(<expression1>, <expression2>, <maxLag>
           [, <order_by_clause> ] ).{ POSITIVE_LAGS | NEGATIVE_LAGS | ZERO_LAGS }
```

Elementos de sintaxis

<expression1> and <expression2>

Valores numéricos entre los que se calcula la correlación cruzada.

<maxLag>

El parámetro **<maxLag>** debe ser un entero positivo que defina el número de coeficientes de correlación cruzada que se devolverán.

<maxLag> ::= INTEGER

<order_by_clause>

Especifica el orden de clasificación de las filas de entrada.

<order_by_clause> ::= ORDER BY <order_by_expression> [, <order_by_expression> [...]]

<order_by_expression> ::=
<column_name> [<collate_clause>] [ASC | DESC] [NULLS FIRST | NULLS LAST]
| <column_position> [<collate_clause>] [ASC | DESC] [NULLS FIRST | NULLS LAST]

<collate_clause> ::= COLLATE <collation_name>

<clausula colateral> especifica la intercalación que se va a utilizar para ordenar los valores en los resultados. **<collate_cláusula>** solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un sinónimo de NVARCHAR).

<collation_name> es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

Descripción

Calcula todos los coeficientes de correlación cruzada entre dos expresiones dadas.

El resultado es una matriz de coeficientes de correlación cruzada de longitud *<maxLag>*.

Si se especifica POSITIVE_LAGS, se devuelven los coeficientes de correlación cruzada con retrasos 1*<maxLag>*.

Si se especifica NEGATIVE_LAGS, se devuelven los coeficientes de correlación cruzada con retrasos -1 *<maxLag>*.

Si se especifica ZERO_LAG, se devuelve un único valor asociado con el retraso 0.

Esta salida de función puede ser no determinista entre los valores de empate.

Ejemplo

Ejemplo: correlación cruzada

Ejecute el siguiente ejemplo de correlación cruzada:

```
CREATE COLUMN TABLE table1 ( ts_id INTEGER, number1 DOUBLE, number2 DOUBLE );

INSERT INTO table1 VALUES ('1', 1, 2);
INSERT INTO table1 VALUES ('2', 2 ,1);
INSERT INTO table1 VALUES ('3', 1 ,2);

SELECT CROSS_CORR(number1, number2, 10 ORDER BY ts_id) FROM table1;
```

Los resultados son los siguientes:

CROSS_CORR(NUMBER1,NUMBER2,10ORDERBYTS_ID)
1.0, -1.0, 1.0

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Agregar funciones](#)
- [Función AUTO_CORR \(agregar\)](#)
- [Función CORR \(agregar\)](#)
- [Función CORR_SPEARMAN \(agregado\)](#)
- [Expresiones](#)
- [Sentencia CREATE TABLE \(definición de datos\)](#)

Función CUBIC_SPLINE_APPROX (ventana)

Reemplaza los valores nulos interpolando los espacios basados en splines cúbicos calculados y extrapolando linealmente cualquier valor nulo inicial o final.

Sintaxis

```
CUBIC_SPLINE_APPROX ( <expression> [, <BoundaryConditionArgument> [, <ExtrapolationModeArgument> [,
[ OVER ( {
    SERIES(...) [ PARTITION BY <col> [,...] [ <window_order_by_clause> ]
| [ PARTITION BY <col>[,...] <window_order_by_clause>
} ) ]
```

Elementos de sintaxis

<expression>

Especifica la expresión de datos de entrada

<BoundaryConditionArgument>

Especifica las condiciones de límite que se utilizan para el cálculo de spline cúbico. El valor predeterminado es SPLINE_TYPE_NATURAL. Si se omite este parámetro, las condiciones límite naturales se utilizan para calcular las splines cúbicas de interpolación.

```
<BoundaryConditionArgument> ::= SPLINE_TYPE_NATURAL | SPLINE_TYPE_NOT_A_KNOT
```

<ExtrapolationModeArgument>

Define el modo de extrapolación aplicado. El valor predeterminado es EXTRAPOLATION_NONE, que indica que la extrapolación no se produce si se omite este parámetro.

```
<ExtrapolationModeArgument> ::= EXTRAPOLATION_NONE
| EXTRAPOLATION_LINEAR
| EXTRAPOLATION_CONSTANT
```

La definición, el valor predeterminado y el uso de ExtrapolationModeArgument junto con Value1Argument y Value2Argument son idénticos a la función LINEAR_APPROX excepto por el cálculo de la pendiente.

Cuando se utiliza el modo EXTRAPOLATION_LINEAR, la pendiente de la línea de extrapolación se establece igual a la pendiente de las splines cúbicas calculadas en los puntos límite. Por ejemplo, la pendiente de la spline calculada en el primer valor no nulo se toma como la pendiente de la línea de extrapolación para los valores nulos iniciales. De manera similar, la pendiente de la spline calculada en el último valor no nulo se toma como la pendiente de la línea de extrapolación para los nulos finales.

La interpolación spline cúbica se puede aplicar a todos los tipos de números que se pueden convertir en tipo doble largo, como INT, LONG, DOUBLE o FLOAT.

<window_order_by_clause>

Determina el orden de clasificación. Esta expresión no debe contener valores NULL ni duplicados. Consulte la sintaxis de esta cláusula ubicada en el tema *Funciones de ventana*.

Descripción

Reemplaza los valores nulos interpolando los espacios basados en splines cúbicos calculados y extrapolando linealmente cualquier valor nulo inicial o final.

Ejemplos

Interpolación spline cúbica natural

Realizar interpolación spline cúbica natural:

```
SELECT CUBIC_SPLINE_APPROX(temperature, 'SPLINE_TYPE_NATURAL') OVER (PARTITION BY station ORDER BY
FROM WEATHER;
```

Para las condiciones límite naturales, el BoundaryConditionArgument se puede omitir. La siguiente consulta devuelve resultados idénticos al anterior:

```
SELECT CUBIC_SPLINE_APPROX(temperature)
       OVER(PARTITION BY station ORDER BY ts) FROM WEATHER;
```

Realice la interpolación spline cúbica utilizando condiciones de límite sin un nudo:

```
SELECT CUBIC_SPLINE_APPROX(temperature, 'SPLINE_TYPE_NOT_A_KNOT')
       OVER(PARTITION BY station ORDER BY ts) FROM WEATHER;
```

Este ejemplo realiza la extrapolación lineal en una serie.

```
SELECT date, val, LINEAR_APPROX(val, 'EXTRAPOLATION_LINEAR') OVER(
    SERIES(SERIES KEY(ts_id) PERIOD FOR SERIES(date)
    EQUIDISTANT INCREMENT BY INTERVAL 1 DAY MISSING ELEMENTS ALLOWED))
FROM InterpolationTable;
```

Información relacionada

[Expresiones](#)

[Funciones de ventana y la especificación de ventana](#)

Función CUME_DIST (ventana)

Devuelve el rango relativo de una fila.

Sintaxis

```
CUME_DIST() <window_specification>
```

Elementos de sintaxis

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para *<window_specification>*, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Devuelve el rango relativo de una fila.

El rango relativo de una fila R se define como NP/NR, donde:

- NP se define como el número de filas anteriores o iguales con R en el orden de ventana de la partición de ventana de R.
- NR se define como el número de filas en el orden de la ventana.

Ejemplos

```
CREATE ROW TABLE ProductSales (ProdName NVARCHAR(50), Description NVARCHAR(20), Sales INT);
INSERT INTO ProductSales VALUES('Tee Shirt','Plain',21);
INSERT INTO ProductSales VALUES ('Tee Shirt','Lettered',22);
INSERT INTO ProductSales VALUES ('Tee Shirt','Team logo',30);
INSERT INTO ProductSales VALUES('Hoodie','Plain',60);
INSERT INTO ProductSales VALUES ('Hoodie','Lettered',65);
```

```
INSERT INTO ProductSales VALUES ('Hoodie','Team logo',80);
INSERT INTO ProductSales VALUES('Ballcap','Plain',8);
INSERT INTO ProductSales VALUES ('Ballcap','Lettered',40);
INSERT INTO ProductSales VALUES ('Ballcap','Team logo',27);

SELECT ProdName, Description, Sales,
       PERCENT_RANK() OVER (ORDER BY Sales ASC) AS Percent_Rank,
       CUME_DIST() OVER (ORDER BY Sales ASC) AS Cume_Dist
FROM ProductSales
ORDER BY Sales DESC;
```

PRODNAME	DESCRIPCIÓN	VENTAS	PERCENT_RANK	CUME_DIST
Sudadera con capucha	Logotipo del equipo	80	1	1
Sudadera con capucha	Letrado	65	0,875	0.8888888888888888
Sudadera con capucha	Sin formato	60	0,75	0.7777777777777778
Ballcap	Letrado	40	0,625	0.6666666666666666
Camiseta	Logotipo del equipo	30	0,5	0.5555555555555556
Ballcap	Logotipo del equipo	27	0,375	0.4444444444444444
Camiseta	Letrado	22	0,25	0.3333333333333333
Camiseta	Sin formato	21	0,125	0.2222222222222222
Ballcap	Sin formato	8	0	0.1111111111111111

Información relacionada

[Expresiones](#)
[Funciones de ventana y la especificación de ventana](#)

Función CURRENT_CONNECTION (varios)

Devuelve el ID de la conexión actual.

Sintaxis

```
CURRENT_CONNECTION
```

Descripción

Devuelve el ID de la conexión actual como un valor entero.

Esta función se llama sin llaves.

Ejemplo

La siguiente consulta devuelve el ID de la conexión actual.

```
SELECT CURRENT_CONNECTION "current connection" FROM DUMMY;
```

Información relacionada

[M CONNECTIONS Vista del sistema](#)

Función CURRENT_DATABASE (varios)

Devuelve una cadena que contiene el nombre de la base de datos de arrendatario actual.

Sintaxis

```
CURRENT_DATABASE ( )
```

Descripción

Devuelve el nombre de la base de datos de arrendatario actual en un entorno SAP HANA.

Ejemplo

El siguiente ejemplo demuestra la operación básica de la función devolviendo el nombre de la base de datos actual:

```
SELECT CURRENT_DATABASE ( ) "current database" FROM DUMMY;
```

Salida de muestra:

base de datos actual
H01

Función CURRENT_DATE (fecha y hora)

Devuelve la fecha actual del sistema local.

Sintaxis

```
CURRENT_DATE
```

Descripción

Devuelve la fecha actual del sistema local.

Se recomienda utilizar fechas UTC en lugar de fechas locales. Consulte la función CURRENT_UTCDATE para obtener más información.

Ejemplo

El siguiente ejemplo devuelve la fecha local y UTC del sistema local:

```
SELECT CURRENT_DATE "Current Date", CURRENT_UTCTIME "Coordinated Universal Time" FROM DUMMY;
```

	Fecha actual	Fecha universal coordinada
1	1 de mayo de 2019	1 de mayo de 2019

```
SELECT CURRENT_DATE "Current Date", CURRENT_UTCTIME "Coordinated Universal Time" FROM DUMMY;
```

	Fecha actual	Fecha universal coordinada
1	1 de mayo de 2019	1 de mayo de 2019

Información relacionada

[Tipos de datos de fecha y hora](#)
[Función CURRENT_UTCTIME \(fecha y hora\)](#)

Función CURRENT_IDENTITY_VALUE (varios)

Devuelve el último valor de identidad insertado en la sesión actual.

Elementos de sintaxis

```
CURRENT_IDENTITY_VALUE()
```

Descripción

Devuelve un valor BIGINT que representa el último valor de identidad insertado en la sesión actual. Si no se ha insertado ningún valor de identidad en la sesión actual, se devuelve NULL.

Ejemplo

El siguiente ejemplo crea la prueba de tabla e inserta una fila con la identidad 101. A continuación, se lee el valor de identidad y se devuelve.

```
CREATE COLUMN TABLE test (objectid BIGINT GENERATED BY DEFAULT AS IDENTITY (START WITH 101), number  
INSERT INTO test (number) VALUES ( 1 );  
SELECT CURRENT_IDENTITY_VALUE() "current identity value" FROM DUMMY;
```

valor de identidad actual
101

Función CURRENT_MVCC_SNAPSHOT_TIMESTAMP (fecha y hora)

Devuelve la marca de tiempo de la instantánea MVCC actual en formato SSSS (segundos después de la medianoche).

Sintaxis

```
CURRENT_MVCC_SNAPSHOT_TIMESTAMP()
```

Descripción

La función `CURRENT_MVCC_SNAPSHOT_TIMESTAMP` devuelve el cronomarcador de la instantánea MVCC actual en formato SSSS.

Ejemplo

El siguiente ejemplo devuelve el cronomarcador de la instantánea MVCC actual:

```
SELECT CURRENT_MVCC_SNAPSHOT_TIMESTAMP() FROM DUMMY;
```

Función CURRENT_OBJECT_SCHEMA (varios)

Devuelve el nombre de esquema actual al crear una vista.

Sintaxis

```
CURRENT_OBJECT_SCHEMA()
```

Descripción

Al crear una vista, utilice la función `CURRENT_OBJECT_SCHEMA`, donde se requiere un nombre de esquema en la definición de la vista, pero un nombre de esquema de codificación fija no funciona. Actualmente, solo las funciones `WORKDAYS_BETWEEN`, `ADD_WORKDAYS` y `CONVERT_CURRENCY` admiten `CURRENT_OBJECT_SCHEMA` como sustitución para un nombre de esquema real.

Ejemplo

El siguiente ejemplo sustituye `CURRENT_OBJECT_SCHEMA` por el nombre del esquema actual (en este caso, `SCHEMA1`):

```
CREATE VIEW SCHEMA1.VIEW1 AS SELECT WORKDAYS_BETWEEN('01', '2014-01-09', '2014-01-10', CURRENT_OBJS
```

Función CURRENT_ROLEGROUP (varios)

Devuelve una cadena que contiene el nombre del grupo de funciones actual.

Sintaxis

```
CURRENT_ROLEGROUP()
```

Descripción

Devuelve una cadena que contiene el nombre del grupo de funciones actual. Se devuelve NULL si no se establece ningún grupo de roles.

Ejemplo

El siguiente ejemplo devuelve el nombre del grupo de funciones actual:

```
SELECT CURRENT_ROLEGROUP ( ) FROM DUMMY;
```

Información relacionada

[Sentencia SET ROLEGROUP \(gestión de sesiones\)](#)

[Declaración UNSET ROLEGROUP \(gestión de sesiones\)](#)

Función CURRENT_SCHEMA (varios)

Devuelve una cadena que contiene el nombre del esquema actual.

Sintaxis

```
CURRENT_SCHEMA
```

Descripción

Devuelve una cadena que contiene el nombre del esquema actual.

Ejemplo

El siguiente ejemplo devuelve el nombre del esquema actual:

```
SELECT CURRENT_SCHEMA "current schema" FROM DUMMY;
```

Función CURRENT_SITE_ID (varios)

Devuelve el ID del sitio que llama la función. Esta función se utiliza principalmente en la replicación del sistema SAP HANA (HSR).

Sintaxis

```
CURRENT_SITE_ID( )
```

Descripción

Cada sitio en un entorno de replicación del sistema SAP HANA (HSR) tiene un ID de sitio. Llamar esta función en un sistema HSR devuelve el ID del sitio que llama la función. Aprender el ID de sitio es útil cuando utiliza el enrutamiento forzado.

Cuando esta función se llama en un sistema no HSR, se devuelve 0.

No haga referencia a la tabla DUMMY en la cláusula FROM al llamar la función. En su lugar, haga referencia a una tabla real en la ubicación de destino (por ejemplo, la tabla <sap_ema>.SVERS, que es la tabla recomendada para un entorno ABAP de SAP NetWeaver).

Esta función devuelve el ID de sitio actual una vez por cada fila de la tabla a la que hace referencia (es decir, el mismo ID de sitio, pero una vez para cada fila de tabla). Por lo tanto, se recomienda especificar TOP 1 al llamar la función para devolver el ID de sitio actual solo una vez. Consulte el ejemplo para saber cómo hacerlo.

Ejemplo

El siguiente ejemplo ficticio devuelve el ID del sitio que ejecuta la consulta:

```
SELECT TOP 1 CURRENT_SITE_ID() FROM mySchema.SVERS ( RESULT_LAG('hana_sr'));
```

Función CURRENT_TIME (fecha y hora)

Devuelve la hora del sistema local.

Sintaxis

```
CURRENT_TIME
```

Descripción

Devuelve la hora actual del sistema local.

Se recomienda utilizar horas UTC en lugar de horas locales. Consulte la función CURRENT_UTCTIME para obtener más información.

Ejemplo

El siguiente ejemplo devuelve la hora local y UTC del sistema:

```
SELECT CURRENT_TIME "Current Time", CURRENT_UTCTIME "Coordinated Universal Time" FROM DUMMY;
```

	Hora actual	Tiempo universal coordinado
1	7:56:51 AM	2:56:51 PM

Información relacionada

- [Tipos de datos de fecha y hora](#)
- [Función CURRENT_UTCTIME \(fecha y hora\)](#)

Función CURRENT_TIMESTAMP (fecha y hora)

Devuelve la información de cronomarcador del sistema local actual.

Sintaxis

```
CURRENT_TIMESTAMP[( <precision> )]
```

Elementos de sintaxis

<precision>

Especifica la precisión de los subsegundos visualizados. <precisión> es un tipo de datos entero con un rango válido de 0-7. Si no se especifica, la precisión predeterminada es tres.

Descripción

Devuelve la información de cronomarcador del sistema local actual.

Se recomienda utilizar cronomarcadores UTC en lugar de cronomarcadores locales. Consulte la función CURRENT_UTCTIMESTAMP para obtener más información.

Ejemplo

El siguiente ejemplo devuelve el cronomarcador local del sistema con cero, tres y siete valores de precisión:

```
SELECT
  CURRENT_TIMESTAMP,
  CURRENT_TIMESTAMP(0),
  CURRENT_TIMESTAMP(3),
  CURRENT_TIMESTAMP(7)
FROM DUMMY;
```

```
'2021-03-09 09:07:36.8110000',
'2021-03-09 09:07:36.0000000',
'2021-03-09 09:07:36.8110000',
'2021-03-09 09:07:36.8118739'
```

Información relacionada

[Tipos de datos de fecha y hora](#)

[Función CURRENT_UTCTIMESTAMP \(fecha y hora\)](#)

Función CURRENT_TRANSACTION_ISOLATION_LEVEL (Otros)

Devuelve una cadena que contiene el nivel de aislamiento de transacción actual.

Sintaxis

```
CURRENT_TRANSACTION_ISOLATION_LEVEL
```

Descripción

Devuelve una cadena que contiene el nivel de aislamiento de transacción actual.

Ejemplo

El siguiente ejemplo devuelve el nivel de aislamiento de transacción actual:

```
SELECT CURRENT_TRANSACTION_ISOLATION_LEVEL "current transaction isolation level" FROM DUMMY;
```

Función CURRENT_UPDATE_STATEMENT_SEQUENCE (varios)

Devuelve el número de sentencias escritas que se han emitido en una transacción incrementado en 1.

Sintaxis

```
CURRENT_UPDATE_STATEMENT_SEQUENCE()
```

Descripción

Devuelve el número de sentencias escritas que se han emitido en una transacción incrementado en 1. Si la transacción nunca ha emitido una transacción de escritura, la función devuelve 1. El valor inicial es 1. En una transacción de lectura, la función siempre devuelve 1.

Ejemplos

El siguiente ejemplo muestra cómo recuperar el número de secuencia de la sentencia de escritura actual de la transacción actual y devuelve 1:

```
CREATE COLUMN TABLE T(id INT);  
INSERT INTO T VALUES (1);  
SELECT CURRENT_UPDATE_STATEMENT_SEQUENCE() "statement sequence number" FROM DUMMY;
```

La siguiente sentencia también devuelve 1:

```
DELETE FROM T;  
SELECT CURRENT_UPDATE_STATEMENT_SEQUENCE() "statement sequence number" FROM DUMMY;
```

Función CURRENT_UPDATE_TRANSACTION (varios)

Devuelve el ID único de la transacción actual cuando está en modo de escritura.

Sintaxis

```
CURRENT_UPDATE_TRANSACTION()
```

Descripción

Devuelve el ID único (BIGINT) de la transacción actual cuando está en modo de escritura. Si la transacción actual está en modo de lectura, la función devuelve 0.

Ejemplo

El siguiente ejemplo devuelve el ID de transacción de actualización actual.

```
SELECT CURRENT_UPDATE_TRANSACTION() "current update transaction" FROM DUMMY;
```

Función CURRENT_USER (varios)

Devuelve el nombre de usuario actual en el contexto de la sentencia actual.

Sintaxis

```
CURRENT_USER
```

Descripción

Devuelve el nombre de usuario actual en el contexto de la sentencia actual. Este es el nombre de usuario que se encuentra actualmente en la parte superior de la pila de autorizaciones.

SESSION_USER y CURRENT_USER son funciones similares, pero hay diferencias. SESSION_USER refleja cómo la sesión conectada al servidor, mientras que CURRENT_USER puede verse afectada por la cláusula SQL SECURITY y refleja cómo se aplican los permisos.

Ejemplo

En el siguiente ejemplo, el USUARIO A crea un procedimiento:

```
CREATE PROCEDURE USER_A.PROC1 LANGUAGE SQLSCRIPT SQL SECURITY DEFINER AS  
BEGIN  
    SELECT CURRENT_USER "current user" FROM DUMMY;  
END;
```

USER B llama el procedimiento USER_A.PROC1, que devuelve USER_A:

```
CALL USER_A.PROC1;
```

El USUARIO A también crea el siguiente procedimiento:

```
CREATE PROCEDURE USER_A.PROC2 LANGUAGE SQLSCRIPT SQL SECURITY INVOKER AS  
BEGIN  
    SELECT CURRENT_USER "current user" FROM DUMMY;  
END;
```

El USUARIO B llama el procedimiento USER_A.PROC2, que devuelve USER_B:

```
CALL USER_A.PROC2;
```

Información relacionada

[Función SESSION_USER \(varios\)](#)

Función CURRENT_USER_ID (varios)

Devuelve el ID de usuario del usuario actual en el contexto de la sentencia actual.

Sintaxis

```
CURRENT_USER_ID()
```

Descripción

Devuelve el ID de usuario del usuario actual en el contexto de la sentencia actual. Este es el ID de usuario que se encuentra actualmente en la parte superior de la pila de autorizaciones.

Ejemplo

```
SELECT CURRENT_USER_ID() from DUMMY;
```

Función CURRENT_USERGROUP (varios)

Devuelve una cadena que contiene el nombre del grupo de usuarios actual.

Sintaxis

```
CURRENT_USERGROUP()
```

Descripción

Devuelve una cadena que contiene el nombre del grupo de usuarios actual. Se devuelve NULL si no se establece ningún grupo de usuarios.

Ejemplo

El siguiente ejemplo devuelve el nombre del grupo de usuarios actual:

```
SELECT CURRENT_USERGROUP () FROM DUMMY;
```

Función CURRENT_UTCDATE (fecha y hora)

Devuelve la fecha UTC actual.

Sintaxis

```
CURRENT_UTCDATE
```

Descripción

Devuelve la fecha UTC actual.

En el código de aplicación, se recomienda utilizar fechas UTC en lugar de fechas locales.

Ejemplo

El siguiente ejemplo devuelve la fecha local y UTC del sistema local:

```
SELECT CURRENT_DATE "Current Date", CURRENT_UTCDATE "Coordinated Universal Date" FROM DUMMY;
```

	Fecha actual	Fecha universal coordinada
1	1 de mayo de 2019	1 de mayo de 2019

Información relacionada

[Tipos de datos de fecha y hora](#)

Función CURRENT_UTCTIME (fecha y hora)

Devuelve la hora UTC actual.

Sintaxis

```
CURRENT_UTCTIME
```

Descripción

Devuelve la hora UTC actual.

Ejemplo

El siguiente ejemplo devuelve la hora local y UTC del sistema:

```
SELECT CURRENT_TIME "Current Time", CURRENT_UTCTIME "Coordinated Universal Time" FROM DUMMY;
```

	Hora actual	Tiempo universal coordinado
1	7:56:51 AM	2:56:51 PM

Información relacionada

[Tipos de datos de fecha y hora](#)

Función CURRENT_UTCTIMESTAMP (fecha y hora)

Devuelve el cronomarcador UTC actual.

Sintaxis

```
CURRENT_UTCTIMESTAMP[(<precision>)]
```

Elementos de sintaxis

<precision>

Especifica la precisión de los subsegundos visualizados. *<precisión>* es un tipo de datos entero con un rango válido de 0-7. Si no se especifica, la precisión predeterminada es tres.

Descripción

Devuelve el cronomarcador UTC actual.

Ejemplo

El siguiente ejemplo devuelve el cronomarcador UTC del sistema con cero, tres y siete valores de precisión:

```
SELECT
  CURRENT_UTCTIMESTAMP
  CURRENT_UTCTIMESTAMP(0)
  CURRENT_UTCTIMESTAMP(3)
  CURRENT_UTCTIMESTAMP(7)
FROM DUMMY;

'2021-03-09 09:07:36.8110000',
'2021-03-09 09:07:36.0000000',
'2021-03-09 09:07:36.8110000',
'2021-03-09 09:07:36.8118739'
```

Información relacionada

[Tipos de datos de fecha y hora](#)

[Función CURRENT_TIMESTAMP \(fecha y hora\)](#)

Función DAYNAME (fecha y hora)

Devuelve el nombre del día de la semana para la fecha especificada.

Sintaxis

```
DAYNAME(<date>)
```

Descripción

Devuelve el día de la semana en inglés para la fecha especificada.

Ejemplo

El siguiente ejemplo devuelve Lunes como día de la semana para la fecha especificada:

```
SELECT DAYNAME ( '2011-05-30' ) "dayname" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función DAYOFMONTH (fecha y hora)

Devuelve el día del mes de la fecha especificada.

Sintaxis

```
DAYOFMONTH(<date>)
```

Descripción

Devuelve un entero para el día del mes de la fecha especificada.

Ejemplo

El siguiente ejemplo devuelve 30 como el número del día del mes de la fecha especificada:

```
SELECT DAYOFMONTH ( '2011-05-30' ) "dayofmonth" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función DAYOFYEAR (fecha y hora)

Devuelve una representación en enteros del día del año para la fecha especificada.

Sintaxis

```
DAYOFYEAR(<date>)
```

Descripción

Devuelve una representación en enteros del día del año para la fecha especificada.

Ejemplo

El siguiente ejemplo devuelve el valor 150 como día del año de la fecha especificada:

```
SELECT DAYOFYEAR ( '2011-05-30' ) "dayofyear" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función DAYS_BETWEEN (fecha y hora)

Calcula el número de días enteros (24 horas) entre dos fechas.

Sintaxis

DAYS_BETWEEN(<date_1>, <date_2>)

Elementos de sintaxis

<date_1>

Especifica el TIMESTAMP inicial para la comparación.

<date_2>

Especifica el TIMESTAMP final para la comparación.

Descripción

Calcula el número de días enteros comprendidos entre <date_1> y <date_2>.

Ejemplo

El siguiente ejemplo devuelve el valor 31 para los días entre las dos fechas especificadas:

```
SELECT DAYS_BETWEEN (TO_DATE ('2009-12-05', 'YYYY-MM-DD'), TO_DATE('2010-01-05', 'YYYY-MM-DD')) "días";
```

El siguiente ejemplo devuelve el valor 0 para los días entre las dos fechas especificadas:

```
SELECT DAYS_BETWEEN('2018-02-07 23:00:00', '2018-02-08 01:00:00') AS sinceDays FROM dummy;
```

El siguiente ejemplo devuelve el valor 1 para los días entre las dos fechas especificadas:

```
SELECT DAYS_BETWEEN('2018-02-07 23:00:00', '2018-02-08 23:00:00') AS sinceDays FROM dummy;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

[Nota SAP 2573900 - Comportamiento modificado de la función DAYS_BETWEEN en HANA 2.0 SPS03](#) 

Función DENSE_RANK (ventana)

Realiza la misma operación de clasificación que la función RANK, excepto que la numeración de rango no se omite cuando se encuentran vínculos.

Sintaxis

```
DENSE_RANK() <window_specification>
```

Elementos de sintaxis

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La función DENSE_RANK realiza la misma operación de clasificación que la función RANK, excepto que la numeración de rango no se omite cuando se encuentran vínculos.

Ejemplos

En este ejemplo, la función RANK devuelve el rango 5 para la fila ('A' , 10 , 0) porque había dos filas que devolvieron el rango de 3. Sin embargo, DENSE_RANK devuelve el rango 4 para la fila ('A' , 10 , 0).

```
CREATE TABLE T (class NVARCHAR(10), val INT, offset INT);
INSERT INTO T VALUES('A', 1, 1);
INSERT INTO T VALUES('A', 3, 3);
INSERT INTO T VALUES('A', 5, null);
INSERT INTO T VALUES('A', 5, 2);
INSERT INTO T VALUES('A', 10, 0);
INSERT INTO T VALUES('B', 1, 3);
INSERT INTO T VALUES('B', 1, 1);
INSERT INTO T VALUES('B', 7, 1);

SELECT class,
       val,
       ROW_NUMBER() OVER (PARTITION BY class ORDER BY val) AS row_num,
       RANK() OVER (PARTITION BY class ORDER BY val) AS rank,
       DENSE_RANK() OVER (PARTITION BY class ORDER BY val) AS dense_rank
FROM T;
```

CLASE	VAL	ROW_NUM	CLASIFICACIÓN	DENSE_RANK
A	1	1	1	1
A	3	2	2	2
A	5	3	3	3
A	5	4	3	3
A	10	5	5	4
B	1	1	1	1
B	1	2	1	1
B	7	3	3	2

Información relacionada

[Expresiones](#)[Funciones de ventana y la especificación de ventana](#)

Función DFT (agregar)

Calcula expresiones de transformación discreta de fourier (DFT) y devuelve una matriz con elementos específicos.

Sintaxis

```
DFT( <expression>, <N> { <series_orderby> | <order_by_clause> } ).{ REAL | IMAGINARY | AMPLITUDE |
```

Elementos de sintaxis

<expression>

Especifica un valor que se supone que es una muestra tomada en un intervalo de tiempo constante. <expresión> no puede contener valores NULL.

<N>

Este parámetro debe ser una potencia de 2. La entrada se rellena con ceros si contiene menos de <N> elementos.

<series_orderby>

La definición SERIES solo se puede utilizar con una serie equidistante.

```
<series_orderby> ::= SERIES( <series_period> <series_equidistant_definition> )
```

La serie no debe contener elementos que faltan. Para obtener más información sobre esta cláusula, consulte la función SERIES_GENERATE.

<order_by_clause>

Especifica el orden de clasificación de las filas de entrada.

```
<order_by_clause> ::= ORDER BY <order_by_expression> [, <order_by_expression> [...]]
```

```
<order_by_expression> ::=  
  <column_name> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]  
  | <column_position> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
```

```
<collate_clause> ::= COLLATE <collation_name>
```

<clausula colateral> especifica la intercalación que se va a utilizar para ordenar los valores en los resultados. <collate_cláusula> solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un sinónimo de NVARCHAR).

<collation_name> es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

Descripción

Calcula la transformada de Fourier discreta de una expresión para los primeros valores <N> y devuelve una matriz con exactamente <N> elementos.

Los valores devueltos dependen del parámetro de salida, que debe ser REAL, IMAGINARY, AMPLITUDE o PHASE.

Ejemplos

El siguiente ejemplo calcula la transformada discreta de Fourier de una columna en una serie equidistante.

```
SELECT DFT(FRACTION_OF_MIN_MAX_RANGE, 4 SERIES(EQUIDISTANT INCREMENT BY 1 PERIOD FOR SERIES(element  
FROM SERIES_GENERATE_INTEGER(1,0,10));
```

El siguiente ejemplo utiliza la MY_TABLE ficticia y devuelve una matriz con 4 números que representan la parte real del resultado.

```
SELECT DFT(col, 4 ORDER BY DATE).REAL FROM MY_TABLE;
```

El siguiente ejemplo utiliza la MY_TABLE ficticia y devuelve una matriz con 8 números que representan la parte imaginaria del resultado.

```
SELECT DFT(col, 8 ORDER BY DATE).IMAGINARY FROM MY_TABLE;
```

El siguiente ejemplo utiliza la MY_TABLE ficticia y devuelve una matriz con 8 números que representan la parte de amplitud (i.e. $\sqrt{\text{REAL}^2 + \text{IMAGINARY}^2}$) del resultado.

```
SELECT DFT(col, 8 ORDER BY DATE).AMPLITUDE FROM MY_TABLE;
```

El siguiente ejemplo utiliza la MY_TABLE ficticia devuelve una matriz con 8 números que representan la parte de fase del resultado y rangos entre $-\pi$ y $+\pi$.

```
SELECT DFT(col, 8 ORDER BY DATE).PHASE FROM MY_TABLE;
```

Información relacionada

[Agregar funciones](#)

[Vista del sistema COLLATIONS](#)

[Función SERIES_GENERATE \(datos de serie\)](#)

Función ESCAPE_DOUBLE_QUOTES (seguridad)

Escapa de comillas dobles en la cadena especificada.

Sintaxis

```
ESCAPE_DOUBLE_QUOTES(<value>)
```

Descripción

Escapa de las comillas dobles en la cadena <valor>, asegurándose de que se utiliza un identificador SQL válido en las sentencias SQL dinámicas para evitar inyecciones SQL. La función devuelve la cadena de entrada con comillas dobles de escape.

Ejemplo

La siguiente consulta utiliza las comillas dobles y devuelve el valor TAB"LE.

```
SELECT ESCAPE_DOUBLE_QUOTES('TAB"LE') "table_name" FROM DUMMY;
```

Función ESCAPE_SINGLE_QUOTES (seguridad)

Escapa de comillas simples en la cadena especificada.

Sintaxis

```
ESCAPE_SINGLE_QUOTES(<value>)
```

Descripción

Escapa las comillas simples (apóstrofes) en la cadena dada <value>, asegurándose de que se utiliza un literal de cadena SQL válido en las sentencias SQL dinámicas para evitar inyecciones SQL. Devuelve la cadena de entrada con comillas simples de escape.

Ejemplos

La siguiente consulta escapa del contenido del parámetro Str'ing a Str' 'ing.

```
SELECT ESCAPE_SINGLE_QUOTES('Str'ing') "string_literal" FROM DUMMY;
```

El siguiente ejemplo de consulta muestra las cadenas recuperadas de una tabla t, tanto sin como con ESCAPE_SINGLE_QUOTES aplicado. La columna col_txt contiene las dos entradas Adam's y Eve.

```
CREATE COLUMN TABLE txt(  
col_txt NVARCHAR(5000) NOT NULL);  
  
INSERT INTO txt VALUES ('Adam's');  
INSERT INTO txt VALUES ('Eve');  
  
SELECT col_txt, escape_single_quotes(col_txt) FROM txt;
```

Los resultados son los siguientes:

COL_TXT	ESCAPE_SINGLE_QUOTES(COL_TXT)
Adam's	Adam''s
Eva	Eva

Función EXP (numérica)

Devuelve el resultado de la base de los logaritmos naturales e elevados a la potencia del argumento especificado.

Sintaxis

```
EXP(<number>)
```

Descripción

Devuelve el resultado de la base de los logaritmos naturales e elevados a la potencia del argumento <número>.

Ejemplo

El siguiente ejemplo devuelve el valor 2,718281828459045 para "exp":

```
SELECT EXP (1.0) "exp" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función EXPRESSION_MACRO (varios)

Devuelve resultados agregados de una consulta.

Sintaxis

```
EXPRESSION_MACRO( <expression_macro_alias> )
```

Elementos de sintaxis

<expression_macro_alias>

Indica el nombre de una macro de expresión.

Descripción

Utilice la función EXPRESSION_MACRO para seleccionar una expresión que se base en columnas y otros EXPRESSION_MACRO de la vista y que también pueda implicar funciones de agregación (como AVG). Una sentencia SELECT con un EXPRESSION_MACRO que implica una función de agregación requiere la misma cláusula GROUP BY que si la función de agregación hubiera aparecido directamente en la sentencia SELECT.

La lista de todas las macros de expresión se encuentra en la vista del sistema VIEW_EXPRESSION_MACROS.

Ejemplo

El siguiente ejemplo crea una tabla, define una vista en la tabla que incluye una macro de expresión y, a continuación, consulta la vista mediante la función EXPRESSION_MACRO para que la macro de expresión realice cálculos en el resultado. En este ejemplo, se devuelve el valor 15 (el promedio de 10 y 20).

```
CREATE TABLE t1(a INT);  
INSERT INTO t1 VALUES(10);  
INSERT INTO t1 VALUES(20);  
CREATE VIEW v1 AS SELECT * FROM t1 WITH EXPRESSION MACROS(AVG(a) AS avgA);  
SELECT EXPRESSION_MACRO(avgA) FROM v1;
```

Información relacionada

[Vista del sistema VIEW_EXPRESSION_MACROS](#)

Función EXTRACT (fecha y hora)

Devuelve la parte solicitada de una fecha especificada.

Sintaxis

```
EXTRACT( {YEAR | MONTH | DAY | HOUR | MINUTE | SECOND} FROM <date> )
```

Descripción

Devuelve la parte solicitada a partir de la fecha especificada.

Ejemplo

El siguiente ejemplo devuelve el valor 2010 del año extraído de la fecha especificada:

```
SELECT EXTRACT (YEAR FROM TO_DATE ('2010-01-04', 'YYYY-MM-DD')) "extract" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función FIRST_VALUE (agregar)

Devuelve el valor del primer elemento de una expresión. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
FIRST_VALUE( <expression> <order_by_clause> )
```

Función de ventana:

```
FIRST_VALUE( <expression> <order_by_clause> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica la expresión de los datos sobre los que operar.

<order_by_clause>

Especifica el orden de clasificación de las filas de entrada.

```
<order_by_clause> ::= ORDER BY <order_by_expression> [, <order_by_expression> [...] ]
```

```
<order_by_expression> ::=  
  <column_name> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]  
  | <column_position> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
```

```
<collate_clause> ::= COLLATE <collation_name>
```

<collate_clause> especifica la intercalación que se va a utilizar para ordenar los valores en los resultados. <collate_clause> solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un sinónimo de NVARCHAR).

<collation_name> es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Devuelve el valor del primer elemento en <expresión> como ordenado por <order_by_clause>.

Se devuelve NULL si el valor es NULL o si <expresión> está vacío.

La salida de la función FIRST_VALUE puede ser no determinista entre los valores de vinculación.

Ejemplo

El siguiente ejemplo devuelve el primer valor de la columna COL1 cuando la tabla está ordenada por COL2:

```
CREATE ROW TABLE T (COL1 DOUBLE, COL2 DOUBLE);

INSERT INTO T VALUES(9, 1);
INSERT INTO T VALUES(4, 5);
INSERT INTO T VALUES(7, 3);

SELECT FIRST_VALUE (COL1 ORDER BY COL2) FROM T;
```

La consulta devuelve 9.

Ejemplo de tipo de datos espaciales (ST_Geometry, ST_Point)

El siguiente ejemplo devuelve el primer valor, POINT (1 1), en formato binario, cuando la tabla se ordena por ID:

```
CREATE TABLE TAB (ID INT, SHAPE ST_Geometry(4326));

INSERT INTO TAB VALUES(1, ST_GeomFromText('POINT(1 1)', 4326));
INSERT INTO TAB VALUES(2, ST_GeomFromText('POINT(2 2)', 4326));
INSERT INTO TAB VALUES(3, ST_GeomFromText('POINT(3 3)', 4326));
INSERT INTO TAB VALUES(4, ST_GeomFromText('POINT(4 4)', 4326));

SELECT FIRST_VALUE(SHAPE ORDER BY ID) FROM TAB;
```

Para devolver un resultado como tipo de datos NVARCHAR, utilice la siguiente sentencia:

```
SELECT FIRST_VALUE(CAST(SHAPE AS NVARCHAR) ORDER BY ID) FROM TAB;
```

Información relacionada

[Funciones de ventana y la especificación de ventana](#)

[Funciones agregadas de ventana](#)

[Agregar funciones](#)

[SAP HANA Cloud, referencia espacial de base de datos de SAP HANA](#)

Función FLOOR (numérica)

Devuelve el entero más grande que no sea mayor que el argumento numérico especificado.

Sintaxis

FLOOR(<number>)

Descripción

Devuelve el valor decimal más grande que no sea mayor que el argumento numérico <número>.

Ejemplo

El siguiente ejemplo devuelve el valor 14.0 para "floor":

```
SELECT FLOOR (14.5) "floor" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función GENERATE_PASSWORD (seguridad)

Genera una contraseña.

Sintaxis

GENERATE_PASSWORD(<password_length> [, <usergroup_name>])

Elementos de sintaxis

<password_length>

Especifica la longitud de la contraseña para generar como un entero.

<usergroup_name>

Especifica un nombre de grupo de usuarios. Si hay parámetros de política de contraseñas definidos para el grupo de usuarios, se aplican durante la generación de contraseñas.

Descripción

Genera y devuelve una contraseña del tipo NVARCHAR(128).

i Nota

La llamada de función puede fallar al visualizar el siguiente mensaje de error fuera de rango si la política de contraseñas y la lista de elementos excluidos son demasiado restrictivos:

password exclude list too restrictive

Dependiendo de la rigurosidad de la lista de exclusión de contraseñas, puede que sea suficiente volver a intentar el comando.

Ejemplo

El siguiente ejemplo devuelve una contraseña generada de 16 caracteres:

```
SELECT  GENERATE_PASSWORD(16) FROM Dummy;
```

GET_DELTA_LAKE_TABLE_CHANGES Función (string)

Utilice esta función de tabla para recuperar el feed de datos de modificación entre determinadas versiones de una tabla delta.

Sintaxis

```
GET_DELTA_LAKE_TABLE_CHANGES ( <input_table>, <min_version> [, <max_version> ] )
```

Elementos de sintaxis

<input_table>

Especifica una tabla virtual o el sinónimo de la tabla. No se admiten las vistas de proyección.

<min_version>

Especifica la versión delta inicial desde la que recuperar las modificaciones (inclusive).

<max_version>

Especifica la versión delta final hasta la que se recuperarán las modificaciones (inclusive). Si no se especifica, se utiliza el valor predeterminado de NULL y se utilizan las modificaciones hasta la última versión incluida.

Descripción

Devuelve una tabla que incluye todas las columnas de la tabla de entrada más tres columnas adicionales, CHANGE_TYPE, COMMIT_VERSION y COMMIT_TIMESTAMP.

La función THE devuelve todas las columnas de la tabla delta de origen junto con

Nombre de columna	Tipo de datos	Descripción
_CHANGE_TYPE	NVARCHAR(16)	Especifica el tipo de modificación. Valores posibles: insert, delete, update_preimage o update_postimage
_COMMIT_VERSION	BIGINT	Especifica la versión delta que contiene la modificación.
_COMMIT_TIMESTAMP	CRONOMARCADOR	Especifica el cronomarcador del commit de la versión delta.

:

Ejemplo

Este ejemplo devuelve las modificaciones entre las versiones 3 y 7, incluidas:

```
SELECT * FROM GET_DELTA_LAKE_TABLE_CHANGES ("MYSCHEMA"."TABLE", 3, 7);
```

GET_DELTA_LAKE_TABLE_VERSION Función (string)

Devuelve la última versión delta de una tabla delta remota.

Sintaxis

```
GET_DELTA_LAKE_TABLE ( <table_ref> )
```

Elementos de sintaxis

<table_ref>

Especifica una tabla virtual, el sinónimo de la tabla o una vista de proyección de cualquiera de los dos.

Descripción

Devuelve la última versión de la tabla especificada como un valor BIGINT.

Ejemplo

Este ejemplo devuelve las modificaciones entre las versiones 3 y 7, incluidas:

```
SELECT * FROM GET_DELTA_LAKE_TABLE_VERSION ("MYSCHEMA"."TABLE") FROM DUMMY;
```

Función GREATEST (varios)

Devuelve el valor más alto entre los argumentos especificados.

Sintaxis

```
GREATEST(<argument> [{, <argument>}...])
```

Descripción

Devuelve el mayor valor entre los argumentos (n1, n2, ...). Si uno de los argumentos es NULL, el retorno es NULL.

Ejemplo

El siguiente ejemplo devuelve bb como el valor más alto:

```
SELECT GREATEST ('aa', 'ab', 'ba', 'bb') "greatest" FROM DUMMY;
```

Función GROUP_SCORE (varios)

Devuelve la relevancia de un registro encontrado.

Sintaxis

```
GROUP_SCORE ( <search_string> IN <score_columns> )
```

Elementos de sintaxis

<score_columns>

```
<score_columns> ::=
{ <single_column>
| <specific_columns_diff_options>
| <specific_columns_same_options>
| <all_columns> }
```

<columna_individual>

```
<single_column> ::=
<column_name> [ <scoring_type> ]
```

<Tipo_puntuación>

```
<scoring_type> ::=
{ EXACT [ <exact_options> ]
| FUZZY [ <fuzzy_options> ]
| LINEAR <scoring_details>
| GAUSSIAN <scoring_details>
| LOGARITHMIC <logarithmic_scoring_details> }
```

<opción_exactas>

```
<exact_options> ::=
{ ABBREVIATION SIMILARITY <value_range>
| AND SYMMETRIC {'ON' | 'OFF'}
| AND THRESHOLD <value_range>
| MINIMAL SCORE <value_range>
| WEIGHT <value_range_2>
| COMPOSE WORDS <positive_integer_constant>
| DECOMPOSE WORDS <positive_integer_constant>
| COMPOUND WORD WEIGHT <value_range_2>
| NON MATCHING TOKEN MODE {'max' | 'min' | 'all' | 'input' | 'table'}
| EXCESS TOKEN WEIGHT <value_range>
| SEARCH MODE <search_mode>
| TOKEN SEPARATORS <token_separators>
| MIME TYPE <mime_type> }
```

<opción_fuzzy>

```
<fuzzy_options> ::=
{ ABBREVIATION SIMILARITY <value_range>
| AND SYMMETRIC {'ON' | 'OFF'}
```

```

| AND THRESHOLD <value_range>
| MINIMAL SCORE <value_range>
| WEIGHT <value_range_2>
| COMPOSE WORDS <positive_integer_constant>
| DECOMPOSE WORDS <positive_integer_constant>
| COMPOUND WORD WEIGHT <value_range_2>
| NON MATCHING TOKEN MODE { 'max' | 'min' | 'all' | 'input' | 'table' }
| ERROR DEVALUE <value_range>
| EXCESS TOKEN WEIGHT <value_range>
| SUBSTRING MATCH { 'off' | 'anywhere' | 'beginning' }
| LANGUAGE TRANSLATION { 'on' | 'off' }
| LENGTH TOLERANCE <value_range>
| MAX DATE DISTANCE { 0 | <positive_integer_constant> }
| MINIMAL SEARCH LENGTH { 0 | <positive_integer_constant> }
| SEARCH MODE <search_mode>
| SIMILARITY CALCULATION MODE <similar_calculation_mode>
| SPELL CHECK FACTOR <value_range>
| MINIMAL TOKEN SCORE <value_range>
| TOKEN SEPARATORS <token_separators>
| MIME TYPE <mime_type> }

```

<valor_rango_1>

Un valor que es mayor o igual que 0,0 y menor o igual que 1. Por ejemplo, 0,326, 0,57 o 0,9.

<valor_rango_2>

Un valor mayor que 0 y menor que 1,0.

<constante flotante positiva>

Un número entero positivo mayor que 1,0. Por ejemplo, 57.326.

<Constante_integración_positiva>

Una constante de número entero positivo. Por ejemplo, 5.

<Constante_flotante>

Un valor no negativo.

<modo_búsqueda>

```

<search_mode> ::=
{ 'alphanum'
| 'alphanum identifier'
| 'house number'
| 'identifier'
| 'postcode'
| 'string'
| 'text' }

```

<similar_calculation_mode>

```

<similar_calculation_mode> ::=
{ 'compare'
| 'type ahead'
| 'search compare'
| 'search'
| 'symmetric search'
| 'substring search'
| 'flexible' }

```

<scoring_details>

```

<scoring_details> ::=
[ MINIMAL SCORE <value_range> ]
[ OFFSET { 0 | <positive_float_constant> } ]
[ DECAY <decay_value> ]
SCALE <positive_float_constant>

```

```
[ SEACH MODE { 'date' | 'numc' }
[ WEIGHT <weight_value> ]
```

<valor_decadencia>

Un valor no inferior a 0,0 pero inferior a 1,0.

<valor_ponderado>

Un valor mayor que 0,0 y menor o igual que 1,0.

<detalles de puntuación logarítmicos>

```
<logarithmic_scoring_details> ::=
[ MINIMAL SCORE <value_range> ]
[ OFFSET { 0 | <positive_float_constant> }
BASE <float_constant>
[ SEACH MODE 'numc' ]
[ WEIGHT <weight_value> ]
```

<specific_columns_diff_options>

```
<specific_columns_diff_options> ::=
( <single_column> [ , <single_column> ]... ] )
```

<specific_columns_same_options>

```
<specific_columns_same_options> ::=
( <single_column> [ , <single_column> ]... ] ) [ <scoring_type> ]
```

<all_columns>

```
<all_columns> ::= * [ <scoring_type> ]
```

Descripción

Utilice GROUP_SCORE para obtener la relevancia de un grupo de registros encontrados. GROUP_SCORE devuelve un valor real entre 0 y 1. La base de datos SAP HANA calcula una puntuación basada en la siguiente información:

- La relevancia, o ponderación, de los atributos. La relevancia de una coincidencia depende de la ponderación de la columna que causó la coincidencia. Puede especificar ponderaciones al crear la vista.
- Precisión en una búsqueda inexacta. Cuanto más exacta sea la coincidencia que se produzca, mayor será la puntuación.

Ejemplos

El siguiente ejemplo crea dos tablas que contienen varias cadenas:

```
CREATE TABLE companies (id int, name varchar(255));
INSERT INTO companies VALUES (1, 'Company One');
INSERT INTO companies VALUES (2, 'Company Two');
INSERT INTO companies VALUES (3, 'Company Three');
```

```
CREATE TABLE locations (company_id int, location nvarchar(255));
INSERT INTO locations VALUES (1, 'Berlin');
INSERT INTO locations VALUES (1, 'Seattle');
INSERT INTO locations VALUES (1, 'Tokyo');
INSERT INTO locations VALUES (2, 'Berlin');
```

```
INSERT INTO locations VALUES (2, 'Tokyo');
INSERT INTO locations VALUES (3, 'London');
```

Opción 1: Palabra clave FUZZY

Utilice la función GROUP_SCORE con la palabra clave FUZZY para verificar la relevancia de los datos con el string de búsqueda "Berlín Seattle Tokio":

```
SELECT name, group_score('Berlin Seattle Tokyo' IN (name, location) FUZZY MINIMAL SCORE 0.0)
FROM companies JOIN locations ON id = company_id
GROUP BY name;
```

Resultados previstos:

NOMBRE	group_score('Berlin Seattle Tokyo' IN (nombre, ubicación) FUZZY MINIMAL SCORE 0.0)
Empresa uno	1.0
Empresa dos	0,580973744392395
Empresa tres	0.14778196811676025

Opción 2: Palabra clave EXACT

Utilice la función GROUP_SCORE con la palabra clave EXACT:

```
SELECT name, group_score('Berlin Seattle Tokyo' IN (name, location) EXACT)
FROM companies JOIN locations ON id = company_id
GROUP BY name;
```

Resultados previstos:

NOMBRE	group_score('Berlin Seattle Tokyo' IN (nombre, ubicación) EXACT)
Empresa uno	1.0
Empresa dos	0
Empresa tres	0

Información relacionada

- [Función SCORE \(varios\).](#)
- [Asignaciones de términos](#)

Función GROUPING (varios)

Determina si una columna especificada se utiliza en la agrupación.

Sintaxis

GROUPING(<column_name>)

Descripción

La función GROUPING se puede utilizar con GROUPING SETS, ROLLUP o CUBE, que devuelven varios niveles de agregación en un único conjunto de resultados. La función devuelve 1 si la columna especificada se utiliza en la agregación, y 0 en caso contrario. La columna de GROUPING debe ser un elemento de GROUPING SETS.

Ejemplo

```
CREATE COLUMN TABLE mySchema.Customers (  
  cust_id INTEGER NOT NULL,  
  cust_name NVARCHAR(20),  
  num_emp INTEGER,  
  region NVARCHAR(20),  
  s_tier NVARCHAR(20),  
  PRIMARY KEY ("CUST_ID")  
);  
  
INSERT INTO mySchema.Customers VALUES( 1, 'CustA', 5, 'NorthEast', 'gold' );  
INSERT INTO mySchema.Customers VALUES( 2, 'CustB', 26, 'NorthWest', 'gold' );  
INSERT INTO mySchema.Customers VALUES( 3, 'CustC', 250, 'NorthEast', 'silver' );  
INSERT INTO mySchema.Customers VALUES( 4, 'CustD', 180, 'SouthEast', 'platinum' );  
INSERT INTO mySchema.Customers VALUES( 5, 'CustE', 32, 'SouthWest', 'silver' );  
INSERT INTO mySchema.Customers VALUES( 6, 'CustF', 45, 'NorthEast', 'platinum' );  
INSERT INTO mySchema.Customers VALUES( 7, 'CustG', 15, 'NorthWest', 'platinum' );  
INSERT INTO mySchema.Customers VALUES( 8, 'CustH', 99, 'SouthEast', 'gold' );  
INSERT INTO mySchema.Customers VALUES( 9, 'CustI', 6, 'NorthEast', 'silver' );  
INSERT INTO mySchema.Customers VALUES( 10, 'CustJ', 101, 'NorthEast', 'silver' );  
INSERT INTO mySchema.Customers VALUES( 11, 'CustK', 108, 'SouthEast', 'silver' );  
  
SELECT cust_name AS "cust_name", cust_id AS "cust_id", region AS "region", s_tier AS "s_tier", num_  
GROUPING (region) AS "gr_reg", GROUPING (s_tier) AS "gr_tier", GROUPING (num_emp) AS "gr_num"  
FROM mySchema.Customers  
GROUP BY GROUPING SETS (  
  (s_tier, region),  
  (region, s_tier),  
  (cust_id, cust_name, num_emp)  
);
```

nombre_cliente	cust_id	región	nivel_ventas	num_emp	gr_reg	gr_tier	gr_num
NULO	NULO	Nordeste	oro	NULO	0	0	1
NULO	NULO	Noroeste	oro	NULO	0	0	1
NULO	NULO	SouthEast	oro	NULO	0	0	1
NULO	NULO	Nordeste	platino	NULO	0	0	1
NULO	NULO	Noroeste	platino	NULO	0	0	1
NULO	NULO	SouthEast	platino	NULO	0	0	1
NULO	NULO	Nordeste	plata	NULO	0	0	1
NULO	NULO	SouthEast	plata	NULO	0	0	1
NULO	NULO	Suroeste	plata	NULO	0	0	1
NULO	NULO	Nordeste	oro	NULO	0	0	1
NULO	NULO	Noroeste	oro	NULO	0	0	1

nombre_cliente	cust_id	región	nivel_ventas	num_emp	gr_reg	gr_tier	gr_num
NULO	NULO	SouthEast	oro	NULO	0	0	1
NULO	NULO	Nordeste	platino	NULO	0	0	1
NULO	NULO	Noroeste	platino	NULO	0	0	1
NULO	NULO	SouthEast	platino	NULO	0	0	1
NULO	NULO	Nordeste	plata	NULO	0	0	1
NULO	NULO	SouthEast	plata	NULO	0	0	1
NULO	NULO	Suroeste	plata	NULO	0	0	1
Aduana	1	NULO	NULO	5	1	1	0
InClte	9	NULO	NULO	6	1	1	0
GrClte	7	NULO	NULO	15	1	1	0
BClte	2	NULO	NULO	26	1	1	0
ClteE	5	NULO	NULO	32	1	1	0
Fte.clte.	6	NULO	NULO	45	1	1	0
H clte.	8	NULO	NULO	99	1	1	0
IDClte	10	NULO	NULO	101	1	1	0
Clte.	11	NULO	NULO	108	1	1	0
FeClte	4	NULO	NULO	180	1	1	0
ClteC	3	NULO	NULO	250	1	1	0

Función GROUPING_ID (varios)

Devuelve un valor entero para identificar a qué conjunto de agrupaciones pertenece cada fila.

Sintaxis

```
GROUPING_ID(<column_name_list>)
```

Descripción

La función GROUPING_ID se puede utilizar con GROUPING SETS, ROLLUP o CUBE, que devuelven varios niveles de agregación en un único conjunto de resultados. GROUPING_ID devuelve un valor entero para identificar a qué conjunto de agrupaciones pertenece cada fila. Cada columna de GROUPING_ID debe ser un elemento de GROUPING SETS.

GROUPING_ID se asigna convirtiendo el vector de bits generado a partir de GROUPING SETS en un número decimal tratando el vector de bits como un número binario. Cuando se compone un vector de bits, se asigna 0 a cada columna especificada en los CONJUNTOS DE GROUPING y 1 en caso contrario en el orden en que aparece en GROUPING SETS. Al tratar el vector de bits como un número binario, esta función devuelve un valor entero como salida.

Ejemplo

La siguiente sentencia utiliza la tabla ficticia `guided_navi_tab` y los resultados indican a qué conjunto de agrupaciones pertenece cada fila:

```
SELECT customer, year, product, SUM(sales),
       GROUPING_ID(customer, year, product)
FROM guided_navi_tab
GROUP BY GROUPING SETS (
  (customer, year, product),
  (customer, year),
  (customer, product),
  (year, product),
  (customer),
  (year),
  (product));
```

CLIENTE	AÑO	PRODUCTO	SUMA (VENTAS)	GROUPING_ID(CLIENTE,AÑO,PRODUCTO),
C1	2009	P1	100	0
C1	2010	P1	50	0
C2	2009	P1	200	0
C2	2010	P1	100	0
C1	2009	P2	200	0
C1	2010	P2	150	0
C2	2009	P2	300	0
C2	2010	P2	150	0
C1	2009	a	300	1
C1	2010	a	200	1
C2	2009	a	500	1
C2	2010	a	250	1
C1	a	P1	150	2
C2	a	P1	300	2
C1	a	P2	350	2
C2	a	P2	450	2
a	2009	P1	300	4
a	2010	P1	150	4
a	2009	P2	500	4
a	2010	P2	300	4
C1	a	a	500	3
C2	a	a	750	3

CLIENTE	AÑO	PRODUCTO	SUMA (VENTAS)	GROUPING_ID(CLIENTE,AÑO, PRODUCTO),
a	2009	a	800	5
a	2010	a	450	5
a	a	P1	450	6
a	a	P2	800	6

Función HAMMING_DISTANCE (string)

Realiza una comparación bit a bit o bitwise entre dos argumentos y devuelve la distancia de hamming.

Sintaxis

```
HAMMING_DISTANCE(<left_hand_side>, <right_hand_side>)
```

Elementos de sintaxis

<left_hand_side>

Especifica un argumento del lado izquierdo con el que se compara **<right_hand_side>**.

```
<lhs> ::= <string>
```

<right_hand_side>

Especifica un argumento del lado derecho para comparar con **<left_hand_side>**

```
<rhs> ::= <string>
```

Descripción

La distancia de Hamming describe las diferencias entre dos argumentos especificados, y refleja el número de posiciones correspondientes que difieren entre sí a través de los dos argumentos. Los argumentos enteros y binarios se comparan por bits, mientras que los argumentos de cadena se comparan por bytes.

La función HAMMING_DISTANCE devuelve -1 si la longitud de los dos argumentos es diferente, y NULL si alguno de los argumentos es NULL.

Ejemplos

El siguiente ejemplo devuelve -1 porque los argumentos son de diferente longitud:

```
SELECT HAMMING_DISTANCE('abc', 'ca') "hamming_distance" FROM DUMMY;
```

El siguiente ejemplo devuelve 0 porque los argumentos son idénticos:

```
SELECT HAMMING_DISTANCE('abc', 'abc') "hamming_distance" FROM DUMMY;
```

El siguiente ejemplo devuelve 0 porque los argumentos son idénticos:

```
SELECT HAMMING_DISTANCE(4, 4) "hamming_distance" FROM DUMMY;
```

El siguiente ejemplo devuelve 3 porque todas las posiciones de los dos argumentos son diferentes de la otra cadena:

```
SELECT HAMMING_DISTANCE('abc', 'cab') "hamming_distance" FROM DUMMY;
```

El siguiente ejemplo devuelve 4 para reflejar la comparación por bits de los argumentos:

```
SELECT HAMMING_DISTANCE(TO_BINARY('abc'), TO_BINARY('cab')) "hamming_distance" FROM DUMMY;
```

El siguiente ejemplo devuelve 3 para reflejar la comparación por bits de los argumentos:

```
SELECT HAMMING_DISTANCE(4, 9) "hamming_distance" FROM DUMMY;
```

El siguiente ejemplo devuelve -1 porque los argumentos binarios son de diferente longitud:

```
SELECT HAMMING_DISTANCE(to_binary('abc'), to_binary('ca')) "hamming_distance" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función HASH_MD5 (varios)

Devuelve un valor de control de 32 bytes de los argumentos concatenados.

Sintaxis

```
HASH_MD5(<argument> [, ...])
```

Elementos de sintaxis

<argument>

Especifica un valor VARBINARY.

```
<argument> ::= <string_literal>
```

Descripción

Devuelve un valor de control VARBINARY de 32 bytes de los argumentos concatenados. El hash se calcula utilizando un algoritmo MD5.

Para garantizar resultados únicos de argumentos concatenados, delimite los argumentos con otra cadena como se muestra en los siguientes ejemplos.

Ejemplo

En el siguiente ejemplo, la consulta genera un valor de control basado en la entrada proporcionada:

```
SELECT HASH_MD5 (TO_BINARY('database')) "hash" FROM DUMMY;
```

Los siguientes ejemplos, que devuelven 7AC66C0F148DE9519B8BD264312C4D64, muestran cómo la función concatena sus argumentos:

```
SELECT HASH_MD5(to_binary('abcd'), TO_BINARY('efg')) "test1" FROM DUMMY;
```

```
SELECT HASH_MD5(TO_BINARY('abc'), TO_BINARY('defg')) "test2" FROM DUMMY;
```

Los siguientes ejemplos muestran cómo puede delimitar los argumentos con otra cadena (en este caso con 00) para garantizar resultados únicos de la concatenación. La primera sentencia devuelve 6BB6BD45C57D57ECC69A9EB81F7409BE, mientras que la segunda sentencia devuelve 1CEDEB6EB7ED12D948F66593589FBD23. Sin el delimitador, ambas sentencias devuelven el mismo valor: 7AC66C0F148DE9519B8BD264312C4D64.

```
SELECT HASH_MD5(TO_BINARY('abcd'), '00', TO_BINARY('efg')) "test1" FROM DUMMY;
```

```
SELECT HASH_MD5(TO_BINARY('abc'), '00', TO_BINARY('defg')) "test2" FROM DUMMY;
```

Función HASH_SHA256 (varios)

Devuelve un valor de control de 32 bytes de los argumentos concatenados.

Sintaxis

```
HASH_SHA256(<argument> [, ...])
```

Elementos de sintaxis

<argument>

Especifica un argumento del tipo VARBINARY.

```
<argument> ::= <string_literal>
```

Descripción

Devuelve un valor de control VARBINARY de 32 bytes de los argumentos concatenados. El hash se calcula mediante un algoritmo SHA256.

Para garantizar resultados únicos de argumentos concatenados, delimite los argumentos con otra cadena como se muestra en los siguientes ejemplos.

Ejemplo

La siguiente consulta devuelve un valor de control para los argumentos especificados:

```
SELECT HASH_SHA256 (TO_BINARY('database')) "hash" FROM DUMMY;
```

Los siguientes ejemplos, que devuelven 7D1A54127B222502F5B79B5FB0803061152A44F92B37E23C6527BAF665D4DA9A, muestran cómo la función concatena sus argumentos:

```
SELECT HASH_SHA256(TO_BINARY('abcd'), TO_BINARY('efg')) "test1" FROM DUMMY;
```

```
SELECT HASH_SHA256(TO_BINARY('abc'), TO_BINARY('defg')) "test2" FROM DUMMY;
```

Los siguientes ejemplos muestran cómo puede delimitar los argumentos con otra cadena (en este caso con 00) para garantizar resultados únicos de la concatenación. La primera sentencia devuelve

156D73B945474C6FB04D7CCAC1E31ACA9425F756801AD487C3561FBE6661A659, mientras que la segunda sentencia devuelve 6DE75B6290747BFA61C10FD0763344002C436475683B2B7E44C7C31FA920F1E3. Sin el delimitador, ambas sentencias devuelven el mismo valor: 7D1A54127B222502F5B79B5FB0803061152A44F92B37E23C6527BAF665D4DA9A.

```
SELECT HASH_SHA256(TO_BINARY('abcd'), '00', TO_BINARY('efg')) "test3" FROM DUMMY;
```

```
SELECT HASH_SHA256(TO_BINARY('abc'), '00', TO_BINARY('defg')) "test4" FROM DUMMY;
```

Función HEXTOBIN (cadena)

Convierte una cadena de caracteres hexadecimales en un valor VARBINARY.

Sintaxis

```
HEXTOBIN(<hexadecimal_string>)
```

Descripción

HEXTOBIN devuelve un valor VARBINARY en el que cada byte del resultado corresponde a dos caracteres de <hexadecimal_string>. Si <hexadecimal_string> no contiene un número par de dígitos, se devuelve un error.

En SAP HANA, se admiten caracteres en minúscula en <hexadecimal_string> y se tratan como mayúsculas.

Ejemplo

Los dos ejemplos siguientes devuelven el valor VARBINARY 608DA975:

```
SELECT HEXTOBIN ('608da975') "Result" FROM DUMMY;
```

```
SELECT HEXTOBIN ('608DA975') "Result" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función HEXTONUM (cadena)

Convierte un valor hexadecimal en un valor de cadena BIGINT.

Sintaxis

```
HEXTONUM(<string> [, -1])
```

Elementos de sintaxis

<string>

Indica el valor hexadecimal que se debe convertir en un valor de string.

-1

El resultado se interpreta como un número negativo si se proporciona un segundo parámetro.

Descripción

Convierte un valor hexadecimal en un valor de string como tipo de datos BIGINT. El valor de entrada debe ser un tipo de cadena NVARCHAR que no supere los 16 caracteres.

Ejemplo

El siguiente ejemplo convierte el valor hexadecimal c2 en un valor de string BIGINT 12,345:

```
CREATE TABLE t2 (c2 NVARCHAR(16));  
INSERT INTO t2 VALUES('7FFFFFFFFFFFFFFF');  
INSERT INTO t2 VALUES('8000000000000000');  
INSERT INTO t2 VALUES('FFFFFFFFFFFFFFF');  
INSERT INTO t2 VALUES('0');  
INSERT INTO t2 VALUES('3039');  
SELECT c2, HEXTONUM(c2) FROM t2;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función HOUR (fecha y hora)

Devuelve una representación en enteros de la porción de hora de la hora especificada.

Sintaxis

```
HOUR(<time>)
```

Descripción

Devuelve una representación en enteros de la porción de hora de la hora especificada.

Ejemplo

El siguiente ejemplo devuelve la hora 12:

```
SELECT HOUR ('12:34:56') "hour" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función IFNULL (varios)

Devuelve la primera expresión de entrada que no sea NULL.

Sintaxis

```
IFNULL(<expresión1>, <expresión2>)
```

Descripción

IFNULL devuelve la primera expresión de entrada no NULL. Si los tipos de datos de <expresión1> y <expresión2> son diferentes, SAP HANA selecciona el tipo de datos con la prioridad más alta. Por ejemplo, entre TIMESTAMP y STRING, TIMESTAMP tiene la prioridad más alta. Consulte la documentación de tipos de datos para obtener más información sobre la prioridad del tipo de datos.

- Devuelve <expresión1> si <expresión1> no es NULO.
- Devuelve <expresión2> si <expresión1> es NULO.
- Devuelve NULL si ambas expresiones de entrada son NULL.

Ejemplo

La siguiente consulta devuelve diff:

```
SELECT IFNULL ('diff', 'same') "ifnull" FROM DUMMY;
```

La siguiente consulta devuelve lo mismo:

```
SELECT IFNULL (NULL, 'same') "ifnull" FROM DUMMY;
```

La siguiente consulta devuelve NULL:

```
SELECT IFNULL (NULL, NULL) "ifnull" FROM DUMMY;
```

Información relacionada

[Expresiones](#)

[Tipos de datos](#)

Función INITCAP (cadena)

Convierte el primer carácter de cada palabra de una cadena especificada en mayúsculas y convierte los caracteres restantes en minúsculas.

Sintaxis

```
INITCAP(<inputString>)
```

Elementos de sintaxis

<inputString>

Especifica el string NVARCHAR que se va a convertir.

<inputString> ::= <string>

Descripción

Una palabra está delimitada por cualquiera de los siguientes caracteres:

- Espacio en blanco
- Línea nueva
- Feed de formulario
- Retorno de carro
- Avance de línea
- Cualquiera de los siguientes: ! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ ` { | } ~

Ejemplo

La siguiente consulta devuelve El ejemplo uno:

```
SELECT INITCAP('the EXAMPLE one') FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

IS_SQL_INJECTION_SAFE Función (seguridad)

Comprueba un identificador SQL especificado para detectar posibles riesgos de inyección SQL.

Sintaxis

```
IS_SQL_INJECTION_SAFE(<value>[, <max_tokens>])
```

Elementos de sintaxis

<value>

Especifica la cadena que se debe verificar.

<value> ::= <string>

<max_tokens>

Especifica el número máximo de tokens esperados en <valor>. El valor predeterminado es 1.

<max_tokens> ::= <integer>

Descripción

Utilice esta función para garantizar que una cadena contenga el número especificado de tokens y comentarios SQL antes de utilizarlo en una sentencia SQL. Se devuelve un 1 (seguro) cuando el número real de tokens encontrados no supera <max_tokens> y no se encontraron comentarios; de lo contrario, se devuelve un 0 (no seguro).

Para el recuento, SAP HANA interpreta los espacios en blanco y los siguientes caracteres como separadores entre tokens:

, () [] . ; : + - * / % ^ < > =

Ejemplo

La siguiente consulta devuelve 0 (no seguro) porque se encontraron dos tokens y el valor predeterminado para comprobar fue 1.

```
SELECT IS_SQL_INJECTION_SAFE('tab,le') "safe" FROM DUMMY;
```

La siguiente consulta devuelve 0 (no es seguro) porque los comentarios no están permitidos.

```
SELECT IS_SQL_INJECTION_SAFE('mytab /*', 4) "safe" FROM DUMMY;
```

Función ISOWEEK (fecha y hora)

Devuelve el número de año y semana ISO para una fecha especificada.

Sintaxis

ISOWEEK(<date>)

Descripción

Devuelve los números de año y semana ISO de la fecha especificada por <date>. El número de semana lleva como prefijo la letra W.

Tanto las funciones WEEK como ISOWEEK devuelven el número de semana para una fecha especificada, pero el formato del resultado es bastante diferente, y las dos funciones pueden gestionar la primera semana del nuevo año de forma diferente. Por

ejemplo, cuando se proporciona la fecha 2017-01-01, la función WEEK considera que la fecha forma parte de la **primera semana de 2017** y devuelve 1, mientras que ISOWEEK considera que la fecha forma parte de la **última semana de 2016** y devuelve 2016-W52.

ISOWEEK tiene 52 o 53 semanas completas, y la semana adicional se considera una semana bisiesta

Ejemplo

El siguiente ejemplo devuelve el valor 2011-W22 para los números de año y semana ISO de la fecha especificada:

```
SELECT ISOWEEK (TO_DATE('2011-05-30', 'YYYY-MM-DD')) "isoweek" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función JSON_QUERY (JSON)

Extrae texto JSON de un elemento de contexto JSON mediante una expresión de ruta SQL/JSON.

Sintaxis

```
JSON_QUERY(  
  <JSON_API_common_syntax>  
  [ <JSON_output_clause> ]  
  [ <JSON_query_wrapper_behavior> ]  
  [ <JSON_query_empty_behavior> ON EMPTY ]  
  [ <JSON_query_error_behavior> ON ERROR ]  
)
```

Elementos de sintaxis

<JSON_API_common_syntax>

Especifica un elemento de contexto JSON y una vía de acceso al elemento de contexto mediante una sintaxis común para la API JSON.

<JSON_API_common_syntax> ::= <JSON_context_item>, <JSON_path_specification>

<JSON_context_item>

Especifica el documento JSON en el que se va a operar, como una columna de tabla, cadena, json o expresión de ruta de la colección.

<JSON_path_Specification>

Especifica la vía de acceso a <JSON_context_item>.

<JSON_path_specification> ::= <JSON_path_mode> <JSON_path_wff>

<JSON_path_mode> ::= STRICT | LAX

Si se produce un error estructural dentro de una expresión de filtro JSON y <JSON_path_mode> se fija en STRICT, se aplica el tratamiento de errores de una expresión de filtro JSON. De lo contrario, un error estructural es un error no controlado.

Cuando la vía de acceso se fija en LAX, se produce una de las siguientes opciones:

- Si una operación requiere una matriz SQL/JSON pero el operando no es una matriz SQL/JSON, el operando se ajusta a una matriz SQL/JSON antes de realizar la operación.
- Si una operación requiere algo que no sea una matriz SQL/JSON, pero el operando es una matriz SQL/JSON, entonces el operando se desenvuelve convirtiendo sus elementos en una secuencia SQL/JSON antes de realizar la operación.

Los índices de matriz empiezan a partir de 0, en lugar de 1 (el estándar SQL).

Si aún hay un error estructural después de aplicar estas resoluciones, el resultado es una secuencia SQL/JSON vacía.

`<JSON_path_wff>` indica una ruta JSON real (por ejemplo, '\$.item1'). `<JSON_path_Specification>` no utiliza comillas dobles.

`<JSON_output_clause>`

Especifica la salida creada por la función JSON_QUERY.

`<JSON_output_clause> ::= RETURNING <data_type>`

`<tipo_datos>`

Especifica el tipo de datos que se debe fijar como el tipo de devolución de la función QUERY JSON. Tipos de datos admitidos: NVARCHAR(`<longitud>`) y JSON.

`<JSON_query_wrapper_behavior> WRAPPER`

Especifica el comportamiento de envoltura de la consulta JSON.

`<JSON_query_wrapper_behavior> WRAPPER`

`<JSON_query_wrapper_behavior> ::=`
 WITHOUT [ARRAY]
 | WITH [CONDITIONAL | UNCONDITIONAL] [ARRAY]

El valor predeterminado es SIN ARRAY WRAPPER. Sin embargo, si se especifica WITH, el valor predeterminado es UNCONDITIONAL ARRAY WRAPPER.

`<JSON_query_empty_behavior> ON EMPTY`

Especifica el comportamiento de la función si los datos relacionados no están en el elemento de contexto. El valor predeterminado es NULO EN VACÍO.

`<JSON_query_empty_behavior> ON EMPTY`

`<JSON_query_empty_behavior> ::=`
 ERROR
 | NULL
 | EMPTY ARRAY
 | EMPTY OBJECT

ERROR EN VACÍO devuelve un error si los datos relacionados no están en el elemento de contexto. NULL ON EMPTY devuelve un valor NULL si los datos relacionados no están en el elemento de contexto. EMPTY ARRAY ON EMPTY devuelve una matriz vacía si los datos relacionados no están en el elemento de contexto. EMPTY OBJECT ON EMPTY devuelve un objeto JSON vacío si los datos relacionados no están en el elemento de contexto.

`<JSON_query_error_behavior> ON ERROR`

Especifica el comportamiento de la función cuando la consulta genera un error. El valor predeterminado es NULL ON ERROR.

`<JSON_query_error_behavior> ON ERROR`

```
<JSON_query_error_behavior> ::=
ERROR
| NULL
| EMPTY ARRAY
| EMPTY OBJECT
```

ERROR ON ERROR devuelve un error si el resultado de la función incluye un error. NULL ON ERROR devuelve un NULL si el resultado de la función incluye un error. EMPTY ARRAY ON ERROR devuelve una matriz vacía si el resultado de la función incluye un error. EMPTY OBJECT ON ERROR devuelve un objeto JSON vacío si el resultado de la función incluye un error.

Descripción

Extrae texto JSON de un elemento de contexto JSON mediante una expresión de ruta SQL/JSON.

Los siguientes tokens se admiten en *<JSON_API_common_sintaxis>*:

Token	Descripción	Ejemplo
\$	El elemento de contexto (el primer argumento de la función).	'\$'
.	El miembro de un objeto.	'\$.item.description'
[]	Notación de paréntesis: - Selector de nombre: "<string>" o '<string>' - Selector de índice: <array-index> (, <array-index>) - Selector de sector de matriz: <start-index>:<end-index> (el elemento del índice final no está incluido en la selección) - Selector "a": <start-index> a <end-index> (el elemento del índice final está incluido en la selección) - Selector comodín: *	'\$[1]' '\$.item.list[1]' '\$["item"]' '\$[1:3]' = '\$[1,2]' '\$[1 to 3]' = '\$[1,2,3]' '\$[1:-1]' '\$.item[*]'
*	El carácter comodín.	'\$.*.description' '\$.item.list[*]'

Ejemplo

La siguiente consulta devuelve el valor {"item1":1,"item2":2,"item3":3}.

```
SELECT JSON_QUERY('{"item1":1, "item2":2, "item3":3}', '$') AS JSONQUERY FROM DUMMY;
```

La siguiente consulta devuelve el valor [1].

```
SELECT JSON_QUERY('{"item1":1, "item2":2, "item3":3}', '$.item1' WITH WRAPPER ) AS JSONQUERY FROM I
```

Información relacionada

[Expresiones](#)

Función JSON_TABLE (JSON)

Consulta un texto JSON y lo presenta como una tabla relacional.

Sintaxis

```
JSON_TABLE(  
  <JSON_API_common_syntax>  
  <JSON_table_columns_clause>  
  [ <JSON_table_error_behavior> ON ERROR ]  
)
```

Elementos de sintaxis

<JSON_API_common_syntax>

Especifica un elemento de contexto JSON y una vía de acceso al elemento de contexto mediante una sintaxis común para la API JSON.

```
<JSON_API_common_syntax> ::= <JSON_context_item>, <JSON_path_specification>
```

<JSON_context_item>

Especifica el documento JSON en el que se va a operar, como una columna de tabla o una cadena. No se admiten las colecciones.

<JSON_path_Specification>

Especifica la vía de acceso a <JSON_context_item>.

```
<JSON_path_specification> ::= <JSON_path_mode> <JSON_path_wff>
```

```
<JSON_path_mode> ::= STRICT | LAX
```

Dependiendo de la ubicación en la gramática, hay tres tipos de expresiones de ruta JSON:

Row pattern path expression

Se utiliza para producir una secuencia SQL/JSON, con un elemento SQL/JSON para cada fila de la tabla de salida.

Column pattern path expression

Se utiliza para buscar la columna dentro del elemento SQL/JSON actual producido por el patrón de filas.

Nested columns pattern path expression

Se utiliza para anular el anidamiento de objetos/matrices JSON anidados (incluso profundamente) en una invocación en lugar de encadenar varias expresiones JSON_TABLE en la sentencia SQL.

Por ejemplo:

Tipo de expresión de vía de acceso	Sintaxis de ejemplo
Expresión de ruta de patrón de fila	'lax \$'

Tipo de expresión de vía de acceso	Sintaxis de ejemplo
Expresión de ruta de patrón de columna	'laxo \$.name'
	'laxo \$.type'
	"lax \$.number"
Expresión de ruta de patrón anidado	'laxo \$.phoneNumber[*]'

Los tipos de error generados por la *<JSON_path_Specification>* son:

- un error de conversión de entrada (por ejemplo, un documento JSON no válido; no se puede analizar sintácticamente)
- un error devuelto por el motor PATH, que evalúa expresiones de ruta JSON
- un error estructural que significa que no hay ninguna ruta coincidente en el documento JSON (por ejemplo, expresión de ruta '\$.ab' frente a {"cd" : 1})
 - Si se produce un error estructural dentro de una expresión de filtro JSON y *<JSON_path_mode>* se fija en STRICT, se aplica el tratamiento de errores de una expresión de filtro JSON. De lo contrario, un error estructural es un error no controlado.
 - Cuando la vía de acceso se fija en LAX, se produce una de las siguientes opciones:
 - Si una operación requiere una matriz SQL/JSON pero el operando no es una matriz SQL/JSON, el operando se ajusta a una matriz SQL/JSON antes de realizar la operación.
 - Si una operación requiere algo que no sea una matriz SQL/JSON, pero el operando es una matriz SQL/JSON, entonces el operando se desenvuelve convirtiendo sus elementos en una secuencia SQL/JSON antes de realizar la operación.
 - Los índices de matriz empiezan a partir de 0, en lugar de 1 (el estándar SQL).
 - Si aún hay un error estructural después de aplicar estas resoluciones, el resultado es una secuencia SQL/JSON vacía.

<JSON_path_wff> indica una ruta JSON real (por ejemplo, '\$.item1'). *<JSON_path_Specification>* no utiliza comillas dobles.

<JSON_table_columns_clause>

Especifica las columnas que se crean.

```
<JSON_table_columns_clause> ::=
  COLUMNS ( <JSON_table_column_definition> [,... ] )

<JSON_table_column_definition> ::=
<JSON_table_ordinality_column_definition>
| <JSON_table_regular_column_definition>
| <JSON_table_formatted_column_definition>
| <JSON_table_nested_columns>
```

<JSON_table_column_Definition>

Define las columnas generadas.

<JSON_table_ordinality_column_Definition>

Define una columna de ordinalidad. Una columna de ordinalidad es similar a una columna definida mediante la función de ventana ROW_NUMBER.

```
<JSON_table_ordinality_column_definition> ::= <column_name> FOR ORDINALITY
```

<JSON_table_regular_column_Definition>

Define las columnas normales. Cada resultado (fila) de una columna de tabla JSON normal es equivalente a un resultado de función JSON_VALUE.

```
<JSON_table_regular_column_definition> ::= <column_name> <data_type>
PATH <JSON_table_column_path_specification>
[ <JSON_table_column_empty_behavior> ON EMPTY ]
[ <JSON_table_column_error_behavior> ON ERROR ]
```

```
<JSON_table_column_path_specification> ::= <JSON_path_specification>
```

```
<JSON_table_column_empty_behavior> ::=
ERROR
| NULL
| DEFAULT <value_expression>
```

```
<JSON_table_column_error_behavior> ::=
ERROR
| NULL
| DEFAULT <value_expression>
```

<nombre_columna>

Especifica el nombre de la columna.

<tipo_datos>

Especifica el tipo de datos.

```
<data_type> ::=
BIGINT
| DATE
| DECIMAL
| DOUBLE
| INT
| NVARCHAR (<int_const>)
| SECONDDATE
| SMALLDECIMAL
| TIME
| TIMESTAMP
```

<JSON_table_column_path_Specification>

Especifica la ruta JSON que especifica de qué valor JSON se extrae el elemento de contexto JSON.

<JSON_table_column_empty_behavior> EN VACÍO

Especifica el comportamiento de la función cuando la columna creada está vacía. El valor predeterminado es NULO EN VACÍO.

ERROR EN VACÍO devuelve un error cuando la columna creada está vacía. NULL ON EMPTY devuelve un valor NULL cuando la columna creada está vacía. DEFAULT <valor_expresión> ON EMPTY devuelve <expresión_valor> cuando la columna creada está vacía.

<JSON_table_column_error_behavior> EN ERROR

Especifica el comportamiento de la función cuando se produce un error durante la creación de la columna. El valor predeterminado es NULL ON ERROR.

ERROR ON ERROR produce un error si el resultado de la función incluye un error. NULL ON ERROR devuelve un NULL si el resultado de la función incluye un error. DEFAULT <valor_expresión> ON ERROR devuelve <expresión_valor> si el resultado de la función incluye un error.

<JSON_table_formatted_column_Definition>

Especifica la definición de columna para columnas con formato en las que los registros de la columna se formatean con sintaxis JSON. Cada resultado (fila) de una columna de tabla JSON normal es equivalente a un resultado de función JSON_QUERY.

```

<JSON_table_formatted_column_definition> ::=
  <column_name> <data_type>
  FORMAT <JSON_representation>
  PATH <JSON_table_column_path_specification>
  [ <JSON_table_formatted_column_wrapper_behavior> WRAPPER ]
  [ <JSON_table_formatted_column_empty_behavior> ON EMPTY ]
  [ <JSON_table_formatted_column_error_behavior> ON ERROR ]

```

<nombre_columna>

Especifica el nombre de la columna.

<tipo_datos>

Especifica el tipo de datos que se debe fijar como el tipo de retorno de la función. Los tipos admitidos son: NVARCHAR.

<Representación_JSON>

Especifica la codificación JSON que se va a utilizar.

```

<JSON_representation> ::=
JSON
| JSON ENCODING { UTF8 }
| JSON ENCODING { UTF16 }
| JSON ENCODING { UTF32 }

```

<JSON_table_column_path_Specification>

Especifica la ruta JSON que especifica de qué valor JSON se extrae el elemento de contexto JSON.

<JSON_table_formatted_column_wrapper_behavior> WRAPPER

Especifica el comportamiento de envoltura de la columna formateada.

```

<JSON_table_formatted_column_wrapper_behavior> ::=
WITHOUT [ ARRAY ]
| WITH [ CONDITIONAL | UNCONDITIONAL ] [ ARRAY ]

```

Cuando se especifica WITHOUT [ARRAY] WRAPPER, la columna con formato no se representa como una matriz JSON. Cuando se especifica WITH [CONDITIONAL | UNCONDITIONAL] [ARRAY] WRAPPER, la columna con formato se establece con una matriz condicional/incondicional. Con una matriz condicional, el resultado se formatea como matriz JSON si el resultado no es ni una matriz JSON ni un objeto JSON. Con una matriz incondicional, el resultado se formatea como matriz JSON si el resultado no es un objeto JSON. La diferencia entre envoltura incondicional y condicional es que la envoltura incondicional envuelve el resultado como una matriz JSON una vez más cuando el resultado es una matriz JSON.

<JSON_table_formatted_column_empty_behavior> EN VACÍO

Especifica el comportamiento de JSON si los datos relacionados no están en el elemento de contexto.

```

<JSON_table_formatted_column_empty_behavior> ::=
ERROR
| NULL
| EMPTY ARRAY
| EMPTY OBJECT

```

ERROR EN VACÍO devuelve un error si los datos relacionados no están en el elemento de contexto. NULL ON EMPTY devuelve un valor NULL si los datos relacionados no están en el elemento de contexto. EMPTY ARRAY ON EMPTY devuelve una matriz vacía si los datos relacionados no están en el elemento de contexto. EMPTY OBJECT ON EMPTY devuelve un objeto JSON vacío si los datos relacionados no están en el elemento de contexto.

<JSON_table_formatted_column_error_behavior> EN ERROR

Especifica el comportamiento cuando la columna formateada emite un error.

```
<JSON_table_formatted_column_error_behavior> ::=
ERROR
| NULL
| EMPTY ARRAY
| EMPTY OBJECT
```

ERROR ON ERROR devuelve un error si la función no devuelve ningún resultado. NULL ON ERROR devuelve un NULL si la función no devuelve ningún resultado. ARRAY EN VACÍO EN ERROR devuelve una matriz vacía si el resultado de la columna con formato está vacío. VACÍO OBJETO EN ERROR devuelve un objeto JSON vacío si el resultado de la columna con formato está vacío.

<JSON_table_nested_column>

Define las columnas anidadas.

```
<JSON_table_nested_columns> ::= NESTED [ PATH ] <JSON_table_nested_path_specification>
<JSON_table_columns_clause>

<JSON_table_nested_path_specification> ::= <JSON_path_specification>
```

- <JSON_table_columns_los_cláusula>: Especifica las columnas que se crean con la función.
- <JSON_path_Specification>: la ruta JSON que especifica qué valor JSON del elemento de contexto JSON se extrae.

<JSON_table_error_behavior>

Especifica el comportamiento de la función cuando se produce un error. El comportamiento predeterminado es VACÍO EN ERROR.

```
<JSON_table_error_behavior> ::= { ERROR | EMPTY }
```

ERROR

Si el resultado de la función incluye un error, se devuelve el error.

VACÍO

Si el resultado de la función incluye un error, devuelve un resultado vacío.

Descripción

Consulta un texto JSON y lo presenta como una tabla relacional.

Todos los errores y el tratamiento de vacíos que resultan de la evaluación de la expresión de la vía de acceso del patrón de columna, así como los errores de conversión de salida, se gestionan mediante <JSON_table_column_empty_behavior>, <JSON_table_formatted_column_empty_behavior> ON EMPTY y ON <JSON_table_formatted_column_ERor_behavior>.

Todos los errores resultantes de la evaluación de la expresión del patrón de fila y la evaluación de la expresión de la vía de acceso del patrón de las columnas anidadas se gestionan mediante <comportamiento JSON_table_error>.

Los siguientes tokens se admiten en <JSON_API_common_sintaxis>:

Token	Descripción	Ejemplo
\$	El elemento de contexto (el primer argumento de la función).	'\$ '
.	El miembro de un objeto.	'\$.item.description '
[]	Notación de paréntesis:	'\$[1] ' '\$.item.list[1] '

Token	Descripción	Ejemplo
	- Selector de nombre: "<string>" o '<string>' - Selector de índice: <array-index> (, <array-index>) - Selector de sector de matriz: <start-index>:<end-index> (el elemento del índice final no está incluido en la selección) - Selector "a": <start-index> a <end-index> (el elemento del índice final está incluido en la selección) - Selector comodín: *	'\$["item"]' '\$[1:3]' = '\$[1,2]' '\$[1 to 3]' = '\$[1,2,3]' '\$[1:-1]' '\$.item[*]'
*	El carácter comodín.	'\$.*.description' '\$.item.list[*]'

Ejemplo

Los siguientes ejemplos utilizan la tabla creada a continuación:

```
CREATE ROW TABLE T1 (A INT, B NVARCHAR(5000));
INSERT INTO T1 VALUES (1, '
{
  "PONumber": 1,
  "Reference": "BSMITH-74635645",
  "Requestor": "Barb Smith",
  "User": "BSMITH",
  "CostCenter": "A50",
  "ShippingInstructions":
    {
      "name": "Barb Smith",
      "Address":
        {
          "street": "100 Fairchild Ave",
          "city": "San Diego",
          "state": "CA",
          "zipCode": 23345,
          "country": "USA"
        },
      "Phone": [{ "type": "Office", "number": "519-555-6310"}]
    },
  "SpecialInstructions": "Surface Mail",
  "LineItems": [
    {"ItemNumber": 1, "Part": {"Description": "Basic Kit", "UnitPrice": 19.95, "UPCCCode": "12345"},
    {"ItemNumber": 2, "Part": {"Description": "Base Kit 2", "UnitPrice": 29.95, "UPCCCode": "67890"},
    {"ItemNumber": 3, "Part": {"Description": "Professional", "UnitPrice": 39.95, "UPCCCode": "13579"},
    {"ItemNumber": 4, "Part": {"Description": "Enterprise", "UnitPrice": 49.95, "UPCCCode": "24680"},
    {"ItemNumber": 5, "Part": {"Description": "Unlimited", "UnitPrice": 59.95, "UPCCCode": "09876"}
  ]
}
');
```

El siguiente ejemplo selecciona de una columna de ordinalidad y una columna regular:

```
SELECT JT.*
FROM JSON_TABLE(T1.B, '$.LineItems[*]'
COLUMNS
(
```

```

    RN FOR ORDINALITY,
    ITEM_NUMBER INT PATH '$.ItemNumber',
    UPC_CODE BIGINT PATH '$.Part.UPCCode'
)
) AS JT;
```

RN	ITEM_NUMBER	UPC_CODE
1	1	73649587162
2	2	83600229374
3	3	33298003521
4	4	91827739856
5	5	22983303876

El siguiente ejemplo selecciona de una columna con formato:

```

SELECT *
FROM JSON_TABLE(T1.B, '$.ShippingInstructions'
COLUMNS
    (
        PHONE NVARCHAR(50) FORMAT JSON PATH '$.Phone'
    )
) AS JT;
```

TELÉFONO
[{"número":"519-555-6310","tipo":"Oficina"}]

El siguiente ejemplo selecciona de una columna anidada:

```

SELECT *
FROM JSON_TABLE(T1.B, '$.ShippingInstructions'
COLUMNS
    (
        NESTED PATH '$.Address'
        COLUMNS
            (
                STREET NVARCHAR(50) PATH '$.street',
                CITY NVARCHAR(50) PATH '$.city'
            )
        )
    )
) AS JT;
```

CALLE	CIUDAD
100 Fairchild Ave	San Diego

El siguiente ejemplo selecciona de una columna de ordinalidad con columnas anidadas:

```

SELECT *
FROM JSON_TABLE(T1.B, '$'
COLUMNS
    (
        RN FOR ORDINALITY,
        USER_NAME NVARCHAR(20) PATH '$.User',
        NESTED PATH '$.LineItems[1,2]'
        COLUMNS
```

```
(
    ORDER_NUMBER FOR ORDINALITY,
    ITEM_NUMBER INT PATH '$.ItemNumber',
    QUANTITY INT PATH '$.Quantity'
)
) AS JT;
```

RN	USER_NAME	ORDER_NUMBER	ITEM_NUMBER	CANTIDAD
1	BSMITH	1	2	1
1	BSMITH	2	3	8

El siguiente ejemplo demuestra la diferencia al especificar CONDITIONAL ARRAY WRAPPER frente a UNCONDITIONAL ARRAY WRAPPER al devolver una matriz JSON:

```
CREATE ROW TABLE r1 ( a INT, b NVARCHAR(5000));
INSERT INTO r1 VALUES (1, '{"menu": {"header": "SVG Viewer","items": [{"id": "Open"}, {"id": "OpenNew", "label": "Open New"}, {"id": "OpenRecent", "label": "Open Recent"}, {"id": "Print", "label": "Print"}, {"id": "Quit", "label": "Quit"}]}');
```

Sentencia SELECT	Valor devuelto
SELECT JSON_QUERY(B, '\$.menu.items' WITH UNCONDITIONAL ARRAY WRAPPER) FROM r1 WITH HINT(IGNORE_PLAN_CACHE);	[[{"id": "Open"}, {"id": "OpenNew", "label": "Open New"}, null, {"id": "OpenRecent", "label": "Open Recent"}, {"id": "Print", "label": "Print"}, {"id": "Quit", "label": "Quit"}]]
SELECT JSON_QUERY(B, '\$.menu.items' WITH CONDITIONAL ARRAY WRAPPER) FROM r1;	[{"id": "Open"}, {"id": "OpenNew", "label": "Open New"}, null, {"id": "OpenRecent", "label": "Open Recent"}, {"id": "Print", "label": "Print"}, {"id": "Quit", "label": "Quit"}]

El siguiente ejemplo muestra un ejemplo de ERROR ON ERROR, que da como resultado un error que indica que los datos de '\$.User' no se pueden analizar en un tipo de datos entero.

```
SELECT *
FROM JSON_TABLE(T1.B, '$'
COLUMNS
(
    RN FOR ORDINALITY,
    USER_NAME INT PATH '$.User' ERROR ON ERROR
)
ERROR ON ERROR
) AS JT;
```

El siguiente ejemplo muestra un ejemplo de EMPTY ON ERROR. El comportamiento predeterminado de JSON_table_error_behavior es EMPTY ON ERROR. Devuelve un resultado de error en lugar de lanzar vacío cuando JSON_table_column_error_behavior es ERROR ON ERROR o JSON_table_column_empty_behavior es ERROR ON EMPTY.

```
SELECT *
FROM JSON_TABLE(T1.B, '$'
COLUMNS
(
    RN FOR ORDINALITY,
    USER_NAME INT PATH '$.User' ERROR ON ERROR
)
) AS JT;
```

Información relacionada

Función JSON_VALUE (JSON)

Extrae un valor SQL de un tipo predefinido de un valor JSON.

Sintaxis

```
JSON_VALUE(  
  <JSON_API_common_syntax>  
  [ <JSON_returning_clause> ]  
  [ <JSON_value_empty_behavior> ON EMPTY ]  
  [ <JSON_value_error_behavior> ON ERROR ]  
)
```

Elementos de sintaxis

<JSON_API_common_syntax>

Especifica un elemento de contexto JSON y una vía de acceso al elemento de contexto mediante una sintaxis común para la API JSON.

<JSON_API_common_syntax> ::= <JSON_context_item>, <JSON_path_specification>

<JSON_context_item>

Especifica el documento JSON en el que se va a operar, como una columna de tabla, cadena, json o expresión de ruta de la colección.

<JSON_path_specification>

Especifica la vía de acceso a <JSON_context_item>.

<JSON_path_specification> ::= <JSON_path_mode> <JSON_path_wff>

<JSON_path_mode> ::= STRICT | LAX

Cuando <JSON_path_mode> se fija en STRICT, si el error estructural se produce dentro de una expresión de filtro JSON, se aplica el tratamiento de errores de una expresión de filtro JSON. De lo contrario, un error estructural es un error no controlado.

Cuando la vía de acceso se fija en LAX, se produce una de las siguientes opciones:

- Si una operación requiere una matriz SQL/JSON pero el operando no es una matriz SQL/JSON, el operando se ajusta a una matriz SQL/JSON antes de realizar la operación.
- Si una operación requiere algo que no sea una matriz SQL/JSON, pero el operando es una matriz SQL/JSON, entonces el operando se desenvuelve convirtiendo sus elementos en una secuencia SQL/JSON antes de realizar la operación.

Los índices de matriz empiezan a partir de 0, en lugar de 1 (el estándar SQL).

Si aún hay un error estructural después de aplicar estas resoluciones, el resultado es una secuencia SQL/JSON vacía.

<JSON_path_wff> indica una ruta JSON real (por ejemplo, '\$.item1'). <JSON_path_specification> no utiliza comillas dobles.

<JSON_returning_clause>

Define el tipo de datos del resultado.

```
<JSON_returning_clause> ::= RETURNING <data_type>

<data_type> ::=
INTEGER
| BOOLEAN
| BIGINT
| DECIMAL
| NVARCHAR(<integer>)
| NCLOB
```

<JSON_value_empty_behavior> ON EMPTY

Especifica el comportamiento de la función cuando los datos relacionados no están en el elemento de contexto. El valor predeterminado es NULL ON EMPTY

```
<JSON_value_empty_behavior> ON EMPTY

<JSON_value_empty_behavior> ::=
ERROR
| NULL
| DEFAULT <value_expression>
```

ERROR EN VACÍO devuelve un error si los datos relacionados no están en el elemento de contexto. NULL ON EMPTY devuelve un valor NULL si los datos relacionados no están en el elemento de contexto. DEFAULT <valor_expresión> ON EMPTY devuelve <expresión_valor> si los datos relacionados no están en el elemento de contexto.

<JSON_value_error_behavior> ON ERROR

Especifica el comportamiento de la función JSON_VALUE cuando se produce un error. El valor predeterminado es NULL ON ERROR.

```
<JSON_value_error_behavior> ON ERROR

<JSON_value_error_behavior> ::=
ERROR
| NULL
| DEFAULT <value_expression>
```

ERROR ON ERROR devuelve un error si los resultados de la función incluyen un error. NULL ON ERROR devuelve un NULL si los resultados de la función incluyen un error. DEFAULT <valor_expresión> ON ERROR devuelve <expresión_valor> si los resultados de la función incluyen un error.

Descripción

Extrae un valor SQL de un tipo predefinido de un valor JSON.

Los siguientes tokens se admiten en <JSON_API_common_sintaxis>:

Token	Descripción	Ejemplo
\$	El elemento de contexto (el primer argumento de la función).	'\$'
.	El miembro de un objeto.	'\$.item.description'
[]	Notación de paréntesis: Selector de nombre: "<string>" o '<string>'	'\$[1]' '\$.item.list[1]' '\$["item"]' '\$[1:3]' = '\$[1,2]'

Token	Descripción	Ejemplo
	- Selector de índice: <array-index> (, <array-index>) - Selector de sector de matriz: <start-index>:<end-index> (el elemento del índice final no está incluido en la selección) - Selector "a": <start-index> a <end-index> (el elemento del índice final está incluido en la selección) - Selector comodín: *	'\$[1 to 3] = '\$[1,2,3]' '\$[1:-1]' '\$.item[*]'
*	El carácter comodín.	'\$.*.description' '\$.item.list[*]'

Ejemplo

La siguiente sentencia devuelve un valor de 10:

```
SELECT JSON_VALUE('{"item1":10}', '$.item1') AS "value" FROM DUMMY;
```

La siguiente sentencia devuelve un valor de 5:

```
SELECT JSON_VALUE(
  '{"item1":{"sub1":10}, "item2":{"sub2":5}, "item3":{"sub3":7}}',
  '$.*.sub2'
) AS "value" FROM DUMMY;
```

La siguiente sentencia devuelve un valor de 0:

```
SELECT JSON_VALUE('[0, 1, 2, 3]', '$[0]') AS "value" FROM DUMMY;
```

Las siguientes sentencias ilustran cómo JSON_VALUE gestiona los valores que faltan y los errores, en función del comportamiento especificado.

La siguiente sentencia devuelve un valor predeterminado cuando falta el valor solicitado (el objeto no contiene "apellido"):

```
SELECT JSON_VALUE(
  '{"firstname":"John"}',
  '$.lastname' DEFAULT 'No last name found' ON EMPTY
) AS "Last Name" FROM DUMMY;
```

La siguiente sentencia emite un error de conversión de tipo cuando el valor extraído no se puede convertir al tipo SQL solicitado:

```
SELECT JSON_VALUE(
  '{"item":"string"}',
  '$.item' RETURNING DECIMAL ERROR ON ERROR
```

```
) AS "Item" FROM DUMMY;
```

La siguiente sentencia emite un error cuando falta el valor solicitado (el objeto no contiene "apellido") y la vía de acceso se evalúa en modo STRICT:

```
SELECT JSON_VALUE(  
    '{"firstname":"John"}',  
    'strict $.lastname' ERROR ON ERROR  
) AS "Last Name" FROM DUMMY;
```

Información relacionada

[Expresiones](#)

Función L2DISTANCE (vector)

Calcula la distancia L2 de dos vectores.

Sintaxis

L2DISTANCE(<vector1>, <vector2>)

Elementos de sintaxis

<vector1>

Especifica el primer vector.

<vector2>

Especifica el segundo vector.

Descripción

Calcula la distancia L2 en el espacio de <vector1> y <vector2>. Ambos vectores deben tener la misma dimensión.

El resultado es un número DOUBLE mayor o igual que 0. Cuanto mayor sea el resultado, mayor será la distancia entre los vectores. Cuanto menor sea el resultado, menor será la distancia entre los vectores.

Para dos vectores **u** y **v**, la distancia L2 se define matemáticamente como sigue:

$$S_{L2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{v} - \mathbf{u}\| = \sqrt{\sum_{i=1}^n (v_i - u_i)^2}$$

Ejemplo

El siguiente ejemplo devuelve 5.0:

```
SELECT L2DISTANCE(  
    TO_REAL_VECTOR('[2, 3, 5]'),
```

```
TO_REAL_VECTOR(' [6, 6, 5] '))  
FROM DUMMY;
```

El siguiente ejemplo devuelve 5.0:

```
SELECT L2DISTANCE(  
    TO_HALF_VECTOR(' [2, 3, 5] '),  
    TO_HALF_VECTOR(' [6, 6, 5] '))  
FROM DUMMY;
```

Información relacionada

[Tipos de datos vectoriales](#)

Función L2NORM (vector)

Calcula la longitud (norma euclidiana) de un vector.

Sintaxis

L2NORM(<vector>)

Elementos de sintaxis

<vector>

Especifica el vector.

Descripción

Calcula la longitud (norma euclidiana) de <vector>. Esto puede ayudar a obtener información sobre un conjunto de datos vectoriales y a encontrar valores atípicos.

La longitud (norma euclidiana) de un vector **v** matemáticamente definido como sigue:

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n v_i^2}$$

Ejemplos

Los siguientes ejemplos devuelven 5.0:

```
SELECT L2NORM(TO_REAL_VECTOR(' [3, 4, 0] ')) FROM DUMMY;  
SELECT L2NORM(TO_HALF_VECTOR(' [3, 4, 0] ')) FROM DUMMY;
```

Información relacionada

[Tipos de datos vectoriales](#)

Función L2NORMALIZE (vector)

Cambia la longitud de un vector a 1 mientras se mantiene la dirección.

Sintaxis

L2NORMALIZE(<vector>)

Elementos de sintaxis

<vector>

Especifica el vector.

Descripción

Cambia la longitud de <vector> a 1 mientras se mantiene la dirección. Esto puede ayudar a obtener resultados de precisión de punto flotante más robustos.

La normalización de un vector $\mathbf{v}$ se define matemáticamente como sigue:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

Ejemplos

El siguiente ejemplo devuelve [0.42426407, 0.56568545, 0.7071068]:

```
SELECT TO_ARRAY(L2NORMALIZE(TO_REAL_VECTOR('[3, 4, 5]')) FROM DUMMY;
```

El siguiente ejemplo devuelve [0.4243164, 0.56591797, 0.70703125]:

```
SELECT TO_ARRAY(L2NORMALIZE(TO_HALF_VECTOR('[3, 4, 5]')) FROM DUMMY;
```

Información relacionada

[Tipos de datos vectoriales](#)

Función LAG (ventana)

Devuelve el valor de las filas de desplazamiento antes de la fila actual.

Sintaxis

```
LAG( <expression> [, <offset> [, <default_expr> ] ] ) <window_specification>
```

Elementos de sintaxis

<offset>

El <offset> debe ser no negativo y el valor predeterminado es 1.

Si el <offset> cruza los límites de la partición, se devuelve el valor <default_expr>. Si no se especifica <default_expr>, se devuelve un valor nulo. El <offset> y <default_expr> se evalúan en la fila actual.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La salida de la función LAG puede ser no determinista entre los valores de empate.

Ejemplos

```
CREATE TABLE T (class NVARCHAR(10), val INT, offset INT);
INSERT INTO T VALUES('A', 1, 1);
INSERT INTO T VALUES('A', 3, 3);
INSERT INTO T VALUES('A', 5, null);
INSERT INTO T VALUES('A', 5, 2);
INSERT INTO T VALUES('A', 10, 0);
INSERT INTO T VALUES('B', 1, 3);
INSERT INTO T VALUES('B', 1, 1);
INSERT INTO T VALUES('B', 7, 1);

SELECT class,
       val,
       offset,
       LEAD(val) OVER (PARTITION BY class ORDER BY val) AS lead,
       LEAD(val,offset,-val) OVER (PARTITION BY class ORDER BY val) AS lead2,
       LAG(val) OVER (PARTITION BY class ORDER BY val) AS lag,
       LAG(val,offset,-val) OVER (PARTITION BY class ORDER BY val) AS lag2
FROM T;
```

CLASE	VAL	OFFSET	LEAD	LEAD2	LAG	LAG2
A	1	1	3	3	nulo	-1
A	3	3	5	10	1	-3
A	5	nulo	5	-5	3	-5
A	5	2	10	-5	5	3
A	10	0	nulo	10	5	10
B	1	3	1	-1	nulo	-1
B	1	1	7	7	1	1
B	7	1	nulo	-7	1	1

Información relacionada

[Expresiones](#)

[Funciones de ventana y la especificación de ventana](#)

Función LAST_DAY (fecha y hora)

Devuelve la fecha del último día del mes que contiene la fecha especificada.

Sintaxis

LAST_DAY(<date>)

Descripción

Devuelve la fecha del último día del mes que contiene la fecha <date>.

Ejemplo

El siguiente ejemplo devuelve el valor 31/01/2010 (u otro formato como 31 de enero de 2010, según la configuración de visualización de la fecha):

```
SELECT LAST_DAY (TO_DATE('2010-01-04', 'YYYY-MM-DD')) "last day" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función LAST_VALUE (agregar)

Devuelve el valor del último elemento de una expresión. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

LAST_VALUE(<expression> <order_by_clause>)

Función de ventana:

LAST_VALUE(<expression> <order_by_clause>) <window_specification>

Elementos de sintaxis

<expression>

Especifica la expresión de los datos sobre los que operar.

<order_by_clause>

Especifica el orden de clasificación de las filas de entrada.

```
<order_by_clause> ::= ORDER BY <order_by_expression> [, <order_by_expression> [...] ]

<order_by_expression> ::=
  <column_name> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
  | <column_position> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]

<collate_clause> ::= COLLATE <collation_name>
```

<clausula colateral> especifica la intercalación que se va a utilizar para ordenar los valores en los resultados. <collate_cláusula> solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un sinónimo de NVARCHAR).

<collation_name> es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Devuelve el valor del último elemento en <expresión> como ordenado por <order_by_cláusula>.

Se devuelve NULL si el valor es NULL o si <expresión> está vacío.

La salida de la función LAST_VALUE puede ser no determinista entre los valores de vinculación.

Ejemplo

El siguiente ejemplo devuelve el último valor de la columna COL1 cuando COL2 ordena la tabla. La consulta devuelve 4.

```
CREATE ROW TABLE T (COL1 DOUBLE, COL2 DOUBLE);

INSERT INTO T VALUES(1, 1);
INSERT INTO T VALUES(4, 5);
INSERT INTO T VALUES(7, 3);

SELECT LAST_VALUE (COL1 ORDER BY COL2) FROM T;
```

Ejemplo de tipo de datos espaciales (ST_Geometry, ST_Point)

El siguiente ejemplo devuelve el último valor, POINT (4 4), en formato binario, cuando la tabla se ordena por ID:

```
CREATE TABLE TAB (ID INT, SHAPE ST_Geometry(4326));

INSERT INTO TAB VALUES(1, ST_GeomFromText('POINT(1 1)', 4326));
INSERT INTO TAB VALUES(2, ST_GeomFromText('POINT(2 2)', 4326));
INSERT INTO TAB VALUES(3, ST_GeomFromText('POINT(3 3)', 4326));
INSERT INTO TAB VALUES(4, ST_GeomFromText('POINT(4 4)', 4326));

SELECT LAST_VALUE(SHAPE ORDER BY ID) FROM TAB;
```

Para devolver un resultado como tipo de datos NVARCHAR, utilice la siguiente sentencia:

```
SELECT LAST_VALUE(CAST(SHAPE AS NVARCHAR) ORDER BY ID) FROM TAB;
```

Información relacionada

[Vista del sistema COLLATIONS](#)[Funciones de ventana y la especificación de ventana](#)[Funciones agregadas de ventana](#)[Agregar funciones](#)[SAP HANA Cloud, referencia espacial de base de datos de SAP HANA](#)

Función LCASE (cadena)

Convierte en minúsculas todos los caracteres de una cadena.

Sintaxis

LCASE(<string>)

Descripción

Convierte todos los caracteres de la cadena <cadena> en minúsculas.

La función LCASE es idéntica a la función LOWER.

Ejemplo

Este ejemplo convierte en minúsculas todos los caracteres de la cadena Test indicada y devuelve la prueba de valor.

```
SELECT LCASE ('Test') "lcase" FROM DUMMY;
```

Información relacionada

[Función LOWER \(cadena\)](#)[Tipos de datos de cadena de caracteres](#)

Función LEAD (ventana)

Devuelve el desplazamiento de las filas después de la fila actual. El <offset> debe ser no negativo y su valor predeterminado es 1.

Sintaxis

LEAD(<expression> [, <offset> [, <default_expr>]]) <window_specification>

Elementos de sintaxis

<offset>

Si el <offset> cruza los límites de la partición, se devuelve el valor <default_expr>. Si no se especifica <default_expr>, se devuelve un valor nulo. El <offset> y <default_expr> se evalúan en la fila actual.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para *<window_Specification>*, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La salida de la función LEAD puede ser no determinista entre los valores de empate.

Ejemplos

```
CREATE TABLE T (class NVARCHAR(10), val INT, offset INT);
INSERT INTO T VALUES('A', 1, 1);
INSERT INTO T VALUES('A', 3, 3);
INSERT INTO T VALUES('A', 5, null);
INSERT INTO T VALUES('A', 5, 2);
INSERT INTO T VALUES('A', 10, 0);
INSERT INTO T VALUES('B', 1, 3);
INSERT INTO T VALUES('B', 1, 1);
INSERT INTO T VALUES('B', 7, 1);

SELECT class,
       val,
       offset,
       LEAD(val) OVER (PARTITION BY class ORDER BY val) AS lead,
       LEAD(val, offset, -val) OVER (PARTITION BY class ORDER BY val) AS lead2,
       LAG(val) OVER (PARTITION BY class ORDER BY val) AS lag,
       LAG(val, offset, -val) OVER (PARTITION BY class ORDER BY val) AS lag2
FROM T;
```

CLASE	VAL	OFFSET	LEAD	LEAD2	LAG	LAG2
A	1	1	3	3	nulo	-1
A	3	3	5	10	1	-3
A	5	nulo	5	-5	3	-5
A	5	2	10	-5	5	3
A	10	0	nulo	10	5	10
B	1	3	1	-1	nulo	-1
B	1	1	7	7	1	1
B	7	1	nulo	-7	1	1

Información relacionada

[Expresiones](#)
[Funciones de ventana y la especificación de ventana](#)

Función LEAST (varios)

Devuelve el valor menor de dos argumentos especificados.

Sintaxis

```
LEAST(<argument_1> [, <argument_2>]...)
```

Descripción

Devuelve el valor menor de los argumentos. Si uno de los argumentos es NULL, el retorno es NULL.

Ejemplo

La siguiente consulta devuelve aa.

```
SELECT LEAST('aa', 'ab', 'ba', 'bb') "least" FROM DUMMY;
```

Función LEFT (cadena)

Devuelve el número especificado de caracteres de una cadena, empezando por el lado izquierdo.

Sintaxis

```
LEFT(<string>, <number>)
```

Elementos de sintaxis

<string>

Especifica la cadena en la que se operará.

<number>

Especifica la longitud de los caracteres a devolver, empezando por el lado izquierdo.

Descripción

Devuelve el primer <número> de caracteres desde el principio de <string>.

Devuelve un valor de cadena vacío si <número> es menor que 1.

Devuelve <string> (sin relleno en blanco) si el valor de <number> es mayor que la longitud de <string>.

Ejemplo

El siguiente ejemplo devuelve los tres caracteres situados más a la izquierda de la cadena Hello:

```
SELECT LEFT ('Hello', 3) "left" FROM DUMMY;
```

El siguiente ejemplo devuelve la cadena Hola porque el valor 10 supera la longitud de la cadena:

```
SELECT LEFT ('Hello', 10) "left" FROM DUMMY;
```

Función LENGTH (cadena)

Devuelve el número de caracteres de una cadena.

Sintaxis

```
LENGTH(<string>)
```

Descripción

Devuelve el número de caracteres de la cadena *<cadena>*.

Los caracteres Unicode de plano suplementario, cada uno de los cuales ocupa 6 bytes en la codificación CESU-8, se cuentan como dos caracteres.

Ejemplo

Este ejemplo devuelve el número de caracteres (14) incluidos en la cadena indicada:

```
SELECT LENGTH ('length of this') "length" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función LINEAR_APPROX (ventana)

Funciona en una serie completa para producir una nueva serie que sustituye los valores que faltan interpolando entre valores no NULL adyacentes y extrapolando cualquier valor nulo inicial o final.

Sintaxis

```
LINEAR_APPROX(<expression> [, <ModeArgument> [, <Value1Argument> [,  
    <Value2Argument>]]]) OVER ({ SERIES(...) [ <window_partition_by_clause> ]  
    [ <window_order_by_clause> ] | [ <window_partition_by_clause> ] <window_order_by_clause> } )  
  
<expression> ::= <identifier>
```

Elementos de sintaxis

<expression>

Este parámetro especifica la columna de datos de entrada.

<ModeArgument>

Este parámetro define el modo de extrapolación aplicado. La siguiente tabla enumera los valores posibles para *<ModeArgument>*.

```
<ModeArgument> ::= EXTRAPOLATION_NONE  
| EXTRAPOLATION_LINEAR  
| EXTRAPOLATION_CONSTANT
```

Modo	Descripción
EXTRAPOLATION_NONE	Evita la extrapolación de valores nulos iniciales y finales. La interpolación se realiza en los valores del medio. Valor predeterminado si se omite <i><ModeArgument></i>. No es necesario especificar los parámetros <i><Value1Argument></i> y <i><Value2Argument></i> al utilizar este modo. Por ejemplo, una entrada de [null, null, 1, 2, null, null, 5, null, null] devuelve [null, null, 1, 2, 3, 4, 5, null, null].
EXTRAPOLATION_LINEAR	Realiza una extrapolación lineal en la que <i><Value1Argument></i> es el valor mínimo y <i><Value2Argument></i> es el valor máximo. Durante la extrapolación de valores nulos iniciales y finales, esta función verifica que los resultados de la extrapolación estén dentro del rango de los valores mínimo y máximo; de lo contrario, los valores se sustituyen por el valor mínimo o máximo, lo que sea apropiado. Se emite una excepción si <i><Value1Argument></i> es mayor que <i><Value2Argument></i>.
EXTRAPOLATION_CONSTANT	Realiza una extrapolación lineal en la que los valores nulos iniciales y finales se sustituyen por los valores especificados por los parámetros <i><Value1Argument></i> y <i><Value2Argument></i>, respectivamente. Cuando <i><Value1Argument></i> no se especifica o es nulo, el primer valor no nulo se utiliza para sustituir los valores nulos iniciales. Cuando <i><Value2Argument></i> no se especifica o es nulo, se usa el último no nulo para reemplazar los valores nulos finales.

La interpolación lineal se puede aplicar a todos los tipos numéricos si se pueden convertir en un tipo doble largo, como INT, LONG, DOUBLE o FLOAT.

<Value1Argument> and <Value2Argument>

Estos parámetros definen argumentos para la extrapolación y pueden ser de cualquier tipo numérico.

<Value1Argument> ::= *<identifier>*
<Value2Argument> ::= *<identifier>*

<table_schema>

<table_schema> ::= [*<schema_name>*].*<table_name>*
<schema_name> ::= *<unicode_name>*
<table_name> ::= *<identifier>*

<table_schema> debe ser un nombre de tabla base y no un nombre de correlación.

<window_order_by_clause>

Determina el orden de clasificación. Esta expresión no debe contener valores NULL ni duplicados. Consulte la sintaxis de esta cláusula ubicada en el tema *Funciones de ventana*.

Descripción

Funciona en una serie completa para producir una nueva serie que sustituye los valores que faltan interpolando entre valores no NULL adyacentes y extrapolando cualquier valor nulo inicial o final.

Al utilizar esta función, se supone que la entrada es equidistante. La definición SERIES debe ser equidistante.

Cuando solo se incluyen valores nulos en la entrada, solo aparecen valores nulos en la salida a menos que los valores nulos se sustituyan por una extrapolación mediante el modo EXTRAPOLATION_CONSTANT.

Al utilizar la sintaxis SERIES, esta función utiliza las propiedades de la siguiente tabla para realizar una aproximación lineal.

Propiedad	Descripción
NO EQUIDISTENTE	Se produce un error si la serie no es equidistante.
ELEMENTOS FALTADOS PERMITIDOS	Cuando se especifica, debe haber exactamente una columna ORDER BY que sea compatible con el tipo de columna de período de serie y el valor INCREMENT BY.
PARTICIÓN POR	Si no se especifica, se usa la propiedad SERIES KEY de la sintaxis SERIES para construir PARTITION BY predeterminado.
ORDEN POR	Si no se especifica, la primera columna PERIOD se añade como ORDER BY.

Se ignoran otras propiedades SERIES, como MINVALUE y MAXVALUE.

Ejemplos

Aproximación lineal de datos de series dispersas

En el siguiente ejemplo, INCREMENT BY INTERVAL se fija en 1 MES, lo que significa que se espera como máximo una fila para cada mes. Aparte de los valores nulos, faltan valores durante varios meses en la tabla. La función LINEAR_APPROX siempre sustituye los valores nulos y no inserta nuevas líneas para los valores que faltan. Sin embargo, si la definición SERIES se especifica en la sentencia SELECT, LINEAR_APPROX tiene en cuenta los meses que faltan al calcular la pendiente entre dos valores no nulos.

```
CREATE COLUMN TABLE "SparseApproxTable" (ts_id NVARCHAR(20), date DAYDATE, val DOUBLE);

INSERT INTO "SparseApproxTable" VALUES('A','2013-11-01', null);
INSERT INTO "SparseApproxTable" VALUES('A','2014-01-01', null);
INSERT INTO "SparseApproxTable" VALUES('A','2014-02-05', 2);
INSERT INTO "SparseApproxTable" VALUES('A','2014-03-07', null);
INSERT INTO "SparseApproxTable" VALUES('A','2014-05-01', 5);
INSERT INTO "SparseApproxTable" VALUES('A','2014-07-27', 7);
INSERT INTO "SparseApproxTable" VALUES('A','2014-12-07', null);
INSERT INTO "SparseApproxTable" VALUES('A','2015-02-07', null);

SELECT LINEAR_APPROX(val, 'EXTRAPOLATION_LINEAR') OVER (SERIES (
    SERIES KEY(ts_id) EQUIDISTANT INCREMENT BY INTERVAL 1 MONTH
    MISSING ELEMENTS ALLOWED PERIOD FOR SERIES(date))
    PARTITION BY ts_id) AS approximated_value
FROM "SparseApproxTable";
```

APPROXIMATED_VALUE
-1
1
2
3
5
7
9.666666666666666

APPROXIMATED_VALUE
11

Aproximación lineal sin definición de serie

Si la definición SERIES no se hubiera especificado en el ejemplo anterior, el resultado habría sido diferente. Para la misma SparseApproxTable, que se ha rellenado anteriormente, el siguiente ejemplo no especifica una serie. El siguiente ejemplo devuelve:

APPROXIMATED_VALUE
-1.0
0,5
2.0
3.5
5.0
7.0
9.0
11.0

```
SELECT LINEAR_APPROX(val, 'EXTRAPOLATION_LINEAR') OVER (PARTITION BY ts_id ORDER BY DATE) AS approx
FROM "SparseApproxTable";
```

En la consulta de ejemplo anterior, se supone que la serie es densa y equidistante, por lo que la columna de fecha no se tiene en cuenta al calcular las aproximaciones.

Aproximación lineal de datos de serie

```
CREATE COLUMN TABLE "InterpolationTable" (TS_ID NVARCHAR(20), date DAYDATE, val DOUBLE);
INSERT INTO "InterpolationTable" VALUES('A','2013-09-30', 1);
INSERT INTO "InterpolationTable" VALUES('A','2013-10-01', 2);
INSERT INTO "InterpolationTable" VALUES('A','2013-10-02', null);
INSERT INTO "InterpolationTable" VALUES('A','2013-10-03', 10);

SELECT date,
       val,
       LINEAR_APPROX (val, 'EXTRAPOLATION_LINEAR') OVER (PARTITION BY TS_ID ORDER BY date) AS LINEAR_APPROX
FROM "InterpolationTable";
```

FECHA	VAL	MYRESULT
30 de septiembre de 2013	1	1
1 de octubre de 2013	2	2
2 de octubre de 2013	?	6
3 de octubre de 2013	10	10

Información relacionada

[Expresiones](#)[Funciones de ventana y la especificación de ventana](#)

Función LN (numérica)

Devuelve el logaritmo natural de un número.

Sintaxis

LN(<number>)

Descripción

Devuelve el logaritmo natural del número especificado.

Ejemplo

Este ejemplo devuelve el logaritmo natural del número 9, que es 2.1972245773362196:

```
SELECT LN (9) "ln" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función LOCALTOUTC (fecha y hora)

Un parámetro de cronomarcador que contiene la hora que se va a convertir entre UTC y la hora local.

Sintaxis

```
LOCALTOUTC (<time> [, <timezone> [, <timezone_dataset>[, <source_schema> ] ] ]  
[, <error_clause> ] )
```

Elementos de sintaxis

<time>

Especifica la hora que se debe convertir entre UTC y la hora local.

<time> ::= <timestamp>

<timezone>

Especifica el huso horario que define la hora local. Para obtener una lista de los husos horarios disponibles, consulte la vista del sistema TIMEZONES. Si el huso horario local no se especifica explícitamente, se utilizará el huso horario local del sistema SAP HANA.

```
<timezone> ::= <string_literal>
```

<timezone_dataset>

Especifica el conjunto de datos en el que buscar el huso horario indicado. Si se especifica, debe fijarse en `platform`, que busca en el conjunto de datos proporcionado por el sistema operativo.

```
<timezone_dataset> ::= { sap | platform }
```

- `sap` busca en el conjunto de datos en las tablas de definición de husos horarios. Para utilizar este valor, importe un conjunto de datos de huso horario (consulte la nota SAP [1791342](#) ). Si utiliza este valor sin importar un conjunto de datos, la función devuelve un error.
- `platform` busca en el conjunto de datos proporcionado por el sistema operativo. Este es el valor predeterminado.

<source_schema>

Especifica el nombre del esquema de datos de huso horario que se va a utilizar.

<error_clause>

Especifica el tratamiento de errores para la función. Si no se especifica, se utiliza el valor predeterminado `ERROR ON ERROR`.

```
<error_clause> ::=
{ ERROR ON ERROR
| NULL ON ERROR
| DEFAULT '<value>' ON ERROR }
```

Descripción

Convierte la hora local *<hora>* de un huso horario a la hora UTC (GMT).

Se desaconseja el uso de cronomarcadores locales; en su lugar, utilice horas UTC. El uso de horas locales o la conversión entre husos horarios locales puede requerir un tratamiento adicional en el código de aplicación.

Si se utiliza una columna como segundo argumento (*<huso horario>*) y la columna contiene una cadena vacía, no se devuelve ningún resultado y se genera un error.

Ejemplos

El siguiente ejemplo devuelve la hora UTC equivalente para la hora EST local especificada:

```
SELECT LOCALTOUTC (TO_TIMESTAMP('2012-01-01 01:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'EST') "localtoutc"
```

El siguiente ejemplo devuelve la hora UTC equivalente para la hora local especificada, utilizando el conjunto de datos de huso horario SAP:

```
SELECT LOCALTOUTC (TO_TIMESTAMP('2012-01-01 01:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'EST', 'sap') "localtoutc"
```

El siguiente ejemplo devuelve la hora UTC equivalente para la hora local actual, utilizando el esquema de datos predeterminado de huso horario `TTZZ_SYSTEM`:

```
SELECT LOCALTOUTC (CURRENT_TIMESTAMP, 'EST', 'sap', TTZZ_SYSTEM) "localtoutc" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor NULL ya que "INVALID" no es una zona horaria válida:

```
SELECT LOCALTOUTC( '2025-09-12 10:30:00', 'INVALID', 'sap' NULL ON ERROR) FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 01-01-2025 00:00:00 ya que "NO VÁLIDO" no es un huso horario válido:

```
SELECT LOCALTOUTC( '2025-09-12 10:30:00', 'INVALID', 'sap' DEFAULT '2025-01-01 00:00:00' ON ERROR) F
```

Información relacionada

[Función UTCTOLOCAL \(fecha y hora\)](#)

Función LOCATE (cadena)

Devuelve la posición de una subcadena dentro de una cadena.

Sintaxis

```
LOCATE( <haystack>, <needle>, [<start_position>] , [ <occurrences> ] )
```

Descripción

La función LOCATE devuelve la posición de una subcadena <aguja> dentro de una cadena <haystack>.

- Si <aguja> no se encuentra dentro de <pajar>, o si <ocurrencias> se establece en menos de 1, entonces se devuelve 0.
- Si <pajar>, <aguja> o <ocurrencias> es (explícitamente) NULL, entonces se devuelve NULL.
- Si no se especifica <ocurrencias>, se devuelve la primera posición coincidente. Una opción de 1 para <ocurrencias> es la misma que no especificarla, mientras que una opción de *n* devuelve la *n-ésima* coincidencia (o 0 si no se encuentran más coincidencias).
- Si <start_position> es un entero positivo, o 0, o no especificado, la dirección coincidente continúa de izquierda a derecha.
- Si <start_position> es negativo, la coincidencia comienza al final de <haystack> y continúa en la dirección inversa (de derecha a izquierda). Por ejemplo:

Extracto	Devoluciones
SELECT LOCATE('AAA', 'A', -1) FROM "DUMMY";	3 - la posición de la última A
SELECT LOCATE('AAA', 'A', -2) FROM "DUMMY";	2 - la posición del penúltimo A
SELECT LOCATE('AAA', 'A', -3) FROM "DUMMY";	1 - la posición del tercer al último A
SELECT LOCATE('AAA', 'A', -4) FROM "DUMMY";	0 - no existe
SELECT LOCATE('ABABAC', 'A', -2) FROM "DUMMY";	5 - la posición de la primera A empezando por la segunda última posición

- Si se encuentra una coincidencia, la posición de coincidencia devuelta siempre es un número positivo, incluso cuando <start_position> es un número negativo.

Ejemplos

En el siguiente ejemplo, se muestra 1 porque *<aguja>* es una cadena vacía.

```
SELECT LOCATE ('length in char', '') "locate" FROM DUMMY;
```

El siguiente ejemplo devuelve la posición inicial (1) de longitud en la cadena longitud en caracteres:

```
SELECT LOCATE ('length in char', 'length') "locate" FROM DUMMY;
```

El siguiente ejemplo devuelve 0 porque el patrón de búsqueda zchar no se encuentra en la cadena indicada:

```
SELECT LOCATE ('length in char', 'zchar') "locate" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función LOCATE_REGEXPR (string)

Busca una cadena para un patrón de expresión regular y devuelve un entero que indica la posición inicial o la posición final más 1 de una ocurrencia de la subcadena coincidente.

Sintaxis

```
LOCATE_REGEXPR( [ <regex_position_start_or_after> ] <pattern>
  [ FLAG <flag> ]
  IN <regex_subject_string>
  [ FROM <start_position> ]
  [ OCCURRENCE <regex_occurrence> ]
  [ GROUP <regex_capture_group> ]
)
```

Elementos de sintaxis

<regex_position_start_or_after>

Busca una cadena para un patrón de expresión regular y devuelve un entero que indica la posición inicial o la posición final más 1 de una ocurrencia de la subcadena coincidente.

<regex_position_start_or_after> ::= START | AFTER

Si **<regex_position_start_or_after>** no se especifica, entonces START es implícito.

<pattern>

Un patrón de búsqueda basado en Perl Compatible Regular Expression (PCRE).

<flag>

El comportamiento coincidente de la función se puede definir mediante el literal **<flag>**. Están disponibles las siguientes opciones:

<flag> ::= 'i' | 'm' | 's' | 'x'

Marcar opciones

Marcar opción	Descripción
i	Activa la coincidencia que no distingue entre mayúsculas y minúsculas
m	Activa el modo multilínea, donde <code><subject_string></code> se tratará como varias líneas y la expresión <code>^</code> y <code>\$</code> coinciden justo después o justo antes, respectivamente, de un terminador de línea o el final de la secuencia de entrada
s	Habilita la expresión <code><.></code> como comodín para que coincida con cualquier carácter, incluido un terminador de línea
x	Permite espacios en blanco y comentarios en el patrón
U	SAP HANA utiliza la biblioteca Expresiones regulares compatibles con Perl (PCRE) para procesar expresiones regulares. Al especificar 'U' (abreviatura de "poco codicioso") se invierte la "codicia" de los cuantificadores para que no sean codiciosos por defecto sino que se vuelvan codiciosos solo cuando vayan seguidos de "?". La coincidencia poco codiciosa a menudo puede funcionar más rápido porque encuentra la coincidencia más corta en momentos en los que solo es interesante saber si hay alguna coincidencia. Para una comprensión completa del comportamiento de coincidencia "codicioso" frente a "poco codicioso" de la biblioteca de expresiones regulares compatibles con Perl (PCRE), visite: https://www.pcre.org/original/doc/html/pcrematching.html .

`<regex_subject_string>`

`<regex_subject_string> ::= <string>`

Especifica una cadena en la que buscar el patrón de expresión regular.

`<start_position>`

El parámetro `<start_position>` es un entero positivo e indica el carácter de `<regex_subject_string>` donde se inicia la búsqueda. Si `<start_position>` no es un entero positivo, entonces se devuelve 0.

`<start_position> ::= <positive_integer>`

`<regex_occurrence>`

Especifica un número entero positivo para la ocurrencia del `<patrón>` en `<regex_subject_string>`. El valor predeterminado es 1.

`<regex_occurrence> ::= <integer>`

Si `<regex_aparece>` no es un entero positivo, entonces se devuelve 0.

`<regex_capture_group>`

El parámetro `<regex_capture_group>` es un número entero no negativo e indica el número del grupo de subcadenas capturado por la expresión regular. El valor predeterminado es 0.

`<regex_capture_group> ::= <integer>`

Si `<regex_capture_group>` es un entero negativo, entonces se devuelve 0.

Descripción

Busca una cadena para un patrón de expresión regular y devuelve un entero que indica la posición inicial o la posición final más 1 de una ocurrencia de la subcadena coincidente.

Si alguno de los siguientes parámetros es NULL: *<pattern>*, *<flag>*, *<regex_subject_string>*, *<start_position>*, *<regex_luck>* o *<regex_capture_group>*, la función devuelve NULL.

Ejemplo

Este ejemplo devuelve la posición de inicio de la parte del día a partir del valor de fecha 20140401; devuelve 7:

```
SELECT LOCATE_REGEXP(START '([[:digit:]]{4})([[:digit:]]{2})([[:digit:]]{2})' IN '20140401' GROUP
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función LOG (numérica)

Devuelve el logaritmo natural de un número y una base especificados.

Sintaxis

LOG(*<base>*, *<number>*)

Descripción

Devuelve el logaritmo natural de un número especificado por *<número>* y una base especificada por *<base>*, donde *<base>* debe ser un valor positivo distinto de 1 y *<número>* debe ser cualquier valor positivo.

Ejemplo

El siguiente ejemplo devuelve el logaritmo natural para 2 base 10, que es 0,30102999566398114:

```
SELECT LOG (10, 2) "log" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función LOWER (cadena)

Convierte en minúsculas todos los caracteres de una cadena.

Sintaxis

LOWER(*<string>*)

Descripción

Convierte todos los caracteres de *<string>* en minúsculas.

La función LOWER es idéntica a la función LCASE.

Ejemplo

Este ejemplo convierte la cadena indicada AnT en minúsculas y devuelve el valor ant:

```
SELECT LOWER ( 'AnT' ) "lower" FROM DUMMY;
```

Información relacionada

[Función LCASE \(cadena\)](#)

[Tipos de datos de cadena de caracteres](#)

Función LPAD (cadena)

Rellena a la izquierda una cadena con espacios, o un patrón especificado, para convertir una cadena en un número especificado de caracteres de longitud.

Sintaxis

```
LPAD(<string>, <number> [, <pattern>])
```

Elementos de sintaxis

<string>

Especifica una cadena que se debe rellenar.

<number>

Especifica la longitud a la que rellenar *<string>*. *<número>* debe ser un número entero.

<pattern>

Especifica una cadena de caracteres que se utilizará para el relleno en lugar de espacios.

Descripción

Rellena a la izquierda el final de *<string>* con espacios para hacer una string de *<number>* caracteres. Si se especifica *<pattern>*, entonces *<string>* se rellena utilizando secuencias de los caracteres indicados hasta que se alcanza la longitud necesaria.

Si la longitud de *<string>* es mayor que *<number>*, no se realiza ningún relleno y el valor resultante se trunca desde el lado derecho hasta la longitud especificada en *<number>*.

LPAD devuelve un valor de cadena vacío si *<número>* es menor que 1.

Ejemplo

El siguiente ejemplo rellena a la izquierda el inicio del fin de la cadena con el patrón 12345 para hacer que una cadena tenga 15 caracteres de longitud y devuelve el valor 123451234512end:

```
SELECT LPAD ('end', 15, '12345') "lpad" FROM DUMMY;
```

En el siguiente ejemplo, *<string>* es más largo que *<n>*, por lo que no se realiza ningún relleno y el resultado se trunca *<string>* a la longitud de *<number>* (es decir, en):

```
SELECT LPAD ('end', 2, '12345') "lpad" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función LTRIM (cadena)

Devuelve una cadena, sin los espacios iniciales.

Sintaxis

```
LTRIM(<string> [, <remove_set>])
```

Descripción

Devuelve la cadena *<string>*, sin los espacios iniciales. Si se especifica *<remove_set>*, LTRIM elimina todos los caracteres contenidos en este conjunto desde el inicio de la cadena *<string>*. Este proceso continúa hasta que se alcanza un carácter que no está en *<remove_set>*.

<remove_set> se trata como un conjunto de caracteres y no como una cadena de búsqueda.

Ejemplo

Este ejemplo elimina todos los caracteres a y b iniciales de la cadena indicada y devuelve el valor Aabend:

```
SELECT LTRIM ('babababAabend','ab') "ltrim" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función MAP (varios)

Busca una expresión dentro de un conjunto de valores y devuelve un resultado especificado.

Sintaxis

```
MAP(<expression>, <search_value>, <result> [, <search_value>, <result> [...] ] [, <default_result>].
```

Elementos de sintaxis

<expression>

Especifica la expresión que se debe buscar en cualquiera de los valores *de* <search>.

<search_value>

Especifica un valor para buscar <expresión> en. Puede especificar más de un valor de búsqueda y, para cada valor <search_value>, puede especificar un <result> correspondiente.

<result>

Especifica el resultado a devolver cuando <expresión> se encuentra en el valor <search_value> correspondiente.

<default_result>

Especifica el resultado predeterminado que se devolverá cuando no se encuentre <expression> en ninguno de los valores *de* <search_value>.

Descripción

Busca <expresión> dentro de un conjunto de valores de búsqueda y devuelve el resultado correspondiente.

- Si no se encuentra <expression> y se define <default_result>, MAP devuelve <default_result>.
- Si <expression> no se encuentra y <default_result> no está definido, MAP devuelve NULL.

<search_value> y los valores <result> correspondientes deben especificarse como pares.

Ejemplo

La siguiente consulta busca 2, la encuentra en el tercer par <search>/<result> (2, 'Two') y devuelve el valor configurado *de* <result> Dos.

```
SELECT MAP(2, 0, 'Zero', 1, 'One', 2, 'Two', 3, 'Three', 'Default') "map" FROM DUMMY;
```

Los siguientes buscan 99 en los valores <search> y, sin encontrarlos, devuelven el valor <default_result> Predeterminado.

```
SELECT MAP(99, 0, 'Zero', 1, 'One', 2, 'Two', 3, 'Three', 'Default') "map" FROM DUMMY;
```

Los siguientes buscan 99 en los valores <search> y, al no encontrarlos, devuelven NULL porque no se ha especificado ningún <default_result>.

```
SELECT MAP(99, 0, 'Zero', 1, 'One', 2, 'Two', 3, 'Three') "map" FROM DUMMY;
```

Información relacionada

[Expresiones](#)

Función MAX (agregar)

Devuelve el valor máximo de la expresión. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
MAX( [ ALL | DISTINCT ] <expression> )
```

Función de ventana:

```
MAX( <expression> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica los datos de entrada para la función.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La función MAX devuelve el valor más grande de un conjunto de valores. Esta función se puede aplicar a una variedad de tipos de datos y el tipo de datos de resultado será consistente con el tipo de datos de entrada. A continuación se muestra una explicación detallada de cómo se comporta la función MAX con diferentes tipos:

- Tipos numéricos: Devuelve el valor numérico más grande del conjunto (por ejemplo, TINYINT, SMALLINT, INTEGER, BIGINT, DECIMAL (p, s), REAL, DOUBLE).
- Tipos de cadena: Devuelve la cadena lexicográficamente más grande (orden alfabético) para los tipos de datos de cadena (por ejemplo, NVARCHAR, VARCHAR).
- Tipos de fecha/hora: Devuelve el valor de fecha u hora más reciente del conjunto (por ejemplo, DATE, TIME, TIMESTAMP).
- Tipos de puntos: Devuelve el punto con la coordenada X o Y más grande.
- Tipos de geometría: Devuelve la geometría con el área o longitud más grande.
- Tipos JSON: compara objetos JSON en función de sus representaciones de cadena, devolviendo la cadena JSON más grande.

Ejemplo

Las siguientes sentencias crean una tabla con datos, por ejemplo:

```
DROP TABLE "MyProducts";
CREATE COLUMN TABLE "MyProducts" (
    "Product_ID" NVARCHAR(10),
    "Product_Name" NVARCHAR(100),
    "Category" NVARCHAR(100),
    "Quantity" INTEGER,
    "Price" DECIMAL(10,2),
    PRIMARY KEY ("Product_ID") );

INSERT INTO "MyProducts" VALUES('P1','Shirts', 'Clothes', 32, 20.99);
INSERT INTO "MyProducts" VALUES('P2','Jackets', 'Clothes', 16, 99.49);
```

```
INSERT INTO "MyProducts" VALUES('P3','Trousers', 'Clothes', 30, 32.99);  
INSERT INTO "MyProducts" VALUES('P4','Coats', 'Clothes', 5, 129.99);  
INSERT INTO "MyProducts" VALUES('P5','Purse', 'Accessories', 3, 89.49);
```

El siguiente ejemplo devuelve 129,99, el precio máximo de los productos en la tabla MyProducts:

```
SELECT MAX("Price") FROM "MyProducts";
```

Información relacionada

[Funciones de ventana y la especificación de ventana](#)

[Funciones agregadas de ventana](#)

[Agregar funciones](#)

[Expresiones](#)

Función MEDIAN (agregar)

Busca la mediana estadística de una expresión de entrada con un tipo de datos numérico. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
MEDIAN( <expression> )
```

Función de ventana:

```
MEDIAN( <expression> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica la expresión de datos de entrada para la función MEDIAN.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La función MEDIAN encuentra la mediana estadística de una expresión de entrada con un tipo de datos numérico. Aunque se agrupa con las funciones agregadas, su cláusula OVER opcional lo posiciona también como una función de Windows.

Se eliminan los valores nulos. Si hay un número par de elementos, se devuelve el promedio de los dos elementos medios. De lo contrario, se devuelve el elemento medio.

El tipo de resultado es el tipo que se selecciona para la expresión "x/2" para un valor x del tipo de datos de entrada.

Ejemplos

Mediana de la entrada de enteros El siguiente ejemplo devuelve un valor medio de 2.

```
CREATE TABLE T (class NVARCHAR(10), date DAYDATE, val INT);
INSERT INTO T VALUES('A', '01.01.1972', 1);
INSERT INTO T VALUES('A', '02.01.1972', 3);
INSERT INTO T VALUES('A', '03.01.1972', null);
INSERT INTO T VALUES('A', '04.01.1972', 2);

SELECT MEDIAN(val) "median value" FROM T;
```

Si el número de valores no nulos es par, se devuelve el promedio de los dos valores medios. Para el siguiente ejemplo, se devuelve el promedio de 2 y 3. Dado que las clases de entrada y salida son las mismas, el entero se redondea. El resultado devuelto es 3.

```
INSERT INTO T VALUES('A', '05.01.1972', 4);
SELECT MEDIAN(val) "median value" FROM T;
```

Mediana de doble entrada El siguiente ejemplo utiliza valores dobles en lugar de enteros. El resultado devuelto es 2,5.

```
CREATE TABLE T (TS_ID NVARCHAR(10), date DAYDATE, val DOUBLE);
INSERT INTO T VALUES('A', '01.01.1972', 1.0);
INSERT INTO T VALUES('A', '02.01.1972', 3.0);
INSERT INTO T VALUES('A', '03.01.1972', null);
INSERT INTO T VALUES('A', '04.01.1972', 2.0);
INSERT INTO T VALUES('A', '05.01.1972', 4.0);

SELECT MEDIAN(val) "median value" FROM T;
```

Mediana como función de ventana El siguiente ejemplo utiliza valores dobles en lugar de enteros.

```
CREATE TABLE T (TS_ID NVARCHAR(10), date DAYDATE, val DOUBLE);
INSERT INTO T VALUES('A', '01.01.1972', 1.0);
INSERT INTO T VALUES('A', '02.01.1972', 3.0);
INSERT INTO T VALUES('A', '03.01.1972', null);
INSERT INTO T VALUES('A', '04.01.1972', 2.0);
INSERT INTO T VALUES('A', '04.01.1972', 4.0);

SELECT MEDIAN(val) OVER (PARTITION BY TS_ID ) AS WF1 FROM T;
```

El resultado devuelto es:

WF1
2.5
2.5
2.5
2.5
2.5

Mediana de ventana deslizante (GROUPS BETWEEN) Ambas sentencias SELECT en el siguiente ejemplo producen resultados idénticos.

```
CREATE TABLE T (TS_ID NVARCHAR(10), date DAYDATE, val DOUBLE);
INSERT INTO T VALUES('A', '01.01.1972', 1.0);
INSERT INTO T VALUES('A', '02.01.1972', 3.0);
```

```
INSERT INTO T VALUES('A', '03.01.1972', null);
INSERT INTO T VALUES('A', '04.01.1972', 2.0);
INSERT INTO T VALUES('A', '04.01.1972', 4.0);

SELECT MEDIAN(val) OVER (PARTITION BY TS_ID ORDER BY date) AS WF2A FROM T;
SELECT MEDIAN(val) OVER (PARTITION BY TS_ID ORDER BY date GROUPS BETWEEN UNBOUNDED PRECEDING AND C
```

Los conjuntos de resultados devueltos son:

WF2A	WF2B
1.0	1.0
2.0	2.0
2.0	2.0
2.5	2.5
2.5	2.5

Mediana de ventana deslizante (FILAS ENTRE) El siguiente ejemplo utiliza FILAS ENTRE FILAS PRECEDING UNBOUNDED Y FILA ACTUAL.

```
CREATE TABLE T (TS_ID NVARCHAR(10), date DAYDATE, val DOUBLE);
INSERT INTO T VALUES('A', '01.01.1972', 1.0);
INSERT INTO T VALUES('A', '02.01.1972', 3.0);
INSERT INTO T VALUES('A', '03.01.1972', null);
INSERT INTO T VALUES('A', '04.01.1972', 2.0);
INSERT INTO T VALUES('A', '04.01.1972', 4.0);

SELECT MEDIAN(val) OVER (PARTITION BY TS_ID ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CUR
```

El resultado devuelto es:

WF3
1.0
2.0
2.0
2.0
2.5

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Agregar funciones](#)
- [Expresiones](#)

Función MEMBER_AT (matriz)

Devuelve valores de una posición de matriz especificada.

Sintaxis

```
MEMBER_AT(<array_value_expression>, <position> [, <default_value>])
```

Descripción

Accede a un elemento de matriz en la posición ordinal especificada y devuelve el valor.

<default_value> indica el valor a devolver si <position> es mayor que la cardinalidad de la <array_value_expression>. Si no se especifica <default_value> y <position> es mayor que la cardinalidad de la <array_value_expression>, entonces se devuelve NULL.

Ejemplo

El siguiente ejemplo crea una tabla e inserta matrices en ella:

```
CREATE COLUMN TABLE ARRAY_TEST (IDX INT, VAL INT ARRAY);
INSERT INTO ARRAY_TEST VALUES (1, ARRAY(1, 2, 3));
INSERT INTO ARRAY_TEST VALUES (2, ARRAY(10, 20, 30, 40));
```

El siguiente ejemplo devuelve el valor en la posición 4 de cada matriz de la tabla. La primera matriz no tiene una posición 4, por lo que se devuelve NULL:

```
SELECT MEMBER_AT(VAL, 4) "member_at" FROM ARRAY_TEST;
```

member_at
NULO
40

El siguiente ejemplo devuelve el valor en la posición 4 de cada matriz de la tabla. La primera matriz no tiene una posición 4, por lo que 1 se devuelve <posición> es mayor que la cardinalidad de la matriz:

```
SELECT MEMBER_AT(VAL, 4, 1) "member_at" FROM ARRAY_TEST;
```

member_at
1
40

Información relacionada

[Expresiones](#)

Función MEMBER_AT (vector)

Devuelve el elemento de vector en la posición especificada.

Sintaxis

```
MEMBER_AT(<vector>, <position> [, <default_value>])
```

Elementos de sintaxis

<vector>

Especifica el vector desde el que la función devuelve el elemento de vector.

<position>

Especifica la posición del elemento de vector que se va a devolver.

El primer elemento vectorial tiene la posición 1. Si <posición> es mayor que la dimensión del <vector>, entonces se devuelve NULL.

<default_value>

Especifica el valor predeterminado si <posición> es mayor que la dimensión del <vector>. Si no se especifica <default_value> y <position> es mayor que la dimensión del <vector>, entonces se devuelve NULL.

Descripción

Devuelve valores de una posición especificada en un vector.

Ejemplos

Los siguientes ejemplos devuelven 5, que es el valor en la posición 3 del vector.

```
SELECT MEMBER_AT(TO_REAL_VECTOR(' [2,3,5] '), 3) "member_at" FROM DUMMY;
SELECT MEMBER_AT(TO_HALF_VECTOR(' [2,3,5] '), 3) "member_at" FROM DUMMY;
```

Los siguientes ejemplos devuelven NULL, porque <posición> (=6) es mayor que la dimensión del vector.

```
SELECT MEMBER_AT(TO_REAL_VECTOR(' [2,3,5] '), 6) "member_at" FROM DUMMY;
SELECT MEMBER_AT(TO_HALF_VECTOR(' [2,3,5] '), 6) "member_at" FROM DUMMY;
```

El siguiente ejemplo devuelve 12, porque <position> (=6) es mayor que la dimensión del vector y se especifica <default_value> (=12).

```
SELECT MEMBER_AT(TO_REAL_VECTOR(' [2,3,5] '), 6, 12) "member_at" FROM DUMMY;
SELECT MEMBER_AT(TO_HALF_VECTOR(' [2,3,5] '), 6, 12) "member_at" FROM DUMMY;
```

En comparación con los elementos REAL_VECTOR, los elementos HALF_VECTOR tienen una precisión reducida y un rango de valores reducido. Esto puede provocar resultados inesperados:

```
SELECT
  TO_NVARCHAR(MEMBER_AT(TO_REAL_VECTOR(' [0.3] '), 1)),
  TO_NVARCHAR(MEMBER_AT(TO_HALF_VECTOR(' [0.3] '), 1))
FROM DUMMY;
```

returns:
0.3 0.30004883

```
SELECT
  TO_NVARCHAR(MEMBER_AT(TO_REAL_VECTOR(' [33333] '), 1)),
  TO_NVARCHAR(MEMBER_AT(TO_HALF_VECTOR(' [33333] '), 1))
FROM DUMMY;
```

returns:

33333 33344

Función MIN (agregado)

Devuelve el valor mínimo de la expresión. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
MIN( [ ALL | DISTINCT ] <expression> )
```

Función de ventana:

```
MIN( <expression> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica los datos de entrada para la función.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La función **MIN** devuelve el valor más pequeño de un conjunto de valores. Estas funciones se pueden aplicar a una variedad de tipos de datos y el tipo de datos de resultado será consistente con el tipo de datos de entrada. A continuación se muestra una explicación detallada de cómo se comporta la función **MIN** con diferentes tipos:

- Tipos numéricos: Devuelve el valor numérico más pequeño del conjunto (por ejemplo, TINYINT, SMALLINT, INTEGER, BIGINT, DECIMAL (p, s), REAL, DOUBLE).
- Tipos de cadena: Devuelve la cadena más pequeña lexicográficamente (orden alfabético) para los tipos de datos de cadena (por ejemplo, NVARCHAR, VARCHAR).
- Tipos de fecha/hora: Devuelve el valor de fecha u hora más temprano del conjunto (por ejemplo, DATE, TIME, TIMESTAMP).
- Tipos de puntos: Devuelve el punto con la coordenada X o Y más pequeña.
- Tipos de geometría: Devuelve la geometría con el área o longitud más pequeña.
- Tipos JSON: compara objetos JSON en función de sus representaciones de cadena, devolviendo la cadena JSON más pequeña.

Ejemplo

Las siguientes sentencias crean una tabla con datos, por ejemplo:

```
DROP TABLE "MyProducts";  
CREATE COLUMN TABLE "MyProducts"(  
    "Product_ID" NVARCHAR(10),  
    "Product_Name" NVARCHAR(100),  
    "Category" NVARCHAR(100),  
    "Quantity" INTEGER,  
    "Price" DECIMAL(10,2),  
    PRIMARY KEY ("Product_ID") );  
  
INSERT INTO "MyProducts" VALUES('P1','Shirts', 'Clothes', 32, 20.99);  
INSERT INTO "MyProducts" VALUES('P2','Jackets', 'Clothes', 16, 99.49);  
INSERT INTO "MyProducts" VALUES('P3','Trousers', 'Clothes', 30, 32.99);  
INSERT INTO "MyProducts" VALUES('P4','Coats', 'Clothes', 5, 129.99);  
INSERT INTO "MyProducts" VALUES('P5','Purse', 'Accessories', 3, 89.49);
```

El siguiente ejemplo devuelve 20,99, el precio mínimo de los productos en la tabla MyProducts:

```
SELECT MIN("Price") FROM "MyProducts";
```

Información relacionada

[Funciones de ventana y la especificación de ventana](#)

[Funciones agregadas de ventana](#)

[Agregar funciones](#)

[Expresiones](#)

Función MINUTE (fecha y hora)

Devuelve una representación en enteros del minuto para la hora especificada.

Sintaxis

```
MINUTE(<time>)
```

Descripción

Devuelve una representación en enteros del minuto para la hora <time>.

Ejemplo

El siguiente ejemplo devuelve el valor 34 como minuto para la hora especificada:

```
SELECT MINUTE ('12:34:56') "minute" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función MOD (numérica)

Devuelve el resto de un número especificado dividido por un divisor especificado.

Sintaxis

`MOD(<number>, <divisor>)`

Descripción

Devuelve el resto de un número <número> dividido por un divisor <divisor>.

Cuando <número> es negativo, esta función actúa de manera diferente a la operación estándar del módulo computacional. La siguiente lista muestra ejemplos de lo que devuelve la función MOD como resultado:

- Si <divisor> es cero, se devuelve <número>.
- Si <number> es mayor que 0 y <number> es menor que <divisor>, entonces se devuelve <number>.
- Si <number> es menor que 0 y <number> es mayor que <divisor>, entonces se devuelve <number>.
- En casos distintos de los mencionados anteriormente, el resto del valor absoluto de <número> dividido por el valor absoluto de <divisor> se utiliza para calcular el resto. Si <number> es menor que 0, entonces el resto devuelto de MOD es un número negativo, y si <number> es mayor que 0, entonces el resto devuelto de MOD es un número positivo.

Ejemplo

El siguiente ejemplo devuelve el valor 3:

```
SELECT MOD (15, 4) "modulus" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor -3:

```
SELECT MOD (-15, 4) "modulus" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función MONTH (fecha y hora)

Devuelve el número del mes a partir de la fecha especificada.

Sintaxis

`MONTH(<date>)`

Descripción

Devuelve el número del mes a partir de la fecha *<date>*.

Ejemplo

El siguiente ejemplo devuelve el valor 5, el mes de la fecha especificada:

```
SELECT MONTH ( '2011-05-30' ) "month" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función MONTHNAME (fecha y hora)

Devuelve el nombre del mes de la fecha especificada.

Sintaxis

```
MONTHNAME (<date>)
```

Descripción

Devuelve el nombre del mes de la fecha *<date>*.

Ejemplo

El siguiente ejemplo devuelve el valor MAY, el nombre del mes de la fecha especificada:

```
SELECT MONTHNAME ( '2011-05-30' ) "monthname" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función MONTHS_BETWEEN (fecha y hora)

Calcula el número de meses entre dos fechas.

Sintaxis

```
MONTHS_BETWEEN (<date_1>, <date_2>)
```

Descripción

Calcula el número de meses comprendido entre *<date_1>* y *<date_2>*. Devuelve NULL si *<date_1>* o *<date_2>* es NULL.

Ejemplos

El siguiente ejemplo devuelve 2 para los meses entre las dos fechas:

```
SELECT MONTHS_BETWEEN(TO_DATE ('2003-01-01'), TO_DATE('2003-03-14')) "months_between" FROM DUMMY;
```

El siguiente ejemplo devuelve -8 para los meses entre las dos fechas:

```
SELECT MONTHS_BETWEEN(TO_DATE ('2003-10-03'), TO_DATE('2003-01-14')) "months_between" FROM DUMMY;
```

El siguiente ejemplo devuelve -9 para los meses entre las dos fechas:

```
SELECT MONTHS_BETWEEN(TO_DATE ('2003-10-15'), TO_DATE('2003-01-14')) "months_between" FROM DUMMY;
```

El siguiente ejemplo devuelve -8 para los meses entre las dos fechas:

```
SELECT MONTHS_BETWEEN(TO_DATE ('2003-10-13'), TO_DATE('2003-01-14')) "months_between" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función NANO100_BETWEEN (fecha y hora)

Calcula la diferencia de tiempo entre dos fechas con la precisión de 0,1 microsegundos.

Sintaxis

```
NANO100_BETWEEN(<timestamp_1>, <timestamp_2>)
```

Descripción

Calcula la diferencia de tiempo entre los argumentos de fecha <timestamp_1> y <timestamp_2>, con la precisión de 0,1 microsegundos.

Ejemplo

El siguiente ejemplo devuelve 477 como diferencia de tiempo entre las dos fechas especificadas:

```
SELECT NANO100_BETWEEN ('2021-01-31 00:00:00.0000176', '2021-01-31 00:00:00.0000653') "nano100 betv
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función NCHAR (cadena)

Devuelve el carácter Unicode con el número de código especificado.

Sintaxis

`NCHAR(<number>)`

Descripción

Devuelve el carácter Unicode con el número de código especificado *<número>*.

Ejemplo

Este ejemplo devuelve el carácter Unicode (A) con el número de código 65:

```
SELECT NCHAR (65) "nchar" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función NDIV0 (numérica)

Devuelve 0 cuando se divide por 0; de lo contrario, devuelve el resultado de la división.

Sintaxis

`NDIV0( <numerator>, <denominator> )`

Descripción

Devuelve 0 si *<denominator>* es 0. De lo contrario, devuelve la división del numerador por el denominador. *<numerator>* y *<denominator>* pueden ser cualquier valor numérico. La función le permite evitar mensajes de error divididos por cero y puede continuar un cálculo con un resultado definido.

Ejemplo

El siguiente ejemplo calcula el precio por unidad dividiendo Revenue por Quantity. La función NDIV0 evita errores de división por cero y el resultado se convierte en un entero para devolver un valor sin decimales.

```
SELECT
  Revenue,
  Quantity,
  CAST(NDIV0(Revenue, Quantity) AS INTEGER) AS PricePerUnit
FROM (
  SELECT 1000 AS Revenue, 4 AS Quantity FROM DUMMY
  UNION ALL
  SELECT 2000, 0 FROM DUMMY
  UNION ALL
  SELECT 2500, 0 FROM DUMMY
  UNION ALL
```

```
SELECT 3000, 5 FROM DUMMY
);
```

Ingresos	Cantidad	Promedio de ingresos
1000	4	250
2000	0	0
2500	0	0
3000	5	600

Función NEXT_DAY (fecha y hora)

Devuelve la fecha del día siguiente a la fecha especificada.

Sintaxis

```
NEXT_DAY(<date>)
```

Descripción

Devuelve la fecha del día siguiente a la fecha especificada.

Ejemplo

El siguiente ejemplo devuelve 2010-01-01 como el día después del 31/12/2009:

```
SELECT NEXT_DAY (TO_DATE ( '2009-12-31' , 'YYYY-MM-DD' )) "next day" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función NEWUID (varios)

Crea un identificador único dentro de una base de datos SAP HANA.

Sintaxis

```
NEWUID ( )
```

Descripción

La función NEWUID actúa como una alternativa a la función SYSUUID. La función SYSUUID genera un identificador único universal, mientras que la función NEWUID genera un identificador único dentro de una base de datos SAP HANA. Ambas funciones devuelven VARBINARY(16).

Ejemplo

El siguiente ejemplo ficticio devuelve un valor unívoco, como C15F6DB61C6A2B00F0000001EC2E2F00:

```
SELECT NEWUID() FROM DUMMY;
```

Función NORMALIZE (cadena)

Aplica un formato de normalización a una expresión de valor de caracteres y devuelve una cadena normalizada.

Sintaxis

```
NORMALIZE( <character_value_expression> [ , <normalized_format> ] )
```

Elementos de sintaxis

<character_value_expression>

Especifica el string NVARCHAR a normalizar.

<normalized_format>

Especifica el formato de codificación que se utilizará para el resultado. El formato predeterminado es NFC.

```
<normalized_format> ::=
NFC
| NFD
| NFKC
| NFKD
```

Las cuatro formas de normalización Unicode son las siguientes:

Formulario	Descripción
Formulario de normalización D (NFD)	Especifica la descomposición canónica.
Formulario de normalización C (NFC)	Especifica la descomposición canónica seguida de la composición canónica.
Formulario de normalización KD (NFKD)	Especifica la descomposición de compatibilidad.
Formulario de normalización KC (NFKC)	Especifica la descomposición de compatibilidad seguida de la composición canónica.

Descripción

El estándar Unicode define dos tipos formales, que son canónicos y la compatibilidad equivalente a los mismos caracteres por cuatro formas de normalización. El algoritmo de normalización Unicode coloca todas las marcas combinadas en un orden específico. Solo la descomposición o la descomposición seguida de la regla de composición se aplica para transformar cada cadena. Los detalles se pueden encontrar en el siguiente Anexo #15 de la norma Unicode: <http://www.unicode.org/reports/tr15/>

Ejemplo

Los ejemplos de esta sección utilizan la función NORMALIZE para devolver el equivalente normalizado de *<Character_value_expression>* basado en la forma *<normalized_format>*.

El siguiente es un ejemplo de equivalencia canónica:

Extracto	Resultado
SELECT NORMALIZE('Å') FROM DUMMY;	Å (0x00C5)
SELECT NORMALIZE('Å') FROM DUMMY;	Å
SELECT NORMALIZE('Å', 'NFC') FROM DUMMY;	Å (0x00C5)
SELECT NORMALIZE('Å', 'NFD') FROM DUMMY;	Å (0x0041 0x030A)
SELECT NORMALIZE('Å', 'NFC') FROM DUMMY;	Å (0x00C5)
SELECT NORMALIZE('Å', 'NFD') FROM DUMMY;	Å (0x0041 0x030A)

i Nota
El ejemplo es ilustrativo y no funciona como se muestra con copiar y pegar.

A continuación se muestra un ejemplo de equivalencia de compatibilidad:

Extracto	Resultado
SELECT NORMALIZE('ع', 'NFKC') FROM DUMMY;	ع (variantes posicionales)
SELECT NORMALIZE ('ع', 'NFKC') FROM DUMMY;	ع (variantes posicionales)
SELECT NORMALIZE('①', 'NFKC') FROM DUMMY;	1 (variantes rodeadas)
SELECT NORMALIZE('ا', 'NFKC') FROM DUMMY;	Error (variantes de ancho)
SELECT NORMALIZE('¼', 'NFKC') FROM DUMMY;	1/4 (fracciones)
SELECT NORMALIZE('i ⁹ ', 'NFD') FROM DUMMY;	i ⁹ (superíndices/subíndices)

Información relacionada

[Expresiones](#)

Función NOW (fecha y hora)

Devuelve la marca de tiempo actual.

Sintaxis

NOW ()

Descripción

Devuelve la marca de tiempo actual.

Ejemplo

El siguiente ejemplo devuelve un valor similar a 2010-01-01 16:34:19.894 para el cronomarcador actual:

```
SELECT NOW ( ) "now" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función NTH_VALUE (agregar)

Devuelve el valor de un elemento en una posición específica de una expresión. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
NTH_VALUE( <expression>, <position> <order_by_clause> )
```

Función de ventana:

```
NTH_VALUE( <expression>, <position> <order_by_clause> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica la expresión en la que se produce la posición del elemento.

<position>

Especifica la posición del elemento en <expresión>.

<order_by_clause>

Especifica el orden de clasificación de las filas de entrada.

```
<order_by_clause> ::= ORDER BY <order_by_expression> [, <order_by_expression> [...] ]
```

```
<order_by_expression> ::=
  <column_name> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
  | <column_position> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
```

`<collate_clause> ::= COLLATE <collation_name>`

`<clausula_colateral>` especifica la intercalación que se va a utilizar para ordenar los valores en los resultados. `<collate_cláusula>` solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un sinónimo de NVARCHAR).

`<collation_name>` es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

`<window_specification>`

Define una ventana en los datos sobre los que opera la función. Para `<window_specification>`, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Devuelve el valor del elemento en la posición especificada por `<posición>` en `<expresión>` como ordenado por `<order_by_cláusula>`.

Se devuelve nulo si el valor es NULL o si el valor de `<posición>` es mayor que el número de elementos en `<expresión>`.

Se produce un error si `<position>` es menor o igual que 0.

La salida de la función NTH_VALUE puede ser no determinista entre los valores vinculados.

Ejemplo

El siguiente ejemplo devuelve el segundo valor de la columna COL1: 700, cuando la tabla está ordenada por COL2.

```
CREATE ROW TABLE T (COL1 DOUBLE, COL2 DOUBLE);
INSERT INTO T VALUES(900, 10);
INSERT INTO T VALUES(400, 50);
INSERT INTO T VALUES(700, 30);
INSERT INTO T VALUES(200, 40);

SELECT NTH_VALUE (COL1, 2 ORDER BY COL2) FROM T;
```

Ejemplo de tipo de datos espaciales (ST_Geometry, ST_Point)

El siguiente ejemplo devuelve el valor enésimo, POINT (3 3), en formato binario, cuando la tabla se ordena por ID:

```
CREATE TABLE TAB (ID INT, SHAPE ST_Geometry(4326));

INSERT INTO TAB VALUES(1, ST_GeomFromText('POINT(1 1)', 4326));
INSERT INTO TAB VALUES(2, ST_GeomFromText('POINT(2 2)', 4326));
INSERT INTO TAB VALUES(3, ST_GeomFromText('POINT(3 3)', 4326));
INSERT INTO TAB VALUES(4, ST_GeomFromText('POINT(4 4)', 4326));

SELECT NTH_VALUE(SHAPE, 3 ORDER BY ID) FROM TAB;
```

Para devolver un resultado como tipo de datos NVARCHAR, utilice la siguiente sentencia:

```
SELECT NTH_VALUE(CAST(SHAPE AS NVARCHAR), 3 ORDER BY ID) FROM TAB;
```

Información relacionada

[Vista del sistema COLLATIONS](#)

[Funciones de ventana y la especificación de ventana](#)

Función NTILE (ventana)

Distribuye filas en un número especificado de intervalos y asigna el número de intervalo empezando por 1 a cada fila del intervalo.

Sintaxis

```
NTILE( <number_of_buckets> ) <window_specification>
```

Elementos de sintaxis

<number_of_buckets>

Especifica el número de intervalos a los que se distribuirán las filas.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La salida de la función NTILE puede ser no determinista entre los valores de empate.

Ejemplos

```
CREATE TABLE T (class NVARCHAR(10), val INT, offset INT);
INSERT INTO T VALUES('A', 1, 1);
INSERT INTO T VALUES('A', 3, 3);
INSERT INTO T VALUES('A', 5, null);
INSERT INTO T VALUES('A', 5, 2);
INSERT INTO T VALUES('A', 10, 0);
INSERT INTO T VALUES('B', 1, 3);
INSERT INTO T VALUES('B', 1, 1);
INSERT INTO T VALUES('B', 7, 1);

SELECT class, val,
       NTILE(3) OVER (PARTITION BY class ORDER BY val) AS NTILE,
       FIRST_VALUE(val) OVER (PARTITION BY class ORDER BY val) AS first,
       LAST_VALUE(val) OVER (PARTITION BY class ORDER BY val) AS last,
       NTH_VALUE(val, 4) OVER (PARTITION BY class ORDER BY val) AS nth
FROM T;
```

CLASE	VAL	NTILE	PRIMERO	ÚLTIMO	NTH
A	1	1	1	1	nulo
A	3	1	1	3	nulo
A	5	2	1	5	5
A	5	2	1	5	5
A	10	3	1	10	5

CLASE	VAL	NTILE	PRIMERO	ÚLTIMO	NTH
B	1	1	1	1	nulo
B	1	2	1	1	nulo
B	7	3	1	7	nulo

Información relacionada

- [Expresiones](#)
- [Funciones de ventana y la especificación de ventana](#)

Función NULLIF (varios)

Determina si dos expresiones son iguales.

Sintaxis

```
NULLIF(<expression1>, <expression2>)
```

Descripción

NULLIF compara los valores de dos expresiones y devuelve NULL si <expresión1> es igual a <expresión2>.

Si <expresión1> no es igual a <expresión2>, NULLIF devuelve <expresión1>.

Si <expresión2> es NULL, NULLIF devuelve <expresión1>.

NULLIF devuelve el mismo tipo de datos que <expresión1>. Sin embargo, si <expresión1> es TINYINT o SMALLINT, NULLIF devuelve un INTEGER.

Ejemplo

La siguiente consulta devuelve diff.

```
SELECT NULLIF ('diff', 'same') "nullif" FROM DUMMY;
```

La siguiente consulta devuelve NULL.

```
SELECT NULLIF('same', 'same') "nullif" FROM DUMMY;
```

Función NUMTOHEX (cadena)

Convierte un valor numérico en un valor hexadecimal NVARCHAR.

Sintaxis

```
NUMTOHEX(<integer> [, <integer>])
```

Elementos de sintaxis

<integer>

Indica el valor numérico que se debe convertir en un valor hexadecimal.

El segundo parámetro controla la longitud del tipo de destino. Si solo se proporciona un parámetro, la longitud depende del tipo de entrada. El segundo parámetro está limitado a una longitud de cadena hexadecimal de 16 del tipo BIGINT.

Descripción

Convierte un valor numérico en un valor hexadecimal como un tipo de datos NVARCHAR. El valor de entrada puede ser un tipo de datos TINYINT, SMALLINT, BIGINT o INTEGER.

Ejemplo

El siguiente ejemplo convierte el valor numérico c1 en un valor NVARCHAR hexadecimal, 3039:

```
CREATE TABLE t1 (c1 BIGINT);
INSERT INTO t1 VALUES(9223372036854775807);
INSERT INTO t1 VALUES(-9223372036854775808);
INSERT INTO t1 VALUES (-1);
INSERT INTO t1 VALUES (0);
INSERT INTO t1 VALUES (12345);
SELECT c1, NUMTOHEX(c1) FROM t1;
```

Función OCCURRENCES_REGEXPR (cadena)

Devuelve el número de coincidencias de una búsqueda de expresión regular dentro de una cadena.

Sintaxis

```
OCCURRENCES_REGEXPR( <pattern> [ FLAG <flag> ] IN <regex_subject_string> [ FROM <start_position> ]
```

Elementos de sintaxis

<pattern>

Especifica un patrón de búsqueda basado en Perl Compatible Regular Expression (PCRE).

<pattern> ::= !!Perl Compatible Regular Expression

<flag>

Especifica el comportamiento coincidente de la función y se puede definir mediante el literal de indicador.

<flag> ::= { 'i' | 'm' | 's' | 'x' }

Marcar opciones

Marcar opción	Descripción
i	Activa la coincidencia que no distingue entre mayúsculas y minúsculas
m	Activa el modo multilínea, donde <code><subject_string></code> se tratará como varias líneas y la expresión <code>^</code> y <code>\$</code> coinciden justo después o justo antes, respectivamente, de un terminador de línea o el final de la secuencia de entrada
s	Habilita la expresión <code><.></code> como comodín para que coincida con cualquier carácter, incluido un terminador de línea
x	Permite espacios en blanco y comentarios en el patrón
U	SAP HANA utiliza la biblioteca Expresiones regulares compatibles con Perl (PCRE) para procesar expresiones regulares. Al especificar 'U' (abreviatura de "poco codicioso") se invierte la "codicia" de los cuantificadores para que no sean codiciosos por defecto sino que se vuelvan codiciosos solo cuando vayan seguidos de "?". La coincidencia poco codiciosa a menudo puede funcionar más rápido porque encuentra la coincidencia más corta en momentos en los que solo es interesante saber si hay alguna coincidencia. Para una comprensión completa del comportamiento de coincidencia "codicioso" frente a "poco codicioso" de la biblioteca de expresiones regulares compatibles con Perl (PCRE), visite:https://www.pcre.org/original/doc/html/pcrematching.html ➦ .

<regex_subject_string>

Especifica la cadena en la que buscar. Si `<regex_subject_string>` está vacío, el resultado estará vacío.

`<regex_subject_string> ::= <string>`

<start_position>

Especifica un entero positivo que indica el carácter de `<regex_subject_string>` donde se inicia la búsqueda. Si `<start_position>` no es un entero positivo, entonces se devuelve -1.

`<start_position> ::= <positive_integer>`

Descripción

Devuelve el número de coincidencias de una búsqueda de expresión regular dentro de una cadena.

Ejemplo

El siguiente ejemplo devuelve el número de ocurrencias de dígitos en la cadena especificada, 'a1b2', y devuelve el valor 2:

```
SELECT OCCURRENCES_REGEXPR('([[:digit:]])' IN 'a1b2') "occurrences_regexpr" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función PARSE_CERTIFICATES (cadena)

Analiza los certificados en formato PEM directamente y emite los detalles del certificado como una tabla interna temporal.

Sintaxis

```
PARSE_CERTIFICATES( <certificate_list> )
```

Elementos de sintaxis

<certificate_list>

Especifica una lista de certificados X.509 en formato PEM. Se admiten cadenas de certificados. Se admiten cadenas de certificados. Se ignoran los certificados no válidos y las claves privadas o públicas. Se pueden enumerar varios certificados de forma secuencial sin puntuación entre ellos. <certificate_list> es un *<literal de cadena>*.

Descripción

Se ignoran los certificados no válidos, las claves privadas o públicas. Varios certificados se pueden escribir simplemente uno tras otro.

Ejemplo

El siguiente ejemplo devuelve el certificado analizado:

```
SELECT * FROM PARSE_CERTIFICATES('-----BEGIN CERTIFICATE-----MIICLDCCAd0gAwIBAgIJA0uPGmW
...
5fChRXXI0iMEKBHniFXjSegIgE3ze0QCXTTxf04hlkmQdWTb+oySl2frEMzi4S+New7o-----END CERTIFICATE-----')
```

Función PARSE_JSON (JSON)

Genera el tipo de resultado SQL JSON analizando sintácticamente la cadena JSON. Es compatible con un tipo de datos de una columna proyectada de Almacén de documentos.

Sintaxis

```
PARSE_JSON(<JSON_context_item>)
```

Elementos de sintaxis

JSON_context_item

Especifica el documento JSON en el que operar, como un tipo nvarchar de columna de tabla o una constante de cadena.

Descripción

En comparación con el procesamiento JSON basado en cadenas, el resultado JSON SQL generado proporciona un resultado bien definido en el contexto de JSON.

Ejemplo

La siguiente sentencia devuelve un resultado de objeto JSON:

```
SELECT parse_json('{ "id": 1000, "address": "Seoul" }') FROM DUMMY;
```

La siguiente sentencia devuelve un resultado de matriz JSON:

```
SELECT parse_json(' [1,[2,3],{ }] ') FROM DUMMY;
```

La siguiente sentencia devuelve un resultado de número JSON:

```
SELECT parse_json('10000') FROM DUMMY;
```

La siguiente sentencia devuelve un resultado de cadena JSON:

```
SELECT parse_json('Tom') FROM DUMMY;
```

La siguiente sentencia devuelve un resultado booleano JSON:

```
SELECT parse_json('true') FROM DUMMY;
```

La siguiente sentencia devuelve un resultado nulo de JSON:

```
SELECT parse_json('null') FROM DUMMY;
```

Información relacionada

[Análisis gramatical de JSON](#)

[JSON como tipo de datos SQL](#)

Función PARTITION_ALLOCATION (varios)

Identifica la asignación de partición para un conjunto determinado de filas antes de ejecutar el comando INSERT.

Sintaxis

```
PARTITION_ALLOCATION(<target_table_schema>, <target_table_name>, <data_input_source>)
```

Elementos de sintaxis

<target_table_schema>

Especifica el esquema para la tabla de salida.

<target_table_name>

Especifica el nombre de la tabla de salida.

<data_input_source>

Especifica el nombre del parámetro de tipo de tabla que contiene los datos que planea insertar en la tabla particionada. El objeto fuente puede ser una variable o una tabla. Si se especifica una tabla, no es necesario particionar la tabla.

Descripción

Devuelve el hash SHA256 de los valores en las columnas de partición de cada fila en los datos de entrada y el ID de partición (lógico) en la tabla de destino.

Ejemplos

Configuración para los siguientes ejemplos:

```
---Create partitioned table T1---
CREATE TABLE T1 (A INT, B DAYDATE, C NVARCHAR(4))
  PARTITION BY RANGE(B) ((PARTITION '2000-01-01' <= VALUES < '2020-01-01') SUBPARTITION BY HASH(A)
    (PARTITION '2020-01-01' <= VALUES < '2025-01-01',
      PARTITION OTHERS));

---Create non-partitioned table T2---
CREATE TABLE T2 (A INT, B DAYDATE, C NVARCHAR(4));

---Insert data into Table T2---
INSERT INTO T2 VALUES (1, '2015-01-01', 'aaaa');
INSERT INTO T2 VALUES (2, '2015-01-02', 'aaab');
INSERT INTO T2 VALUES (3, '2015-01-03', 'aaac');
INSERT INTO T2 VALUES (4, '2016-01-04', 'aaad');
INSERT INTO T2 VALUES (5, '2016-01-04', 'aaad');
```

Este ejemplo devuelve las asignaciones de partición propuestas en T1 si las cinco filas en T2 se insertaron en T1.

```
SELECT * FROM PARTITION_ALLOCATION('SCHEMA_1', 'T1', T2);
```

HASH	PART_ID
B5F0F3D5B1812A0D4D083B2A192062DDE50E94C829E078A6B431F28B44CF9DCA	5
53C6EB2C68554F0ACCC57C2A4397FA0E07F6E8D4361537A0A3C85123ACB1032B	1
D26051295E527511D955D2E27B8143B0E412B82FDCF22B7994EF0554E1494345	2
1E15045CD807F6A04A41B9D6606CD37593D08BD29FFB2F8A2A4D898FB801F5C6	1
2301A665AEBF7ABE250D30415370D3CC74BAF705BB5F30EE1C5D1790184044EC	4

Este ejemplo devuelve la asignación de partición propuesta en T1 si se insertó un subconjunto de las filas en T2 en T1.

```
DO
BEGIN
  ltab = SELECT * FROM T2 WHERE C = 'aaad';
  SELECT * FROM PARTITION_ALLOCATION('SCHEMA_1', 'T1', :ltab);
END;
```

HASH	PART_ID
1E15045CD807F6A04A41B9D6606CD37593D08BD29FFB2F8A2A4D898FB801F5C6	1

HASH	PART_ID
2301A665AEBF7ABE250D30415370D3CC74BAF705BB5F30EE1C5D1790184044EC	4

Este ejemplo combina la salida con los datos originales:

```
WITH p AS (SELECT * FROM SYS.PARTITION_ALLOCATION('SYSTEM','T1', T2)),
      d AS (SELECT hash_sha256(to_binary(b), to_binary(a)) hash, b, a FROM T2)
SELECT p.part_id, d.a, d.b FROM p, d WHERE p.hash = d.hash;
```

Información relacionada

[Operaciones de partición \(SAP HANA Cloud, guía de administración de base de datos de SAP HANA\)](#)

PARTITION_HASH Función (varios)

Calcula la fila de partición HASH a la que se podría asignar un valor especificado.

Sintaxis

```
PARTITION_HASH(<value>)
```

Elementos de sintaxis

<value>

Especifica un valor que se podría insertar en una columna que se podría utilizar para la partición HASH.

Descripción

En función del tipo de columna del valor, la función PARTITION_HASH calcula la partición que se debe asignar si se ha insertado el valor. La función admite los siguientes tipos de datos:

- TINYINT
- SMALLINT
- INT
- BIGINT
- DECIMAL
- DECIMAL(p,s)
- NVARCHAR
- VARBINARIO
- FECHA
- TIEMPO
- CRONOMARCADOR

- SECONDDATE

Ejemplos

El siguiente ejemplo utiliza la siguiente tabla con particiones HASH:

```
CREATE TABLE T1 (a INT) PARTITION BY HASH(a) PARTITIONS 5;
```

Este ejemplo calcula que el valor 12345 podría residir en la partición 2.

```
SELECT MOD(PARTITION_HASH(12345), 5) + 1 FROM DUMMY;
```

Resultados:

	MOD(PARTITION_HASH(12345),5)+1
	2

Función PERCENT_RANK (ventana)

Calcula el porcentaje de valores que son menores o mayores que el valor actual, según la especificación ORDER BY para la ventana.

Sintaxis

```
PERCENT_RANK() <window_specification>
```

Elementos de sintaxis

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Para cualquier fila del grupo de filas en un conjunto de resultados, PERCENT_RANK calcula el porcentaje de valores que son menores o mayores que el valor actual, según la especificación ORDER BY para la ventana.

PERCENT_RANK devuelve números enteros (fraccionarios) que empiezan en 0 y terminan con 1, como se define en el orden de clasificación (ASC o DESC) en la cláusula ORDER BY.

El rango relativo de una fila R se define como (RANK()-1)/(NR-1), donde NR se define como el número de filas en la partición de ventana de R.

Ejemplos

En este ejemplo, las filas se particionan por Nombre de producto, creando un grupo de filas para cada nombre de producto. Para cada fila, la columna PERCENT_RANK expresa el rango del valor de ventas para esa fila como un porcentaje. Específicamente,

dado que se especifica ORDER BY (Ventas ASC), PERCENT_RANK devuelve el porcentaje de valores en el mismo grupo que tienen un valor inferior.

```
CREATE ROW TABLE ProductSales (ProdName NVARCHAR(50), Type NVARCHAR(20), Sales INT);
INSERT INTO ProductSales VALUES('Tee Shirt','Plain',21);
INSERT INTO ProductSales VALUES('Tee Shirt','Lettered',22);
INSERT INTO ProductSales VALUES('Tee Shirt','Team logo',30);
INSERT INTO ProductSales VALUES('Hoodie','Plain',60);
INSERT INTO ProductSales VALUES('Hoodie','Lettered',65);
INSERT INTO ProductSales VALUES('Hoodie','Team logo',80);
INSERT INTO ProductSales VALUES('Hoodie','Vintage',67);
INSERT INTO ProductSales VALUES('Ballcap','Plain',8);
INSERT INTO ProductSales VALUES('Ballcap','Lettered',40);
INSERT INTO ProductSales VALUES('Ballcap','Team logo',27);

SELECT ProdName, Type, Sales,
       PERCENT_RANK() OVER (PARTITION BY ProdName ORDER BY Sales ASC) AS Percent_Rank
FROM ProductSales
ORDER BY Sales DESC;
```

PRODNAME	Tipo	VENTAS	PERCENT_RANK
Sudadera con capucha	Logotipo del equipo	80	1
Sudadera con capucha	Vintage	67	0.6666666666666666
Sudadera con capucha	Letrado	65	0.3333333333333333
Sudadera con capucha	Sin formato	60	0
Ballcap	Letrado	40	1
Camiseta	Logotipo del equipo	30	1
Ballcap	Logotipo del equipo	27	0,5
Camiseta	Letrado	22	0,5
Camiseta	Sin formato	21	0
Ballcap	Sin formato	8	0

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Agregar funciones](#)
- [Función RANK \(ventana\)](#)

Función PERCENTILE_CONT (ventana)

Devuelve un valor interpolado mediante el valor de percentil de una constante.

Sintaxis

```
PERCENTILE_CONT( <constant_literal> ) WITHIN GROUP ( ORDER BY <expression> [ ASC | DESC ] [ NULLS F
```

Elementos de sintaxis

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Devuelve un valor interpolado utilizando el valor de percentil de <constant_literal>.

Se supone una distribución continua de los valores de <expresión>. Solo se puede especificar una única <expresión> en la cláusula ORDER BY; esta expresión debe evaluarse a un tipo numérico. El <constant_literal> debe ser un valor numérico entre, e incluyendo, de 0 a 1, porque es un valor de percentil. En la interpolación, los valores nulos se descartan en el cálculo y todos los índices se basan en 1. Los detalles de la interpolación son los siguientes:

```
NR = the number of
rows in the window partition R* P = the specified
percentile value** INDEX1 = floor( 1 + ( P * (NR - 1) ) )
INDEX2 = ceil( 1 + ( P * (NR - 1) ) )
COEF1 = INDEX2 - ( 1 + ( P * (NR - 1) ) )
COEF2 = INDEX1 - ( 1 + ( P * (NR - 1) ) )

if INDEX1 == INDEX2
then returns (value at INDEX1)
else returns ( ( value at INDEX1 ) * COEF1 + ( value at INDEX2 ) * COEF2 )
```

Ejemplos

```
DROP TABLE p_cont;
CREATE TABLE p_cont (class NVARCHAR(10), val INT, offset INT);
INSERT INTO p_cont VALUES('A', 1, 1);
INSERT INTO p_cont VALUES('A', 3, 3);
INSERT INTO p_cont VALUES('A', 5, null);
INSERT INTO p_cont VALUES('A', 5, 2);
INSERT INTO p_cont VALUES('A', 10, 0);
INSERT INTO p_cont VALUES('B', 1, 3);
INSERT INTO p_cont VALUES('B', 1, 1);
INSERT INTO p_cont VALUES('B', 7, 1);

SELECT class,
       val,
       PERCENTILE_CONT(0.125) WITHIN GROUP (ORDER BY val) OVER (PARTITION BY class) AS pc1,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY val) OVER (PARTITION BY class) AS pc2,
       PERCENTILE_CONT(0.875) WITHIN GROUP (ORDER BY val) OVER (PARTITION BY class) AS pc3
FROM p_cont;
```

CLASE	VAL	CeBe1	CeBe2	CeBe 3
A	1	2.0	5.0	7.5
A	3	2.0	5.0	7.5
A	5	2.0	5.0	7.5
A	5	2.0	5.0	7.5
A	10	2.0	5.0	7.5
B	1	1.0	1.0	5.5

CLASE	VAL	CeBe1	CeBe2	CeBe 3
B	1	1.0	1.0	5.5
B	7	1.0	1.0	5.5

Información relacionada

- [Expresiones](#)
- [Funciones de ventana y la especificación de ventana](#)
- [Función PERCENTILE_DISC \(ventana\)](#)

Función PERCENTILE_DISC (ventana)

Devuelve el primer valor cuya distribución acumulativa es mayor o igual que el valor de percentil de una constante.

Sintaxis

PERCENTILE_DISC(<constant_literal>) WITHIN GROUP (ORDER BY <expression> [ASC | DESC] [NULLS F

Elementos de sintaxis

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Devuelve el primer valor cuya distribución acumulativa es mayor o igual que el valor de percentil de <constant_literal>.

Se supone una distribución discreta de los valores de <expresión>. Solo se puede especificar una única <expresión> en la cláusula ORDER BY; esta expresión debe evaluarse a un tipo numérico. El <constant_literal> debe ser un valor numérico entre, e incluyendo, de 0 a 1, porque es un valor de percentil. El valor de retorno siempre está en el conjunto de valores de <expresión>.

Ejemplos

```
CREATE TABLE p_disc (class NVARCHAR(10), val INT, offset INT);
INSERT INTO p_disc VALUES('A', 1, 1);
INSERT INTO p_disc VALUES('A', 3, 3);
INSERT INTO p_disc VALUES('A', 5, null);
INSERT INTO p_disc VALUES('A', 5, 2);
INSERT INTO p_disc VALUES('A', 10, 0);
INSERT INTO p_disc VALUES('B', 1, 3);
INSERT INTO p_disc VALUES('B', 1, 1);
INSERT INTO p_disc VALUES('B', 7, 1);

SELECT class,
       val,
       PERCENTILE_DISC(0.125) WITHIN GROUP (ORDER BY val) OVER (PARTITION BY class) AS pd1,
       PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY val) OVER (PARTITION BY class) AS pd2,
       PERCENTILE_DISC(0.875) WITHIN GROUP (ORDER BY val) OVER (PARTITION BY class) AS pd3
FROM p_disc;
```

CLASE	VAL	PD1	PD2	PD3
A	1	1	5	10
A	3	1	5	10
A	5	1	5	10
A	5	1	5	10
A	10	1	5	10
B	1	1	1	7
B	1	1	1	7
B	7	1	1	7

Información relacionada

- [Expresiones](#)
- [Funciones de ventana y la especificación de ventana](#)
- [Función PERCENTILE_CONT \(ventana\)](#)

Función POWER (numérica)

Calcula un número base especificado elevado a la potencia de un exponente especificado.

Sintaxis

```
POWER(<base>, <exponent>)
```

Descripción

Calcula un número base especificado elevado a la potencia de un exponente especificado.

Ejemplo

El siguiente ejemplo devuelve el valor 1024:

```
SELECT POWER (2, 10) "power" FROM DUMMY;
```

Información relacionada

- [Tipos de datos numéricos](#)

Función QUARTER (fecha y hora)

Devuelve el trimestre de la fecha especificada.

Sintaxis

```
QUARTER(<date>, [, <start_month> ])
```

Descripción

Devuelve el año numérico y el trimestre de la fecha <date>. El primer trimestre empieza en el mes especificado por <start_month>. Si no se especifica <start_month>, se supone que el primer trimestre empieza en enero.

Ejemplo

El siguiente ejemplo devuelve el valor 2011 - Q4 como año y trimestre de la fecha especificada:

```
SELECT QUARTER (TO_DATE('2012-01-01', 'YYYY-MM-DD'), 2) "quarter" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función RAND (numérica)

Devuelve un valor DOUBLE pseudoaleatorio.

Sintaxis

```
RAND()
```

Descripción

Devuelve un valor DOUBLE pseudoaleatorio en el rango de 0 a menos de 1,0.

Los valores generados por la función RAND no son seguros para fines criptográficos o de seguridad. Utilice la función RAND_SECURE para valores que sean seguros para fines criptográficos o de seguridad.

Ejemplo

El siguiente ejemplo devuelve un valor DOUBLE pseudoaleatorio similar a 0,4610119133779396:

```
SELECT RAND() FROM DUMMY;
```

Información relacionada

[Función RAND_SECURE \(numérica\)](#)

[Tipos de datos numéricos](#)

Función RANDOM_PARTITION (ventana)

Divide un conjunto de entrada aleatoriamente en tres subconjuntos separados asignando un número de conjunto a cada fila del conjunto de entrada.

Sintaxis

```
RANDOM_PARTITION(<training_set_size>, <validation_set_size>, <test_set_size>, <seed>) <window_spec>
```

Elementos de sintaxis

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Los tres subconjuntos en los que RANDOM_PARTITION divide el conjunto se denominan **formación, validación y prueba**. Es posible que la unión de estos tres subconjuntos no sea el conjunto de datos inicial completo.

<training_set_size>, <validation_set_size> y <test_set_size> pueden ser valores numéricos que especifican los tamaños de los conjuntos de entrenamiento, validación y prueba, respectivamente. Cuando los tres valores son menores que uno, los tamaños se expresan como fracciones del tamaño total de entrada. Cuando los tres valores son mayores o iguales a uno, representan el número real de elementos en las diferentes particiones. <seed> especifica la semilla para la generación de números aleatorios. Cuando se establece en cero, <seed> se inicializa con la hora actual.

Los números de set se definen de la siguiente manera:

Establecer número	Fijar
0	El valor no está asignado a ningún conjunto
1	El conjunto de formación
2	El conjunto de validación
3	El conjunto de test

Las particiones estratificadas se admiten mediante la cláusula PARTITION BY.

Para generar particiones reproducibles proporcionando un valor para <seed>, asegúrese de que el conjunto de entradas se ordene de la misma manera en cada llamada a la función. El valor <seed> debe ser mayor que cero para generar particiones reproducibles; un valor de cero no crea particiones reproducibles. No utilice <seed> sin la cláusula ORDER BY. Las particiones pueden ser no deterministas entre los valores de vinculación en la <window_order_by_expression>.

Ejemplos

Particiones estratificadas

El siguiente ejemplo crea particiones estratificadas (particiones aleatorias por estación con tamaños de partición fraccionales). Dado que los números de partición se asignan aleatoriamente a las filas, el resultado puede diferir de llamada a llamada.

```
CREATE TABLE weather (station INT, ts DATE, temperature FLOAT);
INSERT INTO weather VALUES (1, '2014-01-01', 0);
INSERT INTO weather VALUES (1, '2014-01-02', 3);
```

```

INSERT INTO weather VALUES (1, '2014-01-03', 4.5);
INSERT INTO weather VALUES (1, '2014-01-04', 6);
INSERT INTO weather VALUES (1, '2014-01-05', 6.3);
INSERT INTO weather VALUES (1, '2014-01-06', 5.9);
INSERT INTO weather VALUES (2, '2014-01-01', 1);
INSERT INTO weather VALUES (2, '2014-01-02', 3.4);
INSERT INTO weather VALUES (2, '2014-01-03', 5);
INSERT INTO weather VALUES (2, '2014-01-04', 6.7);
INSERT INTO weather VALUES (2, '2014-01-05', 4.6);
INSERT INTO weather VALUES (2, '2014-01-06', 6.9);

```

```

SELECT *, RANDOM_PARTITION(0.5, 0.2, 0.3, 0) OVER (PARTITION BY station ORDER BY ts) AS part_num FF

```

ESTACIÓN	SC	TEMPERATURA	PART_NUM
1	01.01.2014	0.0	1
1	02.01.2014	3.0	1
1	03.01.2014	4.5	2
1	04.01.2014	6.0	3
1	05.01.2014	6.3	1
1	06.01.2014	5.9	3
2	01.01.2014	1.0	1
2	02.01.2014	3.4	1
2	03.01.2014	5.0	2
2	04.01.2014	6.7	3
2	05.01.2014	4.9	1
2	06.01.2014	6.9	3

Particiones aleatorias con tamaños de partición explícitos

El siguiente ejemplo crea particiones aleatorias con tamaños de partición explícitos:

```

DROP TABLE weather;
CREATE ROW TABLE weather (station INT, ts DATE, temperature FLOAT);
INSERT INTO weather VALUES(1, '2014-01-01', 0.0);
INSERT INTO weather VALUES(1, '2014-01-02', 3.0);
INSERT INTO weather VALUES(1, '2014-01-03', 4.5);
INSERT INTO weather VALUES(1, '2014-01-04', 6.0);
INSERT INTO weather VALUES(1, '2014-01-05', 6.3);
INSERT INTO weather VALUES(1, '2014-01-06', 5.9);
INSERT INTO weather VALUES(2, '2014-01-01', 1.0);
INSERT INTO weather VALUES(2, '2014-01-02', 3.4);
INSERT INTO weather VALUES(2, '2014-01-03', 5.0);
INSERT INTO weather VALUES(2, '2014-01-04', 6.7);
INSERT INTO weather VALUES(2, '2014-01-05', 4.6);
INSERT INTO weather VALUES(2, '2014-01-06', 6.9);

```

```

SELECT *, RANDOM_PARTITION(5, 2, 3, 0) OVER (ORDER BY temperature) AS part_num FROM weather;

```

ESTACIÓN	SC	TEMPERATURA	PART_NUM
1	01.01.2014	0	0

ESTACIÓN	SC	TEMPERATURA	PART_NUM
2	01.01.2014	1	1
1	02.01.2014	3	1
2	02.01.2014	3.4	1
1	03.01.2014	4.5	1
2	05.01.2014	4.6	1
2	03.01.2014	5	2
1	06.01.2014	5.9	0
1	04.01.2014	6	2
1	05.01.2014	6.3	3
2	04.01.2014	6.7	3
2	06.01.2014	6.9	3

Información relacionada

[Expresiones](#)
[Funciones de ventana y la especificación de ventana](#)

Función RAND_SECURE (numérica)

Devuelve un valor pseudoaleatorio que es seguro para fines criptográficos o de seguridad.

Sintaxis

RAND_SECURE()

Descripción

Devuelve un valor DOUBLE en el rango de 0,0 a menos de 1,0.

Los valores generados por la función RAND_SECURE son seguros para fines criptográficos o de seguridad.

La función RAND ofrece un mejor rendimiento que la función RAND_SECURE, pero devuelve un valor que no es seguro para fines criptográficos o de seguridad.

Ejemplo

El siguiente ejemplo devuelve un valor DOUBLE similar a 0,9646101191337793.

```
SELECT RAND_SECURE() from dummy;
```

Información relacionada

[Función RAND \(numérica\)](#)
[Tipos de datos numéricos](#)

Función RANK (ventana)

Devuelve el rango de una fila dentro de una partición, empezando por 1.

Sintaxis

```
RANK() <window_specification>
```

Elementos de sintaxis

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La función RANK devuelve valores duplicados en la secuencia de clasificación cuando hay vínculos entre los valores y se omiten las siguientes clasificaciones.

Ejemplos

En este ejemplo, los datos se particionan por el nombre del producto y las ventas de tipos dentro de los productos se clasifican. El ballcap liso está clasificado 4 en lugar de 3 debido a los números de ventas para el logotipo del equipo y las versiones con letras del ballcap, que están empatados en el rango 2.

```
CREATE ROW TABLE ProductSales (ProdName NVARCHAR(50), Type NVARCHAR(20), Sales INT);
INSERT INTO ProductSales VALUES('Tee Shirt','Plain',21);
INSERT INTO ProductSales VALUES('Tee Shirt','Lettered',22);
INSERT INTO ProductSales VALUES('Tee Shirt','Team logo',30);
INSERT INTO ProductSales VALUES('Hoodie','Plain',60);
INSERT INTO ProductSales VALUES('Hoodie','Lettered',65);
INSERT INTO ProductSales VALUES('Hoodie','Team logo',80);
INSERT INTO ProductSales VALUES('Ballcap','Vintage',60);
INSERT INTO ProductSales VALUES('Ballcap','Plain',8);
INSERT INTO ProductSales VALUES('Ballcap','Lettered',40);
INSERT INTO ProductSales VALUES('Ballcap','Team logo',40);

SELECT ProdName, Type, Sales,
RANK() OVER ( PARTITION BY ProdName ORDER BY Sales DESC ) AS Rank
FROM ProductSales
ORDER BY ProdName, Type;
```

PRODNAME	Tipo	VENTAS	CLASIFICACIÓN
Ballcap	Letrado	40	2
Ballcap	Sin formato	8	4
Ballcap	Logotipo del equipo	40	2
Ballcap	Vintage	60	1

PRODNAME	Tipo	VENTAS	CLASIFICACIÓN
Sudadera con capucha	Letrado	65	2
Sudadera con capucha	Sin formato	60	3
Sudadera con capucha	Logotipo del equipo	80	1
Camiseta	Letrado	22	2
Camiseta	Sin formato	21	3
Camiseta	Logotipo del equipo	30	1

Información relacionada

[Expresiones](#)
[Funciones de ventana y la especificación de ventana](#)

Función RECORD_COMMIT_TIMESTAMP (varios)

Devuelve un cronomarcador COMMIT para la fila especificada de la tabla dada.

Sintaxis

```
RECORD_COMMIT_TIMESTAMP(<table_name_or_alias>)
```

Elementos de sintaxis

<table-name-or-alias>

Especifica el nombre o alias de la tabla para la que devolver los cronomarcadores COMMIT.

Descripción

Devuelve un cronomarcador COMMIT para la fila especificada de la tabla dada si la tabla está realizando un seguimiento de los cronomarcadores COMMIT para cada fila. El cronomarcador COMMIT se basa en un contador interno que aumenta en las operaciones commit y no contiene una fecha ni una hora. Si la tabla no realiza un seguimiento de los cronomarcadores COMMIT, la función devuelve NULL. El cronomarcador COMMIT no es un cronomarcador real, sino el valor del contador interno, que aumenta en cada operación de commit de una transacción.

En función del tipo de tabla, las sentencias DDL, como ALTER TABLE, pueden afectar al valor devuelto por la función RECORD_COMMIT_TIMESTAMP:

- Para las tablas de almacenamiento en columnas, el valor devuelto no se ve afectado por las operaciones DDL.
- Para las tablas de almacenamiento en filas, el valor devuelto se cambia al cronomarcador COMMIT de la transacción que realizó la operación DDL.

Ejemplo

Ejecute las siguientes sentencias para crear y completar dos tablas, AA y BB, donde AA registra los cronomarcadores COMMIT y BB no.

```
CREATE TABLE AA (A int) RECORD COMMIT TIMESTAMP;  
CREATE TABLE BB (B int);  
INSERT INTO AA VALUES (1);  
INSERT INTO BB VALUES (1);  
COMMIT;
```

Supongamos que la transacción se confirma con un cronomarcador COMMIT de 10. La ejecución de la siguiente sentencia devuelve el cronomarcador COMMIT de la transacción, en este caso, 10.

```
SELECT RECORD_COMMIT_TIMESTAMP(AA) FROM AA;
```

La ejecución de una sentencia similar en la tabla BB devuelve NULL, ya que la opción RECORD COMMIT TIMESTAMP no estaba fijada para la tabla BB.

```
SELECT RECORD_COMMIT_TIMESTAMP(BB) FROM BB;
```

Función RECORD_ID (varios)

Genera un ID para cada fila de los resultados.

Sintaxis

```
RECORD_ID( <table> )
```

Elementos de sintaxis

<table>

Especifica el almacenamiento en columnas o la tabla de almacenamiento en filas para los que generar ID de fila.

<t> debe estar ubicado en la cláusula FROM más cercana sintácticamente y no puede ser una tabla temporal, una tabla derivada o cualquier otro tipo de subconsulta. Por ejemplo, se admiten estas sintaxis:

```
SELECT RECORD_ID(t) FROM t;  
SELECT RECORD_ID(tab) FROM t AS tab;
```

Sin embargo, no se admiten estas sintaxis:

```
SELECT RECORD_ID(v) FROM (SELECT * FROM t) AS v;  
SELECT a, RECORD_ID(st) FROM (SELECT * FROM t) AS st;
```

Descripción

Devuelve un conjunto de resultados con un valor BIGINT único para cada fila de los resultados. Estos valores siguen siendo válidos para la transacción actual.

Ejemplos

El siguiente ejemplo muestra el uso de la función RECORD_ID para seleccionar un valor de un ID de fila especificado:

```
CREATE COLUMN TABLE T1 (column1 INT);
INSERT INTO T1 VALUES (5);
INSERT INTO T1 VALUES (10);
SELECT RECORD_ID(T1) AS RECORD_ID FROM T1;
```

La consulta SELECT devuelve el siguiente resultado:

RECORD_ID
1
2

Esta consulta SELECT devuelve el siguiente resultado:

```
SELECT * FROM T1 WHERE RECORD_ID(T1) = 2;
```

COLUMN1
10

Función REPLACE (cadena)

Busca en una cadena todas las ocurrencias de una cadena especificada y las sustituye por otra cadena especificada.

Sintaxis

```
REPLACE(<original_string>, <search_string>, <replace_string>)
```

Elementos de sintaxis

<original_string>

Especifica la cadena en la que realizar la búsqueda y sustitución. Si <original_string> está vacío, el resultado es un string vacío.

<search_string>

Especifica el patrón que se debe buscar en <cadena_original>.

<replace_string>

Especifica la cadena por la que sustituir <search_string>.

Descripción

Busca en <original_string> todas las ocurrencias de <search_string> y las sustituye por <replace_string>.

- Si dos subcadenas solapadas coinciden con <search_string> en <original_string>, solo se sustituye la primera ocurrencia.
- Si <original_string> no contiene una ocurrencia de <search_string>, entonces <original_string> se devuelve sin modificar.
- Si <original_string>, <search_string> o <replace_string> son NULL, entonces se devuelve NULL.

Ejemplo

El siguiente ejemplo modifica todas las ocurrencias de **DOWN** en el string original a **UP** y devuelve el valor **UPGRADE UPWARD**:

```
SELECT REPLACE ( 'DOWNGRADE DOWNWARD', 'DOWN', 'UP' ) "replace" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función REPLACE_REGEXPR (cadena)

Busca una cadena para un patrón de expresión regular y devuelve la cadena con una o cada ocurrencia del patrón de expresión regular que se sustituye mediante una cadena de sustitución.

Sintaxis

```
REPLACE_REGEXPR(<pattern> [ FLAG <flag> ]  
IN <regex_subject_string>  
[ WITH <replacement_string> ]  
[ FROM <start_position> ]  
[ OCCURRENCE <regex_replace_occurrence> ])
```

Elementos de sintaxis

<pattern>

Especifica un patrón de búsqueda basado en Perl Compatible Regular Expression (PCRE).

```
<pattern> ::= !!Perl Compatible Regular Expression
```

<flag>

Especifica el comportamiento de coincidencia que se va a utilizar.

```
<flag> ::= 'i' | 'm' | 's' | 'x'
```

Marcar opciones

Marcar opción	Descripción
i	Activa la coincidencia que no distingue entre mayúsculas y minúsculas
m	Activa el modo multilínea, donde <i><subject_string></i> se tratará como varias líneas y la expresión ^ y \$ coinciden justo después o justo antes, respectivamente, de un terminador de línea o el final de la secuencia de entrada
s	Habilita la expresión <,> como comodín para que coincida con cualquier carácter, incluido un terminador de línea
x	Permite espacios en blanco y comentarios en el patrón
U	SAP HANA utiliza la biblioteca Expresiones regulares compatibles con Perl (PCRE) para procesar expresiones regulares. Al especificar 'U' (abreviatura de "poco codicioso") se invierte la "codicia" de los cuantificadores para que no sean codiciosos por defecto sino que se vuelvan codiciosos solo cuando vayan seguidos de "?". La coincidencia poco codiciosa a menudo puede funcionar más rápido porque encuentra la coincidencia más corta en momentos en los que solo es interesante saber si hay alguna coincidencia.

Marcar opción	Descripción
	Para una comprensión completa del comportamiento de coincidencia "codicioso" frente a "poco codicioso" de la biblioteca de expresiones regulares compatibles con Perl (PCRE), visite: https://www.pcre.org/original/doc/html/pcrematching.html ➦ .

<regex_subject_string>

Especifica la cadena a la que se debe aplicar el patrón de búsqueda. Si <regex_subject_string> está vacío, el resultado estará vacío.

<regex_subject_string> ::= <string_literal>

<replacement_string>

Especifica la cadena que sustituye todas las ocurrencias encontradas por la función. El valor predeterminado es una cadena vacía.

<replacement_string> ::= <string_literal>

<start_position>

Especifica el carácter de <regex_subject_string> en el que se inicia la búsqueda, si el valor es un entero positivo. De lo contrario, se devuelve NULL.

<start_position> ::= <numeric_literal>

<regex_replace_occurrence>

Especifica un número entero no negativo o ALL, e indica la ocurrencia de la operación de sustitución. Si <regex_replace_luck> es un entero negativo, entonces <regex_subject_string> se devuelve sin ningún cambio. El valor predeterminado es ALL.

<regex_replace_occurrence> ::= <numeric_literal> | ALL

Descripción

Busca una cadena para un patrón de expresión regular y devuelve la cadena con una o cada ocurrencia del patrón de expresión regular que se sustituye mediante una cadena de sustitución.

Si alguno de los siguientes parámetros es NULL, la función devuelve NULL: <pattern>, <flag>, <regex subject_string>, <replacement_string>, <start_position> o <regex_replace_luck>.

Ejemplo

El siguiente ejemplo utiliza una expresión regular para sustituir el formato de fecha 2014-04-01 por el valor 01/04/2014:

```
SELECT REPLACE_REGEXPR('([[:digit:]]{4})-([[:digit:]]{2})-([[:digit:]]{2})' IN '2014-04-01'
WITH '\3/\2/\1' OCCURRENCE ALL) "replace_regexpr" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función RESULT_CACHE_ID (varios)

Devuelve el ID de caché de una entrada de caché de resultados.

Sintaxis

```
RESULT_CACHE_ID()
```

Descripción

Si no se especifica ninguna entrada de caché de resultados, se devuelve NULL.

Ejemplo

El siguiente ejemplo ficticio devuelve el ID de caché para la entrada de caché de resultados V1, 40000001.

```
SELECT DISTINCT RESULT_CACHE_ID () FROM V1 WITH HINT (RESULT_CACHE);
```

Función RESULT_CACHE_REFRESH_TIME (varios)

Devuelve la última hora de actualización de caché de una entrada de caché de resultado.

Sintaxis

```
RESULT_CACHE_REFRESH_TIME()
```

Descripción

Si no se proporciona ninguna entrada de caché de resultados, se devuelve NULL.

Ejemplo

El siguiente ejemplo ficticio devuelve la última hora de actualización de caché de la entrada de caché de resultado V1, 2015-11-20 01:01:01.010101.

```
SELECT DISTINCT RESULT_CACHE_REFRESH_TIME() FROM V1;
```

Función RIGHT (cadena)

Devuelve el número especificado de caracteres/bytes de una cadena, empezando por el lado derecho.

Sintaxis

```
RIGHT(<string>, <number>)
```

Elementos de sintaxis

<string>

Especifica la cadena en la que se operará.

<number>

Especifica la longitud de caracteres o bytes a devolver, empezando por el lado derecho.

Descripción

Devuelve los <número> caracteres/bytes más a la derecha de la cadena <cadena>.

Devuelve un valor de cadena vacío si <número> es menor que 1.

Devuelve la cadena especificada por <string> (sin relleno en blanco) si el valor de <number> es mayor que la longitud de la cadena especificada por <string>.

Ejemplo

El siguiente ejemplo devuelve los 3 caracteres situados más a la derecha de la cadena indicada (789):

```
SELECT RIGHT('HI0123456789', 3) "right" FROM DUMMY;
```

El siguiente ejemplo devuelve toda la cadena de entrada porque el valor 20 supera la longitud de la cadena de entrada:

```
SELECT RIGHT('HI0123456789', 20) "right" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función ROUND (numérica)

Redondea el argumento especificado a la cantidad especificada de posiciones después de la coma decimal.

Sintaxis

```
ROUND(<number> [, <position> [, <rounding_mode>]])
```

Elementos de sintaxis

<number>

Especifica el argumento numérico que se va a redondear.

<position>

Especifica la cantidad de posiciones después de la coma decimal a la que redondear <número>. El valor predeterminado es 0. Un entero positivo o negativo se redondea a la posición especificada a la derecha o izquierda del punto decimal, respectivamente.

<rounding_mode>

Define cómo se debe realizar el redondeo.

```
<rounding_mode> ::=  
ROUND_HALF_UP  
| ROUND_HALF_DOWN  
| ROUND_HALF_EVEN  
| ROUND_UP  
| ROUND_DOWN  
| ROUND_CEILING  
| ROUND_FLOOR
```

ROUND_HALF_UP

El valor se redondea por exceso a la siguiente cifra de redondeo. Este es el valor predeterminado.

Si el valor se encuentra exactamente a mitad de camino entre dos valores redondeados, se redondea hacia arriba lejos de cero (como se hace en el redondeo comercial).

ROUND_HALF_DOWN

El valor se redondea por defecto a la siguiente cifra de redondeo. Si el valor se encuentra exactamente a mitad de camino entre dos valores redondeados, se redondea hacia abajo hacia cero.

ROUND_HALF_EVEN

El valor se redondea a la siguiente cifra de redondeo. Si el valor se encuentra exactamente a medio camino entre dos valores redondeados, se redondea al valor cuya última posición decimal es un número par.

ROUND_UP

El valor siempre se redondea de cero a la cifra redonda más grande.

ROUND_DOWN

El valor siempre se redondea hacia cero, a la cifra redonda más pequeña.

ROUND_CEILING

El valor siempre se redondea en una dirección positiva, al valor mayor.

ROUND_FLOOR

El valor siempre se redondea en una dirección negativa, al valor más pequeño.

Descripción

Redondea el argumento *<n>* a la cantidad especificada de posiciones (*<pos>*) después de la coma decimal.

Al utilizar ROUND con tipos de punto flotante, REAL y DOUBLE, la precisión de la representación numérica puede afectar al resultado. En este caso, la precisión del tipo utilizado influye en el número real de dígitos después de la coma decimal. Por ejemplo, el resultado de la siguiente sentencia es 399.7099914550781 no 399.71:

```
SELECT ROUND(TO_REAL(399.71429443359375),2) from DUMMY;
```

Ejemplos

El siguiente ejemplo devuelve el valor 16.2:

```
SELECT ROUND (16.16, 1) "round" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 20 :

```
SELECT ROUND (16.16, -1) "round" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 438,8:

```
SELECT ROUND( 438.75, 1, ROUND_HALF_UP) "round" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 438,7:

```
SELECT ROUND( 438.75, 1, ROUND_HALF_DOWN) "round" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 438,8:

```
SELECT ROUND( 438.75, 1, ROUND_HALF_EVEN) "round" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 438,8:

```
SELECT ROUND( 438.71, 1, ROUND_UP) "round" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 438,7:

```
SELECT ROUND( 438.79, 1, ROUND_DOWN) "round" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 438,8:

```
SELECT ROUND( 438.75, 1, ROUND_CEILING) "round" FROM DUMMY;
```

El siguiente ejemplo muestra la diferencia al especificar un entero positivo y negativo para *<pos>*:

```
SELECT ROUND(1234.1234, 1) "round" FROM DUMMY; ==> 1234.1000  
SELECT ROUND(1234.1234, -1) "round" FROM DUMMY; ==> 1230.0000
```

Información relacionada

[Tipos de datos numéricos](#)

Función RPAD (cadena)

Rellena a la derecha una cadena con espacios o un patrón específico para convertir una cadena en un número especificado de caracteres de longitud.

Sintaxis

```
RPAD(<string>, <number> [, <pattern>])
```

Elementos de sintaxis

<string>

Especifica la cadena que se debe rellenar.

<number>

Especifica la longitud para rellenar `<string>`. `<número>` debe ser un número entero.

`<pattern>`

Especifica una cadena de caracteres, en lugar de espacios, para utilizar para el relleno.

Descripción

Rellena a la derecha el final de `<string>` con espacios o caracteres para convertir una string en `<number>` caracteres de longitud. Si se especifica `<pattern>`, entonces `<string>` se rellena utilizando secuencias de los caracteres indicados hasta que se alcanza la longitud necesaria.

Si la longitud de `<string>` es mayor que `<n>`, no se realiza ningún relleno y el valor resultante se trunca a la longitud especificada en `<number>`.

RPAD devuelve un valor de cadena vacío si `<número>` es menor que 1.

Ejemplo

El siguiente ejemplo rellena a la derecha el final de la cadena con el patrón 12345 para hacer que una cadena tenga 15 caracteres de longitud y devuelve el valor end123451234512:

```
SELECT RPAD ('end', 15, '12345') "right padded" FROM DUMMY;
```

En el siguiente ejemplo, `<str>` es más largo que `<n>`, por lo que no se realiza ningún relleno y el resultado se trunca `<str>` a la longitud de `<n>` (es decir, en):

```
SELECT RPAD ('end', 2, '12345') "right padded" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función RTRIM (cadena)

Devuelve una cadena recortada de todos los espacios finales.

Sintaxis

```
RTRIM(<string> [,<remove_set> ])
```

Descripción

Devuelve una cadena recortada de todos los espacios finales. Si se especifica `<remove_set>`, RTRIM elimina todos los caracteres contenidos en el conjunto especificado del final de la cadena. Este proceso continúa hasta que se alcanza un carácter que no está en el `<remove_set>`.

`<remove_set>` se trata como un conjunto de caracteres y no como una cadena de búsqueda.

Ejemplo

Este ejemplo elimina todos los caracteres a y b finales de la cadena indicada y devuelve el valor endabA:

```
SELECT RTRIM ('endabAabbabab','ab') "rtrim" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función ROW_NUMBER (ventana)

Numera secuencialmente las filas dentro de una partición de un conjunto de resultados, con la primera fila de cada partición asignada como 1.

Sintaxis

```
ROW_NUMBER() <window_specification>
```

Elementos de sintaxis

```
<window_specification>
```

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

La salida de la función ROW_NUMBER puede ser no determinista entre los valores de vinculación. El orden de la secuencia viene determinado por la <windows_order_by_block> dentro de la cláusula OVER windows.

Ejemplos

El siguiente ejemplo devuelve las cifras de ventas clasificadas en orden descendente por tipo de producto. Los datos se particionan por el nombre del producto y el número de fila se devuelve para cada tipo dentro de su producto.

```
CREATE ROW TABLE ProductSales (ProdName NVARCHAR(50), Type NVARCHAR(20), Sales INT);
INSERT INTO ProductSales VALUES ('Tee Shirt','Plain',21);
INSERT INTO ProductSales VALUES ('Tee Shirt','Lettered',22);
INSERT INTO ProductSales VALUES ('Tee Shirt','Team logo',30);
INSERT INTO ProductSales VALUES ('Hoodie','Plain',60);
INSERT INTO ProductSales VALUES ('Hoodie','Lettered',65);
INSERT INTO ProductSales VALUES ('Hoodie','Team logo',80);
INSERT INTO ProductSales VALUES ('Ballcap','Plain',8);
INSERT INTO ProductSales VALUES ('Ballcap','Lettered',40);
INSERT INTO ProductSales VALUES ('Ballcap','Team logo',27);

SELECT ProdName, Type, Sales,
       ROW_NUMBER() OVER (PARTITION BY ProdName ORDER BY Sales DESC) AS row_num
FROM ProductSales
ORDER BY ProdName, Sales DESC;
```

PRODNAME	DESCRIPCIÓN	VENTAS	ROW_NUM
Ballcap	Letrado	40	1

PRODNAME	DESCRIPCIÓN	VENTAS	ROW_NUM
Ballcap	Logotipo del equipo	27	2
Ballcap	Sin formato	8	3
Sudadera con capucha	Logotipo del equipo	80	1
Sudadera con capucha	Letrado	65	2
Sudadera con capucha	Sin formato	60	3
Camiseta	Logotipo del equipo	30	1
Camiseta	Letrado	22	2
Camiseta	Sin formato	21	3

En el siguiente ejemplo, los valores ROW_NUM para las posiciones son diferentes porque no hay ninguna especificación `<windows_order_by_cláusula>`:

```
SELECT ProdName, Type, Sales,
       ROW_NUMBER() OVER (PARTITION BY ProdName) AS row_num
FROM ProductSales
ORDER BY ProdName, Sales DESC;
```

PRODNAME	DESCRIPCIÓN	VENTAS	ROW_NUM
Ballcap	Letrado	40	2
Ballcap	Logotipo del equipo	27	3
Ballcap	Sin formato	8	1
Sudadera con capucha	Logotipo del equipo	80	3
Sudadera con capucha	Letrado	65	2
Sudadera con capucha	Sin formato	60	1
Camiseta	Logotipo del equipo	30	3
Camiseta	Letrado	22	2
Camiseta	Sin formato	21	1

Información relacionada

- [Expresiones](#)
- [Funciones de ventana y la especificación de ventana](#)

Función SCORE (varios)

Devuelve la relevancia de un registro encontrado.

Sintaxis

Sintaxis 1 - Sin parámetros

```
SCORE()
```

Sintaxis 2 - Con parámetros

```
SCORE ( <search_string> IN <score_columns> )
```

Elementos de sintaxis

<score_columns>

```
<score_columns> ::=
{ <single_column>
| <specific_columns_diff_options>
| <specific_columns_same_options>
| <all_columns> }
```

<columna_individual>

```
<single_column> ::=
<column_name> [ <scoring_type> ]
```

<Tipo_puntuación>

```
<scoring_type> ::=
{ EXACT [ <exact_options> ]
| FUZZY [ <fuzzy_options> ]
| LINEAR <linear_scoring_details>
| GAUSSIAN <gaussian_scoring_details>
| LOGARITHMIC <logarithmic_scoring_details> }
```

<opción_exactas>

```
<exact_options> ::=
{ ABBREVIATION SIMILARITY <value_range>
| AND SYMMETRIC {'ON' | 'OFF'}
| AND THRESHOLD <value_range>
| MINIMAL SCORE <value_range>
| WEIGHT <value_range_2>
| COMPOSE WORDS <positive_integer_constant>
| DECOMPOSE WORDS <positive_integer_constant>
| COMPOUND WORD WEIGHT <value_range_2>
| NON MATCHING TOKEN MODE {'max' | 'min' | 'all' | 'input' | 'table'}
| EXCESS TOKEN WEIGHT <value_range>
| SEARCH MODE <search_mode>
| TOKEN SEPARATORS <token_separators>
| MIME TYPE <mime_type> }
```

<opción_fuzzy>

```
<fuzzy_options> ::=
{ ABBREVIATION SIMILARITY <value_range>
| AND SYMMETRIC {'ON' | 'OFF'}
| AND THRESHOLD <value_range>
| MINIMAL SCORE <value_range>
| WEIGHT <value_range_2>
| COMPOSE WORDS <positive_integer_constant>
| DECOMPOSE WORDS <positive_integer_constant>
| COMPOUND WORD WEIGHT <value_range_2>
| NON MATCHING TOKEN MODE {'max' | 'min' | 'all' | 'input' | 'table'}
| ERROR DEVALUATE <value_range> }
```

```

| EXCESS TOKEN WEIGHT <value_range>
| SUBSTRING MATCH { 'off' | 'anywhere' | 'beginning' }
| LANGUAGE TRANSLATION { 'on' | 'off' }
| LENGTH TOLERANCE <value_range>
| MAX DATE DISTANCE { 0 | <positive_integer_constant> }
| MINIMAL SEARCH LENGTH { 0 | <positive_integer_constant> }
| SEARCH MODE <search_mode>
| SIMILARITY CALCULATION MODE <similar_calculation_mode>
| SPELL CHECK FACTOR <value_range>
| MINIMAL TOKEN SCORE <value_range>
| TOKEN SEPARATORS <token_separators>
| MIME TYPE <mime_type> }

```

<valor_rango_1>

Un valor que es $\geq 0,0$ o ≤ 1 . Por ejemplo, 0,326, 0,57 o 0,9.

<valor_rango_2>

Un valor donde $n > 0$ y $n < 1,0$.

<constante flotante positiva>

Un número entero positivo mayor que 1,0. Por ejemplo, 57.326.

<Constante_integración_positiva>

Una constante de número entero positivo. Por ejemplo, 5.

<Constante_flotante>

Un valor no negativo.

<modo_búsqueda>

```

<search_mode> ::=
{ 'alphanum'
| 'alphanum identifier'
| 'house number'
| 'identifier'
| 'postcode'
| 'string'
| 'text' }

```

<tipo_mimo>

El tipo MIME es necesario para ejecutar una búsqueda de texto en una columna BLOB.

```

<mime_type> ::=
{ 'application/x-cscompr'
| 'application/x-abap-rawstring' }

```

Los tipos MIME que se admiten actualmente se pueden utilizar para buscar en el código fuente ABAP almacenado en REPOSRC.DATA (application/x-cscompr) y en REPOTEXT.DATA (application/x-abap-rawstring). Otros datos binarios no se pueden utilizar para la búsqueda y la aplicación debe convertirlos en un NCLOB para permitir la búsqueda de texto.

<token_separators>

Si se definen separadores de token, solo se utilizan estos caracteres para dividir un texto en tokens. Si no se especifican separadores de token, se utiliza el tokenizador predeterminado. El tokenizador predeterminado también puede dividir texto en idiomas que no utilizan espacios entre tokens (por ejemplo, chino o japonés).

```

<token_separators> ::= <string_of_separator_characters>

```

Solo se admiten caracteres ASCII como separadores. El carácter de espacio y los caracteres ASCII no impresos (código ASCII 1..31) siempre se utilizan como separadores de token.

<similar_calculation_mode>

```
<similar_calculation_mode> ::=
{ 'compare'
| 'type ahead'
| 'search compare'
| 'search'
| 'symmetric search'
| 'substring search'
| 'flexible' }
```

<detalles_de_escoración_lineal>

```
<scoring_details> ::=
[ MINIMAL SCORE <value_range> ]
[ OFFSET { 0 | <positive_float_constant> } ]
[ DECAY <linear_decay_value> ]
SCALE <positive_float_constant>
[ SEARCH MODE { 'date' | 'numc' } ]
[ WEIGHT <weight_value> ]
```

<gaussian_scoring_details>

```
<scoring_details> ::=
[ MINIMAL SCORE <value_range> ]
[ OFFSET { 0 | <positive_float_constant> } ]
[ DECAY <gaussian_decay_value> ]
SCALE <positive_float_constant>
[ SEARCH MODE { 'date' | 'numc' } ]
[ WEIGHT <weight_value> ]
```

<valor_decadencia_lineal>

Un valor mayor o igual que 0,0 pero menor que 1,0.

<valor_decay_gaussiano>

Un valor mayor que 0,0 pero menor que 1,0.

<valor_decadencia>

Un valor no inferior a 0,0 pero inferior a 1,0.

<valor_ponderado>

Un valor mayor que 0,0 y menor o igual que 1,0.

<detalles de puntuación logarítmicos>

```
<logarithmic_scoring_details> ::=
[ MINIMAL SCORE <value_range> ]
[ OFFSET { 0 | <float_constant> } ]
[ BASE <float_constant> ]
[ SEARCH MODE 'numc' ]
[ WEIGHT <weight_value> ]
```

<specific_columns_diff_options>

```
<specific_columns_diff_options> ::=
( <column_name> [ , <column_name> ],... )
```

<specific_columns_same_options>

```
<specific_columns_same_options> ::=
( <column_name> [ , <column_name> ],... ) [ <scoring_type> ]
```

<all_columns>

```
<all_columns> ::= * [ <scoring_type> ]
```

Descripción

La **sintaxis 1** solo se puede utilizar en consultas de búsqueda mediante el predicado CONTAINS. Utilice SCORE para obtener la relevancia de un registro encontrado. SCORE devuelve un valor real entre 0 y 1. La base de datos SAP HANA calcula una puntuación basada en la siguiente información:

- La relevancia, o ponderación, de los atributos en una búsqueda mediante el predicado CONTAINS. La relevancia de una coincidencia depende de la ponderación de la columna que causó la coincidencia. Puede especificar ponderaciones al crear la vista o en el predicado CONTAINS.
- Precisión en una búsqueda inexacta. Cuanto más exacta sea la coincidencia que se produzca, mayor será la puntuación.

La **sintaxis 2** no se puede utilizar con el predicado CONTAINS. Sustituye la sintaxis 1.

Ejemplos

Sintaxis 1

El siguiente ejemplo crea una tabla que contiene una cadena y le añade un índice de texto inexacto.

```
CREATE TABLE T1 (CONTENT NCLOB MEMORY THRESHOLD NULL);
CREATE FUZZY SEARCH INDEX IDX1 ON T1(CONTENT) SEARCH MODE TEXT;

INSERT INTO T1 VALUES('This is a test.');
```

Utilice la función SCORE para verificar la relevancia del contenido de la tabla con el string de búsqueda "is":

```
SELECT SCORE(), CONTENT FROM T1 WHERE CONTAINS(CONTENT, 'is', EXACT('searchMode=text'));
```

PUNTUACIÓN	CONTENIDO
1	Esto es una prueba.

Aún utilizando TABLA T1, el siguiente ejemplo añade valores adicionales a la tabla.

```
INSERT INTO T1 VALUES('example');
INSERT INTO T1 VALUES('examples');
```

Utilice la función SCORE para verificar la similitud del contenido de la tabla con la cadena "ejemplo":

```
SELECT SCORE(), CONTENT FROM T1 WHERE CONTAINS(CONTENT, 'example',FUZZY(0.8, 'searchMode=text'));
```

PUNTUACIÓN	CONTENIDO
1	ejemplo
0,9527000188827515	ejemplos

Sintaxis 2

El siguiente ejemplo crea una tabla que contiene una cadena y le añade un índice de texto inexacto:

```
CREATE TABLE T2 (CONTENT NCLOB MEMORY THRESHOLD NULL);
CREATE FUZZY SEARCH INDEX IDX2 ON T2(CONTENT) SEARCH MODE TEXT;

INSERT INTO T2 VALUES('This is a test.');
```

Utilice la función SCORE para verificar la relevancia del contenido de la tabla con el string de búsqueda "is":

```
SELECT SCORE('is' IN CONTENT EXACT SEARCH MODE 'text') AS SCORE, CONTENT FROM T2 WHERE SCORE('is' IN
```

PUNTUACIÓN	CONTENIDO
1	Esto es una prueba.

Aún utilizando TABLA T2, el siguiente ejemplo añade valores adicionales a la tabla.

```
INSERT INTO T2 VALUES('example');
INSERT INTO T2 VALUES('examples');
```

Utilice la función SCORE para verificar la similitud del contenido de la tabla con la cadena "ejemplo":

```
SELECT SCORE('example' IN CONTENT FUZZY MINIMAL SCORE 0.8 MINIMAL TOKEN SCORE 0.8 SEARCH MODE 'text') AS SCORE, CONTENT FROM T2 WHERE SCORE('example' IN CONTENT FUZZY MINIMAL SCORE 0.8 MINIMAL TOKEN SCORE 0.8 SEARCH MODE 'text')
```

PUNTUACIÓN	CONTENIDO
1	ejemplo
0,9527000188827515	ejemplos

Información relacionada

- [Función GROUP SCORE \(varios\)](#)
- [CONTIENE el predicado](#)
- [Asignaciones de términos](#)

Función SECOND (fecha y hora)

Devuelve la segunda parte de una hora especificada.

Sintaxis

```
SECOND(<time>)
```

Descripción

Devuelve un valor de segundos para una hora determinada.

Los subsegundos se incluyen para los tipos de datos TIMESTAMP.

Ejemplos

El siguiente ejemplo devuelve el valor 56 para los segundos de la hora especificada:

```
SELECT SECOND ( '12:34:56' ) "second" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 56,789 para los subsegundos de la hora especificada:

```
SELECT SECOND ( '2014-03-25 12:34:56.789' ) "subseconds" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función SECONDS_BETWEEN (fecha y hora)

Calcula el número de segundos entre dos fechas especificadas.

Sintaxis

```
SECONDS_BETWEEN(<date_1>, <date_2>)
```

Descripción

Calcula el número de segundos comprendidos entre los argumentos de fecha <date_1> y <date_2>, que semánticamente es igual a (<date_2> - <date_1>).

Ejemplo

El siguiente ejemplo devuelve el valor 2.678.400 como número de segundos entre las dos fechas especificadas:

```
SELECT SECONDS_BETWEEN ( '2009-12-05', '2010-01-05' ) "seconds between" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función SERIES_DISAGGREGATE (datos de serie)

Genera una tabla de series completa con filas desagregadas en particiones definidas.

Sintaxis

```

SERIES_DISAGGREGATE(
    <source_increment_by>, <target_increment_by>
    [, <value_range> ] )
    { SERIES_DISAGGREGATE_TINYINT
    SERIES_DISAGGREGATE_SMALLINT
    SERIES_DISAGGREGATE_INTEGER
    SERIES_DISAGGREGATE_BIGINT
    SERIES_DISAGGREGATE_SMALLDECIMAL
    SERIES_DISAGGREGATE_DECIMAL
    SERIES_DISAGGREGATE_TIME
    SERIES_DISAGGREGATE_DATE
    SERIES_DISAGGREGATE_SECONDDATE
    SERIES_DISAGGREGATE_TIMESTAMP }
    (<source_increment_by>, <target_increment_by>, <min_value>, <max_value>)

```

Elementos de sintaxis

<value_range>

Define el rango de valores.

```

<value_range> ::= <min_value> [, <max_value>]

<min_value> ::= <real_const> | <datetime_const>
<max_value> ::= <real_const> | <datetime_const>

```

El <min_value> es el <min_value> mayor de las dos tablas especificadas y el <max_value> es el <max_value> menor de las dos tablas especificadas.

<source_increment_by>

Establece el valor de incremento de origen.

```

<source_increment_by> ::= <real_const> | <datetime_const>

```

<target_increment_by>

Establece el valor de incremento de destino.

```

<target_increment_by> ::= <real_const> | <datetime_const>

```

<constant_literal>

El parámetro <constant_literal> se define como <real_const> o <datetime_const>.

```

<constant_literal> ::= <real_const> | <datetime_const>

```

Es una constante de número entero, una constante de intervalo o un número definido de elementos entre un mínimo y un máximo. Si el tipo de período es DATETIME, el número debe estar precedido por INTERVAL seguido de una unidad de tiempo, como YEAR, MONTH, DAY, HOUR, MINUTE o SECOND. Se permite la notación exponencial.

Descripción

Para los tipos de fecha/hora, el parámetro <increment_by> debe ser una cadena con el formato "INTERVAL unidades numéricas" donde las unidades son YEAR, MONTH, DAY, HOUR, MINUTE o SECOND y el número debe ser un número entero, a menos que el tipo de unidades sea SECOND y el tipo de datos asociado sea TIMESTAMP.

El parámetro <min_value> para las versiones numéricas de esta función no necesita alinearse con cero numérico con respecto al valor <increment_by>. Puede especificar un <source_increment> de 2 y un <min_value> de 1, lo que da como resultado una serie de fuentes de 1, 3, 5, etc.

Esta función devuelve una tabla con las siguientes columnas:

Nombre de columna	Tipo de columna	Descripción
SOURCE_PERIOD_START	PERIOD_TYPE	Especifica el inicio del período de origen que ha generado esta fila.
SOURCE_PERIOD_END	PERIOD_TYPE	Especifica el fin del período de origen que ha generado esta fila.
GENERATED_PERIOD_START	PERIOD_TYPE	Muestra el inicio del período representado por esta fila, incluido el período_inicio. Este valor es un intervalo cerrado al inicio.
GENERATED_PERIOD_END	PERIOD_TYPE	Muestra el fin del período representado por esta fila como un intervalo abierto. El período representado por esta fila consta de todos los tiempos que son mayores o iguales que el inicio y menores que el final.
ELEMENT_NUMBER_IN_SOURCE_PERIOD	BIGINT	Especifica el número de elemento de este período dentro de su intervalo de origen.
ELEMENT_NUMBER_IN_GENERATED_SERIES	BIGINT	Especifica el número de elemento dentro de todo el conjunto de resultados.
FRACTION_OF_SOURCE_PERIOD	DOUBLE	Especifica la fracción de la duración del período de origen que cubre este período generado.
FRACTION_OF_MIN_MAX_RANGE	DOUBLE	Especifica la fracción de la duración de todos los períodos generados que cubre este período generado.

Ejemplos

El siguiente ejemplo genera una serie de fechas que van del 01/01/1999 al 04/01/2001:

```
SELECT * FROM SERIES_DISAGGREGATE_DATE
      ('INTERVAL 1 year', 'INTERVAL 3 MONTH', '1999-01-01', '2001-01-04' );
```

El siguiente resultado se devuelve en el ejemplo anterior:

SOURCE_PERIOD_START	SOURCE_PERIOD_END	GENERATED_PERIOD_START	GENERATED_PERIOD_END	ELEMENT_NUMBE
1 de enero de 1999	1 de enero de 2000	1 de enero de 1999	1 de abril de 1999	1
1 de enero de 1999	1 de enero de 2000	1 de abril de 1999	1 de julio de 1999	1
1 de enero de 1999	1 de enero de 2000	1 de julio de 1999	1 de octubre de 1999	1
1 de enero de 1999	1 de enero de 2000	1 de octubre de 1999	1 de enero de 2000	1
1 de enero de 2000	1 de enero de 2001	1 de enero de 2000	1 de abril de 2000	2
1 de enero de 2000	1 de enero de 2001	1 de abril de 2000	1 de julio de 2000	2
1 de enero de 2000	1 de enero de 2001	1 de julio de 2000	1 de octubre de 2000	2

SOURCE_PERIOD_START	SOURCE_PERIOD_END	GENERATED_PERIOD_START	GENERATED_PERIOD_END	ELEMENT_NUMBE
1 de enero de 2000	1 de enero de 2001	1 de octubre de 2000	1 de enero de 2001	2

Función SERIES_ELEMENT_TO_PERIOD (datos de serie)

Devuelve el valor de período de serie asociado con el número de elemento de serie basado en uno especificado.

Sintaxis

```
SERIES_ELEMENT_TO_PERIOD( <element_number>, { <increment_by>, <min_value>, <max_value> } )
```

Elementos de sintaxis

<element_number>

Especifica el número de elemento.

```
<element_number> ::= INTEGER
```

<increment_by>

Se define como una constante de número entero, una constante de intervalo o un número definido de elementos entre un mínimo y un máximo.

```
<increment_by> ::= <real_const> | <interval_const>
```

Si el tipo de período es DATETIME, el número debe estar precedido por INTERVAL seguido de una unidad de tiempo, como YEAR, MONTH, DAY, HOUR, MINUTE o SECOND. Se permite la notación exponencial.

<min_value>

Especifica el valor mínimo. Puede ser un valor numérico o de fecha/hora.

```
<min_value> ::= <real_const> | <datetime_const>
```

<max_value>

Especifica el valor máximo. Puede ser un valor numérico o de fecha/hora.

```
<max_value> ::= <real_const> | <datetime_const>
```

Descripción

Devuelve el valor del período de serie asociado con el número de elemento de serie basado en uno proporcionado, donde $\text{rounded_period} = \text{SERIES_ROUND}(\text{período}, \text{intervalo}, \text{round_mode})$ y $\text{elemento} = 1 + (\text{rounded_period} - \text{min_value}) / \text{intervalo}$.

Ejemplos

El siguiente ejemplo elige el quinto elemento de una serie de 0 a 10 en incrementos de 2 y devuelve 8:

```
SELECT SERIES_ELEMENT_TO_PERIOD(5, 2, 0, 10) "val" FROM DUMMY;
```

El siguiente ejemplo elige el sexto elemento de una serie de 1 a 10 en incrementos de 1,25 y devuelve 7,25:

```
SELECT SERIES_ELEMENT_TO_PERIOD(6, 1.25, 1, 10) "val" FROM DUMMY;
```

El siguiente ejemplo elige el séptimo elemento de una serie de fechas del 01/01/2014 al 31/12/2014 en incrementos de 1 día y devuelve el 7 de enero de 2014:

```
SELECT SERIES_ELEMENT_TO_PERIOD(7, 'INTERVAL 1 DAY', '2014-01-01', '2014-12-31')
       "val" FROM DUMMY;
```

El siguiente ejemplo elige el séptimo elemento de una serie de fechas del 01/01/2014 al 31/12/2014 en incrementos de 1 mes y devuelve 1 de julio de 2014:

```
SELECT SERIES_ELEMENT_TO_PERIOD(7, 'INTERVAL 1 MONTH',
       '2014-01-01', '2014-12-31') "val" FROM DUMMY;
```

El siguiente ejemplo toma el elemento 500.000th de una serie cronológica de 2014-01-01 a 2014-12-31 en incrementos de 1,5 segundos y devuelve 9 de enero de 2014 4:19:58.5 PM:

```
SELECT SERIES_ELEMENT_TO_PERIOD(500000, 'INTERVAL 1.5 SECOND',
       '2014-01-01 00:00:00.000', '2014-12-31') "val" FROM DUMMY;
```

Función SERIES_FILTER (ventana)

Realiza un cálculo filtrado en una serie de datos.

Sintaxis

```
SERIES_FILTER(<filter_parameter> => <expression> [, <filter_parameter> => <expression> ... ])
OVER (
  [ <series_definition> | <series_reference> ]
  [ <window_partition_by_clause> ]
  [ ORDER BY <window_order_by_expression> ]
  [ <window_frame_clause> ]
)
```

Elementos de sintaxis

<filter_parameter>

<filter_parameter> ::= VALUE | METHOD_NAME | ALPHA | BETA

VALUE es un parámetro obligatorio que especifica la columna a la que se aplica el filtro. La columna debe representar datos numéricos. NULL valores no permitidos.

METHOD_NAME es un parámetro obligatorio que especifica el nombre del método de filtro. Especifique SINGLESMOOTH para el alisamiento exponencial simple o DOUBLESMOOTH para el alisamiento exponencial doble.

ALPHA es un parámetro decimal de alisamiento opcional para el nivel de la serie que oscila entre 0 y 1. El valor predeterminado es 0,1.

BETA es un parámetro decimal de alisamiento para la tendencia de la serie que oscila entre 0 y 1. El valor predeterminado es 0,1. Este parámetro solo es necesario para el alisamiento exponencial doble.

SERIES_FILTER implementa varios algoritmos de filtrado que se identifican por METHOD_NAME. Los valores NULL finales de la serie de entrada se sustituyen por valores previstos.

Todos los algoritmos de filtro requieren que la serie sea equidistante y no debe contener ningún valor que falte. La función devuelve un error cuando se llama con una propiedad de serie que indica una propiedad que no coincide.

<window_partition_by_clause>

Especifica una o más particiones que se crearán utilizando las expresiones proporcionadas.

```
<window_partition_by_clause> ::= PARTITION BY <expression> [ { , <expression> } ... ]
```

<window_order_by_expression>

Determina el orden de la entrada. Solo se debe proporcionar una expresión en la <window_order_by_expression> para esta función.

Esta expresión no debe contener valores nulos ni duplicados. Consulte la descripción de la cláusula <window_order_by_cláusula> en el tema *Funciones de ventana* para obtener otra información de sintaxis para esta cláusula.

<window_frame_clause>

Limita físicamente las filas dentro de una partición especificando un número de filas que preceden o siguen a la fila actual.

```
<window_frame_clause> ::= <window_frame_unit> <window_frame_extent>
```

```
<window_frame_unit> ::= ROWS
```

```
<window_frame_extent> ::= <window_frame_start> | <window_frame_between>
```

```
<window_frame_start> ::= UNBOUNDED PRECEDING | <window_frame_preceding> | CURRENT ROW
```

```
<window_frame_preceding> ::= <unsigned_integer> PRECEDING
```

```
<window_frame_between> ::= BETWEEN <lower_window_frame_bound> AND <upper_window_frame_bound>
```

```
<lower_window_frame_bound> ::= <window_frame_bound>
```

```
<upper_window_frame_bound> ::= <window_frame_bound>
```

```
<window_frame_bound> := <window_frame_start> | UNBOUNDED FOLLOWING | <window_frame_following>
```

```
<window_frame_following> ::= <unsigned_integer> FOLLOWING
```

ROWS requiere que se especifique una cláusula ORDER BY.

UNBOUND PRECEDING especifica que la ventana empieza en la primera fila de la partición. Solo se puede especificar como <window_frame_start>. CURRENT ROW especifica que la ventana empieza o termina en la fila actual cuando se utiliza con FILAS.

PRECEDING indica que el número de filas o valores que deben preceder a la fila actual. <unsigned_integer> debe ser 0 o un literal de entero positivo.

<lower_window_frame_bound> y <upper_window_frame_bound> especifican el límite inferior (inicial) y el superior (final) del marco de ventana. El límite superior no puede ser menor que el límite inferior.

UNBOUNDED FOLLOWING especifica que la ventana termina en la última fila de la partición. UNBOUNDED FOLLOWING solo se puede especificar como un final de marco de ventana.

FOLLOWING indica el número de filas o valores a seguir a la fila actual. Cuando se especifica como el inicio del marco de ventana, el fin del marco de ventana debe ser <unsigned_integer> FOLLOWING. El <unsigned_integer> debe ser 0 o un literal de entero positivo.

Descripción

Las funciones de ventana le permiten dividir los conjuntos de resultados de una consulta, o una partición lógica de una consulta, en grupos de filas llamadas particiones de ventana.

Una partición de ventana se especifica mediante una o más expresiones en la cláusula OVER.

Las funciones de ventana que no sean `ROW_NUMBER`, `PERCENTILE_DISC`, `PERCENTILE_CONT` y las funciones de agregado de ventanas deben tener una cláusula `ORDER BY` en la cláusula `OVER`.

Los conjuntos de resultados primero se particionan según lo especificado por la cláusula `PARTITION BY`, y luego se ordenan por `ORDER BY` especificación de cláusula dentro de la partición de ventana. A continuación, las funciones de ventana se aplican a cada fila dentro de los límites de partición de ventana.

La cláusula `ORDER BY` en la cláusula `OVER` solo evalúa la función de ventana de modo que el orden de las filas resultantes no sea determinista si no se especifica por `ORDER BY` para `SELECT`.

El marco de ventana predeterminado de la función de ventana depende de si se especifica una cláusula `ORDER BY`. Si se especifica una cláusula `ORDER BY`, el marco de ventana predeterminado se convierte en 'entre `UNBOUNDED PRECEDING` y `CURRENT ROW`': La función de ventana calcula en filas anteriores o iguales a la fila actual. Como resultado, la función devuelve valores acumulados.

Si no se especifica una cláusula `ORDER BY`, el marco de ventana predeterminado se convierte en 'entre `UNBOUNDED PRECEDING` y `UNBOUNDED FOLLOWING`': La función de ventana calcula toda la partición de ventana. Como resultado, la función devuelve el mismo valor dentro de una partición de ventana independientemente de la fila actual.

Ejemplos

El siguiente ejemplo ilustra el alisamiento exponencial simple y doble con la previsión.

```
DROP TABLE weather;
CREATE COLUMN TABLE weather (ts DAYDATE, temperature FLOAT);
INSERT INTO weather VALUES('2014-01-01', 0);
INSERT INTO weather VALUES('2014-01-02', 3);
INSERT INTO weather VALUES('2014-01-03', 4.5);
INSERT INTO weather VALUES('2014-01-04', 6);
INSERT INTO weather VALUES('2014-01-05', 6.3);
INSERT INTO weather VALUES('2014-01-06', 6.9);
INSERT INTO weather VALUES('2014-01-07', NULL);
INSERT INTO weather VALUES('2014-01-08', NULL);

SELECT ts, temperature,
       SERIES_FILTER(VALUE => temperature, METHOD_NAME => 'SINGLESMOOTH', ALPHA => 0.2) OVER (ORDER BY ts) AS SMOOTH1,
       SERIES_FILTER(VALUE => temperature, METHOD_NAME => 'DOUBLESMOOTH', ALPHA => 0.2, BETA => 0.3) OVER (ORDER BY ts) AS SMOOTH2
FROM weather;
```

SC	TEMPERATURA	HES	DES
01.01.2014	0,0	NULL	NULL
02.01.2014	3.0	0	3
03.01.2014	4.5	0.6000000000000001	6
04.01.2014	6.0	1.3800000000000001	8.61
05.01.2014	6.3	2.3040000000000003	10.8414
06.01.2014	6.9	3.1032	12.414036
07.01.2014	NULL	3.86256	13.46130264
08.01.2014	NULL	3.86256	15.61137648

El siguiente ejemplo utiliza un marco de ventana `ROWS` especificación:

```
CREATE TABLE T (class NVARCHAR(10), val INT, offset INT);
INSERT INTO T VALUES('A', 1, 1);
INSERT INTO T VALUES('A', 3, 3);
INSERT INTO T VALUES('A', 5, null);
INSERT INTO T VALUES('A', 5, 2);
INSERT INTO T VALUES('A', 10, 0);
INSERT INTO T VALUES('B', 1, 3);
INSERT INTO T VALUES('B', 1, 1);
INSERT INTO T VALUES('B', 7, 1);

SELECT class,
       val,
       SUM(val) OVER (PARTITION BY class ORDER BY val) AS s0,
       SUM(val) OVER (PARTITION BY class ORDER BY val ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOI
       SUM(val) OVER (PARTITION BY class ORDER BY val ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
       SUM(val) OVER (PARTITION BY class ORDER BY val ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)
       SUM(val) OVER (PARTITION BY class ORDER BY val ROWS BETWEEN UNBOUNDED PRECEDING AND 1 FOLLOWING)
       SUM(val) OVER (PARTITION BY class ORDER BY val ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)
       SUM(val) OVER (PARTITION BY class ORDER BY val ROWS BETWEEN CURRENT ROW AND CURRENT ROW) AS s6,
       SUM(val) OVER (PARTITION BY class ORDER BY val ROWS BETWEEN CURRENT ROW AND 1 FOLLOWING) AS s7
FROM T;
```

CLASE	VAL	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16
A	1	1	24	1	nulo	4	24	1	4	24	1	nulo	4	23	8	1	1	1
A	3	4	24	4	1	9	23	3	8	24	4	1	9	20	10	4	3	4
A	5	14	24	9	4	14	20	5	10	23	8	4	13	15	15	9	5	8
A	5	14	24	14	9	24	15	5	15	20	10	8	20	10	10	14	5	10
A	10	24	24	24	14	24	10	10	10	15	15	10	15	nulo	nulo	24	10	15
B	1	2	9	1	nulo	2	9	1	2	9	1	nulo	2	8	8	1	1	1
B	1	9	9	2	1	9	8	1	8	9	2	1	9	7	7	2	1	2
B	7	9	9	9	2	9	7	7	7	8	8	2	8	nulo	nulo	9	7	8

Información relacionada

- [Expresiones](#)
- [Funciones de ventana y la especificación de ventana](#)

Función SERIES_GENERATE (datos de serie)

Genera una tabla de series completa basada en la definición de serie especificada.

Sintaxis

```
SERIES_GENERATE_<generation_spec>
```

Elementos de sintaxis

```
<generation_spec>
```

Una versión de la función que especifica el tipo de datos para los parámetros generados.

<generation_spec> ::= <data_type> <generate_parameters>

<tipo_datos>

El tipo de datos a generar.

<data_type> ::=
TINYINT
| SMALLINT
| INTEGER
| BIGINT
| SMALLDECIMAL
| DECIMAL
| TIME
| DAYDATE
| SECONDDATE
| TIMESTAMP

<generate_parameter>

Especifica el valor de incremento utilizado para generar parámetros.

<generate_parameters> ::=
(<increment_by>, <min_value>, <max_value>)

<increment_by> ::= { <numeric_literal> | INTERVAL <date_literal> }

<increment_by> define el tamaño del período. Si el tipo de período es DATETIME, el número debe estar precedido por INTERVAL seguido de una unidad de tiempo, como YEAR, MONTH, DAY, HOUR, MINUTE o SECOND.

Descripción

El rango se define mediante los parámetros <min_value> y <max_value>, y el tamaño del período se define mediante el parámetro <increment_by>. <min_value> y <max_value> especifica el valor de incremento utilizado para generar valores debe alinearse con el valor <increment_by>. Los intervalos generados tienen una semántica cerrada y abierta, por lo que GENERATED_PERIOD_END no pertenece al intervalo.

El parámetro <min_value> para las versiones numéricas de esta función no necesita alinearse con cero numérico con respecto al valor <increment_by>. Puede especificar un incremento de origen de 2 y un <min_value> de 1, lo que da como resultado una serie de fuentes de 1, 3, 5, etc.

Esta función devuelve una tabla con las siguientes columnas:

Nombre de columna	Tipo de columna	Descripción
GENERATED_PERIOD_START	PERIOD_TYPE	Especifica el inicio del período representado por esta fila, incluido el parámetro <period_start>. Se trata de un intervalo cerrado al inicio.
GENERATED_PERIOD_END	PERIOD_TYPE	Especifica el fin del período representado por esta fila como un intervalo abierto. El período representado por esta fila consta de todos los tiempos que son mayores o iguales que el inicio y menores que el final.
ELEMENT_NUMBER	BIGINT	Especifica el número de elemento de este período dentro de la serie generada. Este

		valor es equivalente al número de fila del conjunto de resultados.
FRACTION_OF_MIN_MAX_RANGE	DOBLE	Especifica la relación entre la duración de este período y la longitud de todos los períodos generados por esta función. La suma de FRACTION_OF_MIN_MAX_RANGE es cercana a 1.0. Para intervalos de tamaño fijo, como 1 DÍA, el valor se calcula como 1/numPeriods. Para intervalos de tamaño no fijos, como MES y AÑO, el valor se calcula individualmente para cada fila, que se calcula por el intervalo de tiempo del intervalo dividido por el intervalo de tiempo de todo el resultado.

Ejemplos

El siguiente ejemplo genera una serie de decimales que van del 0 al 10 que aumenta en 2,5:

```
SELECT * FROM SERIES_GENERATE_DECIMAL(2.5, 0, 10);
```

El siguiente resultado se devuelve en el ejemplo anterior:

GENERATED_PERIOD_START	GENERATED_PERIOD_END	ELEMENT_NUMBER	FRACTION_OF_MIN_MAX_RANGE
0	2.5	1	0,25
2.5	5	2	0,25
5	7.5	3	0,25
7.5	10	4	0,25

El siguiente ejemplo genera una serie de enteros que van del 1 al 5 que se incrementa en 2:

```
SELECT * FROM SERIES_GENERATE_INTEGER(2, 1, 5);
```

El siguiente resultado se devuelve en el ejemplo anterior:

GENERATED_PERIOD_START	GENERATED_PERIOD_END	ELEMENT_NUMBER	FRACTION_OF_MIN_MAX_RANGE
1	3	1	0,5
3	5	2	0,5

El siguiente ejemplo genera una serie de cronomarcadores que van del 01/01/1999 al 02/01/1999 que aumenta en intervalos de 30 segundos:

```
SELECT * FROM SERIES_GENERATE_TIMESTAMP(  
    'INTERVAL 30 SECOND', '1999-01-01', '1999-01-02');
```

El siguiente resultado se devuelve en el ejemplo anterior:

GENERATED_PERIOD_START	GENERATED_PERIOD_END	ELEMENT_NUMBER	FRACTION_OF_MIN_MAX_RANGE
1 de enero de 1999 12:00:00.0 AM	1 de enero de 1999 12:00:30.0 AM	1	0.0003472222222222224
1 de enero de 1999 12:00:30.0 AM	1 de enero de 1999 12:01:00.0 AM	2	0.0003472222222222224
1 de enero de 1999 12:01:00.0 AM	1 de enero de 1999 12:01:30.0 AM	3	0.0003472222222222224
1 de enero de 1999 12:01:30.0 AM	1 de enero de 1999 12:02:00.0 AM	4	0.0003472222222222224
...			
1 de enero de 1999 11:59:00.0 PM	1 de enero de 1999 11:59:30.0 PM	2.879	0.0003472222222222224
1 de enero de 1999 11:59:30.0 PM	2 de enero de 1999 12:00:00.0 AM	2.880	0.0003472222222222224

Función SERIES_PERIOD_TO_ELEMENT (datos de serie)

Devuelve el número de elemento de serie basado en uno al que está asociado el valor de período indicado.

Sintaxis

```
SERIES_PERIOD_TO_ELEMENT(  
  <value>, {<increment_by>, <min_value>, <max_value> [, <rounding_mode>]  
  [, <rounding_mode>]}  
)
```

Elementos de sintaxis

<value>

Especifica que el parámetro <value> puede ser un valor numérico o un tipo de fecha/hora.

```
<value> ::= INTEGER | DOUBLE | TIMESTAMP
```

<increment_by>

Establece el valor de incremento.

```
<increment_by> ::= <identifier>
```

Si el parámetro <value> es un tipo de fecha/hora, entonces el parámetro <increment_by> debe ser una cadena con el formato "Unidades de número INTERVAL" donde las unidades son AÑO, MES, DÍA, HORA, MINUTO o SEGUNDO y el número debe ser un entero, a menos que el tipo de unidad sea SECOND y el tipo de datos asociado sea TIMESTAMP. Si el valor es numérico, el parámetro <increment_by> debe ser un valor numérico que defina el período de la serie.

<min_value>

Fija el valor de período mínimo.

```
<min_value> ::= <integer>
```

Para una serie equidistante, existe una asignación entre períodos y elementos. Los períodos se encuentran en el espacio de las columnas de períodos, normalmente cronomarcadores. Los elementos son siempre BIGINT. El elemento 1 representa el primer período, asociado con *<min_value>*.

<max_value>

Fija el valor de período máximo.

```
<max_value> ::= <integer>
```

<rounding_mode>

Especifica que si el valor de período no está en un valor de período de serie, se redondea al valor de período de serie más cercano, que devuelve el número de elemento asociado.

```
<rounding_mode> ::= ROUND_HALF_UP
| ROUND_HALF_DOWN
| ROUND_HALF_EVEN
| ROUND_UP
| ROUND_DOWN
| ROUND_CEILING
| ROUND_FLOOR
```

Los modos de redondeo admitidos son:

Modo de redondeo	Descripción
ROUND_HALF_UP	Especifica que el valor se redondea al valor de serie más cercano. Los valores que se encuentran a mitad de camino entre dos valores de serie se redondean hacia arriba lejos de cero. Este es el valor predeterminado.
ROUND_HALF_DOWN	Especifica que el valor se redondea al valor de serie más cercano. Los valores que se encuentran a mitad de camino entre dos valores redondos se redondean hacia abajo hacia cero.
ROUND_HALF_EVEN	Especifica que el valor se redondea al valor de serie más cercano. Los valores que se encuentran a mitad de camino entre dos valores redondeados se redondean al valor de serie par basado en el número de elemento.
ROUND_UP	Especifica que el valor siempre se redondea de cero al valor de serie mayor.
ROUND_DOWN	Especifica que el valor siempre se redondea hacia cero al valor de serie más pequeño.
ROUND_CEILING	Especifica que el valor siempre se redondea en una dirección positiva al valor de serie mayor.
ROUND_FLOOR	Especifica que el valor siempre se redondea en una dirección negativa al valor de serie más pequeño.

<series_table>

Si se utiliza una referencia de tabla de serie, especifica que los valores para *<increment_by>*, *<min_value>* y *<max_value>* de la definición SERIES TABLE se utilizan como parámetros para esta función.

```
<series_table> ::= <identifier>
```

Descripción

Devuelve el número de elemento de serie basado en uno con el que se asocia el valor de período indicado, donde período = $\text{min_value} + (\text{elemento} - 1) * \text{intervalo}$.

Ejemplos

El siguiente ejemplo devuelve el resultado 4:

```
SELECT SERIES_PERIOD_TO_ELEMENT(5, 2, 0, 10, ROUND_HALF_UP) "element" FROM DUMMY;
```

El siguiente ejemplo devuelve el resultado 3:

```
SELECT SERIES_PERIOD_TO_ELEMENT(5, 2, 0, 10, ROUND_HALF_DOWN) "element" FROM DUMMY;
```

El siguiente ejemplo recoge el siguiente elemento redondeado a la baja de 2014-01-05 12:00:00 de una serie de fechas de 2014-01-01 a 2014-12-31 en incrementos de 1 día. Devuelve el resultado 5:

```
SELECT SERIES_PERIOD_TO_ELEMENT(
  '2014-01-05 12:00:00',
  'INTERVAL 1 DAY',
  '2014-01-01',
  '2014-12-31',
  ROUND_HALF_DOWN) "element" FROM DUMMY;
```

Función SERIES_ROUND (datos de serie)

Redondea un valor especificado al valor de serie utilizando las opciones de redondeo especificadas.

Sintaxis

```
SERIES_ROUND( <value>, <increment_by>[, <rounding_mode> [, <alignment_expression> ] ] )
```

Elementos de sintaxis

<value>

Especifica que el parámetro de valor puede ser un valor numérico o un tipo de fecha/hora.

```
<value> ::= { <real_const> | <datetime_const> }
```

<increment_by>

Si el parámetro <value> es numérico, entonces el valor <increment_by> debe ser un valor numérico que defina el período de la serie.

```
<increment_by> ::= <interval_const>
```

Si el parámetro <value> es un tipo de fecha/hora, entonces el valor <increment_by> debe ser una cadena con el formato "Unidades de número INTERVAL" donde las unidades son AÑO, MES, DÍA, HORA, MINUTO o SEGUNDO y el número debe ser un entero, a menos que el tipo de unidad sea SECOND y el tipo de datos asociado sea TIMESTAMP.

<rounding_mode>

Especifica que el cero natural que sirve como base para el redondeo depende del tipo de datos y del intervalo.

```
<rounding_mode> ::= ROUND_HALF_UP
| ROUND_HALF_DOWN
| ROUND_HALF_EVEN
| ROUND_UP
| ROUND_DOWN
| ROUND_CEILING
| ROUND_FLOOR
```

Para tipos numéricos, cero es el cero numérico. Para los tipos de fecha con las unidades de intervalo DAY o inferiores, cero es medianoche. Para los intervalos MES y AÑO, cero es la medianoche del primer día del mes y la medianoche del primer día del año, respectivamente. Se admiten los siguientes modos de redondeo:

Modo	Descripción
ROUND_HALF_UP	Especifica el valor predeterminado. El valor se redondea al valor de serie más cercano. Los valores que se encuentran a mitad de camino entre dos valores de serie se redondean hacia arriba, lejos de cero.
ROUND_HALF_DOWN	Especifica que el valor se redondea al valor de serie más cercano. Los valores que se encuentran a mitad de camino entre dos valores redondos se redondean hacia abajo, hacia cero.
ROUND_HALF_EVEN	Especifica que el valor se redondea al valor de serie más cercano. Los valores que se encuentran a mitad de camino entre dos valores redondeados se redondean al valor de serie par, en función del número de elemento.
ROUND_UP	Especifica que el valor siempre se redondea de cero al valor de serie mayor.
ROUND_DOWN	Especifica que el valor siempre se redondea hacia cero al valor de serie más pequeño.
ROUND_CEILING	Especifica que el valor siempre se redondea en una dirección positiva al valor de serie mayor.
ROUND_FLOOR	Especifica que el valor siempre se redondea en una dirección negativa al valor de serie más pequeño.

<alignment_expression>

Especifica que este parámetro debe ser convertible al tipo de datos <value>.

```
<alignment_expression> ::= { <real_const> | <datetime_const> }
```

Se debe especificar un <rounding_mode> al utilizar este parámetro.

Descripción

Redondea un valor especificado al valor de serie utilizando las opciones de redondeo especificadas.

Ejemplos

El siguiente ejemplo muestra cómo redondear 4,5 a una serie incrementada en 3. Devuelve el resultado 6:

```
SELECT SERIES_ROUND(4.5, 3, ROUND_HALF_UP) "result" FROM DUMMY;
```

El siguiente ejemplo muestra cómo redondear 4,5 a una serie incrementada en 3. El siguiente ejemplo muestra el resultado 3:

```
SELECT SERIES_ROUND(4.5, 3, ROUND_HALF_DOWN) "result" FROM DUMMY;
```

El siguiente ejemplo muestra cómo redondear por defecto una fecha al principio del año. Devuelve el resultado 1 de enero de 2013:

```
SELECT SERIES_ROUND('2013-05-24', 'INTERVAL 1 YEAR', ROUND_DOWN) "result"  
FROM DUMMY;
```

El siguiente ejemplo muestra cómo redondear por exceso un tiempo al siguiente intervalo de 10 minutos. Devuelve el resultado 4:30:00 AM:

```
SELECT SERIES_ROUND('04:25:01', 'INTERVAL 10 MINUTE') "result" FROM DUMMY;
```

Función SESSION_CONTEXT (varios)

Devuelve el valor de la variable de sesión especificada asignada al usuario actual.

Sintaxis

```
SESSION_CONTEXT(<session_variable>)
```

Descripción

La <session_variable> puede estar predefinida o definida por el usuario. Las variables de sesión predefinidas que puede fijar el cliente son APPLICATION, APPLICATIONUSER y TRACEPROFILE.

Las variables de sesión se pueden definir o modificar mediante una sentencia SET [SESSION] <variable_name> = <value>, y se pueden anular mediante una sentencia UNSET [SESSION] <variable_name>.

SESSION_CONTEXT devuelve un NVARCHAR con una longitud máxima de 512 caracteres.

Ejemplo

La siguiente consulta devuelve el valor HDBStudio o un valor similar en función de sus opciones:

```
SELECT SESSION_CONTEXT('APPLICATION') "session context" FROM DUMMY;
```

Información relacionada

[Declaraciones de gestión de sesiones](#)

[Función SESSION_USER \(varios\)](#)

[M_SESSION_CONTEXT Vista de sistema](#)

[Sentencia SET \[SESSION\] \(gestión de sesiones\)](#)

[Declaración UNSET \[SESSION\] \(gestión de sesiones\)](#).

Función SESSION_USER (varios)

Devuelve el nombre de usuario de la sesión actual.

Sintaxis

SESSION_USER

Descripción

SESSION_USER y CURRENT_USER son funciones similares, pero hay diferencias. SESSION_USER refleja cómo la sesión conectada al servidor, mientras que CURRENT_USER puede verse afectada por la cláusula SQL SECURITY y refleja cómo se aplican los permisos.

Ejemplo

La siguiente consulta devuelve el nombre de usuario de la sesión actual:

```
SELECT SESSION_USER "session user" FROM DUMMY;
```

Tenga en cuenta el siguiente procedimiento de modo de definición ficticio declarado por USER_A:

```
CREATE PROCEDURE USER_A.PROC1 LANGUAGE SQLSCRIPT SQL SECURITY DEFINER AS
BEGIN
    SELECT SESSION_USER "session user" , CURRENT_USER "current user" FROM DUMMY;
END;
```

La siguiente consulta ficticia devuelve USER_B cuando USER_B ejecuta USER_A.PROC:

```
CALL USER_A.PROC1;
```

Tenga en cuenta el siguiente procedimiento de modo de invocador ficticio declarado por USER_A:

```
CREATE PROCEDURE USER_A.PROC2 LANGUAGE SQLSCRIPT SQL SECURITY INVOKER AS
BEGIN
    SELECT SESSION_USER "session user" , CURRENT_USER "current user" FROM DUMMY;
END;
```

La siguiente consulta ficticia devuelve USER_B cuando USER_B ejecuta USER_A.PROC:

```
CALL USER_A.PROC2;
```

Información relacionada

[Declaraciones de gestión de sesiones](#)

[M_SESSION_CONTEXT Vista de sistema](#)

[Función SESSION_CONTEXT \(varios\)](#)

[Función CURRENT_USER \(varios\)](#)

Función SIGN (numérica)

Devuelve el signo (positivo o negativo) del argumento numérico especificado.

Sintaxis

`SIGN(<number>)`

Descripción

Devuelve 1 si <número> es un valor positivo, -1 si <número> es un valor negativo, 0 si <número> es igual a cero y NULL si <número> es igual a NULL.

Ejemplo

El siguiente ejemplo devuelve el valor -1 para "signo":

```
SELECT SIGN (-15) "sign" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función SIN (numérica)

Devuelve el seno de un ángulo expresado en radianes.

Sintaxis

`SIN(<number>)`

Descripción

Devuelve el seno de <número>, donde <número> es un ángulo expresado en radianes.

Ejemplo

En el siguiente ejemplo, se muestra el valor 1.0 para "sine":

```
SELECT SIN ( 3.141592653589793/2) "sine" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función SINH (numérica)

Devuelve el seno hiperbólico de un ángulo expresado en radianes.

Sintaxis

`SINH(<number>)`

Descripción

Devuelve el seno hiperbólico de <número>, donde el argumento es un ángulo expresado en radianes.

Ejemplo

El siguiente ejemplo devuelve el valor 0 para "sinh":

```
SELECT SINH (0.0) "sinh" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función SOUNDEX (cadena)

Convierte caracteres alfabéticos en un código de sonido que representa su sonido.

Sintaxis

`SOUNDEX(<string>)`

Elementos de sintaxis

<string>

Especifica la cadena que se va a convertir en código de sonido.

Descripción

Utilice la función SOUNDEX para encontrar nombres que suenen iguales pero que se escriben de forma diferente.

SOUNDEX devuelve un código que consta de una letra y tres números. La letra es la primera letra de la palabra, y los números representan las siguientes tres consonantes de la palabra, distintas de H, W e Y, que son ignoradas. Las vocales también se ignoran, a menos que sean la primera letra de la cadena. Las letras dobles se cuentan como una letra. Si es necesario, se añaden ceros al final para garantizar un código de cuatro caracteres.

Se ignoran los siguientes caracteres:

- Espacios en blanco
- Líneas nuevas
- Feeds de formulario

- Devoluciones de carro
- Feeds de línea
- Cualquiera de los siguientes: ! " # \$ % & ' () * + , - . / : ; < = > ? @ [] ^ _ ` { | } ~

Ejemplo

El siguiente ejemplo ficticio devuelve dos códigos idénticos, S530, que representan el sonido de cada nombre:

```
SELECT SOUNDEX ('Smith'), SOUNDEX ('Smythe') FROM DUMMY;
```

Este ejemplo ficticio busca en MyTable.MyColumn valores que suenan como Smith:

```
SELECT * FROM MyTable WHERE SOUNDEX(MyColumn) = SOUNDEX('SMITH');
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función SQRT (numérica)

Devuelve la raíz cuadrada del argumento especificado.

Sintaxis

```
SQRT(<number>)
```

Descripción

Devuelve la raíz cuadrada del argumento numérico <número>.

Ejemplo

El siguiente ejemplo devuelve el valor 1.4142135623730951 para "sqrt":

```
SELECT SQRT (2) "sqrt" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función STATEMENT_EXECUTION_HOST (varios)

Devuelve el host del nodo que ejecuta la sentencia.

Sintaxis

STATEMENT_EXECUTION_HOST()

Descripción

Esta función se evalúa una vez por consulta. La misma información aparece en la columna HOST, como la consulta de la vista del sistema M_CONNECTIONS; sin embargo, el uso de esta función es más rápido y menos costoso.

Ejemplo

```
SELECT STATEMENT_EXECUTION_HOST() FROM DUMMY;

DO BEGIN
  DECLARE lv_host NVARCHAR(256) = STATEMENT_EXECUTION_HOST();
  DECLARE lv_port INTEGER = STATEMENT_EXECUTION_PORT();
  SELECT * FROM M_CE_CALCSCENARIOS WHERE HOST = :lv_host AND PORT = :lv_port;
END;
```

Función STATEMENT_EXECUTION_PORT (varios)

Devuelve el puerto del nodo que ejecuta la sentencia.

Sintaxis

STATEMENT_EXECUTION_PORT()

Descripción

Esta función se evalúa una vez por consulta. La misma información aparece en la columna HOST, como la consulta de la vista del sistema M_CONNECTIONS; sin embargo, el uso de esta función es más rápido y menos costoso.

Ejemplo

```
SELECT STATEMENT_EXECUTION_HOST() FROM DUMMY;

DO BEGIN
  DECLARE lv_host NVARCHAR(256) = STATEMENT_EXECUTION_HOST();
  DECLARE lv_port INTEGER = STATEMENT_EXECUTION_PORT();
  SELECT * FROM M_CE_CALCSCENARIOS WHERE HOST = :lv_host AND PORT = :lv_port;
END;
```

Función STDDEV (agregado)

Devuelve la desviación estándar de la expresión dada como raíz cuadrada de la función VAR. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

STDDEV([ALL | DISTINCT] <expression>)

Función de ventana:

```
STDDEV( <expression> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica los datos de entrada para la función.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

El tipo de datos resultante para esta función depende del tipo de datos de entrada, como se describe en la tabla siguiente:

Tipo de resultado basado en entrada							
TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL (p, s)	DECIMAL	REAL	DOUBLE
DECIMAL (9,6)	DECIMAL (11,6)	DECIMAL (16,6)	DECIMAL (25,6)	DECIMAL (p', s)	DECIMAL	REAL	DOUBLE

i Nota

Para DECIMAL(p, s), el tipo de resultado DECIMAL(p', s) cumple las siguientes reglas de precisión:

- p' = 18, si p ≤ 18
- p' = 28, si 18 < p ≤ 28
- p' = 38, si 28 < p ≤ 38

Para obtener más información, consulte *Expresiones* en la sección *Información relacionada*.

Ejemplo

Las siguientes sentencias crean una tabla con datos, por ejemplo:

```
DROP TABLE "MyProducts";
CREATE COLUMN TABLE "MyProducts"(
  "Product_ID" NVARCHAR(10),
  "Product_Name" NVARCHAR(100),
  "Category" NVARCHAR(100),
  "Quantity" INTEGER,
  "Price" DECIMAL(10,2),
  PRIMARY KEY ("Product_ID") );

INSERT INTO "MyProducts" VALUES('P1','Shirts', 'Clothes', 32, 20.99);
INSERT INTO "MyProducts" VALUES('P2','Jackets', 'Clothes', 16, 99.49);
INSERT INTO "MyProducts" VALUES('P3','Trousers', 'Clothes', 30, 32.99);
```

```
INSERT INTO "MyProducts" VALUES('P4','Coats','Clothes', 5, 129.99);
INSERT INTO "MyProducts" VALUES('P5','Purse','Accessories', 3, 89.49);
```

El siguiente ejemplo devuelve 46,13, la desviación estándar de los valores de la columna Precio:

```
SELECT STDDEV("Price") FROM "MyProducts";
```

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Función STDDEV_POP \(agregar\)](#)
- [Función STDDEV_SAMP \(agregar\)](#)
- [Agregar funciones](#)
- [Expresiones](#)

Función STDDEV_POP (agregar)

Devuelve la desviación estándar de una expresión determinada como raíz cuadrada de la función VAR_POP.

Sintaxis

```
STDDEV_POP(<expression>)
```

Descripción

Devuelve la desviación estándar de la expresión dada como raíz cuadrada de la función VAR_POP.

El tipo de datos resultante para esta función depende del tipo de datos de entrada, como se describe en la tabla siguiente:

Tipo de resultado basado en entrada							
TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL (p, s)	DECIMAL	REAL	DOUBLE
DECIMAL (9,6)	DECIMAL (11,6)	DECIMAL (16,6)	DECIMAL (25,6)	DECIMAL (p', s)	DECIMAL	REAL	DOUBLE

i Nota

Para DECIMAL(p, s), el tipo de resultado DECIMAL(p', s) cumple las siguientes reglas de precisión:

- p' = 18, si p ≤ 18
- p' = 28, si 18 < p ≤ 28
- p' = 38, si 28 < p ≤ 38

Para obtener más información, consulte *Expresiones* en la sección *Información relacionada*.

Ejemplos

El siguiente ejemplo devuelve 0 para la desviación estándar de la expresión especificada:

```
CREATE ROW TABLE RTABLE (A INT);
INSERT INTO RTABLE VALUES (1);
SELECT STDDEV_POP(A) "STDDEVPOP" FROM RTABLE;
```

Los siguientes ejemplos devuelven 0,5 para la desviación estándar de la expresión especificada:

```
INSERT INTO RTABLE VALUES (2);
SELECT STDDEV_POP(A) "STDDEVPOP" FROM RTABLE;
```

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Función STDDEV \(agregado\)](#)
- [Función STDDEV_SAMP \(agregar\)](#)
- [Agregar funciones](#)
- [Expresiones](#)

Función STDDEV_SAMP (agregar)

Devuelve la desviación estándar de la expresión dada como raíz cuadrada de la función VAR_SAMP.

Sintaxis

```
STDDEV_SAMP(<expression>)
```

Descripción

Devuelve la desviación estándar de la <expresión> dada como la raíz cuadrada de la función VAR_SAMP.

El tipo de datos resultante para esta función depende del tipo de datos de entrada, como se describe en la tabla siguiente:

Tipo de resultado basado en entrada							
TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL (p, s)	DECIMAL	REAL	DOUBLE
DECIMAL (9,6)	DECIMAL (11,6)	DECIMAL (16,6)	DECIMAL (25,6)	DECIMAL (p', s)	DECIMAL	REAL	DOUBLE

i Nota

Para DECIMAL(p, s), el tipo de resultado DECIMAL(p', s) cumple las siguientes reglas de precisión:

- p' = 18, si p ≤ 18
- p' = 28, si 18 < p ≤ 28
- p' = 38, si 28 < p ≤ 38

Para obtener más información, consulte *Expresiones* en la sección *Información relacionada*.

Ejemplos

El siguiente ejemplo devuelve NULL para la desviación estándar de la expresión especificada, como raíz cuadrada de la función VAR_SAMP:

```
CREATE ROW TABLE RTABLE (A INT);
INSERT INTO RTABLE VALUES (1);
SELECT STDDEV_SAMP(A) "STDDEVSAMP" FROM RTABLE;
```

El siguiente ejemplo devuelve 0,707107 para la desviación estándar de la expresión especificada, como raíz cuadrada de la función VAR_SAMP:

```
INSERT INTO RTABLE VALUES (2);
SELECT STDDEV_SAMP(A) "STDDEVSAMP" FROM RTABLE;
```

Información relacionada

[Funciones de ventana y la especificación de ventana](#)

[Funciones agregadas de ventana](#)

[Función STDDEV \(agregado\)](#)

[Función STDDEV_POP \(agregar\)](#)

[Agregar funciones](#)

[Expresiones](#)

Función STRING_AGG (agregar)

Devuelve la cadena de concatenación de la expresión especificada.

Sintaxis

```
STRING_AGG( <expression>[, <delimiter> ] [ <order_by_clause> ] )
```

Elementos de sintaxis

<expression>

Especifica una expresión NVARCHAR con valores que se deben concatenar. Si los valores de entrada son un tipo de datos diferente a NVARCHAR, se intentará el casting implícito.

Por ejemplo, si la columna NUM tiene cinco valores enteros (1, 2, 3, 4, 5), STRING_AGG("NUM",0) devuelve 102030405.

<delimiter>

Especifica el carácter que se utilizará como delimitador al agregar valores.

<order_by_clause>

Especifica el orden de clasificación de las filas de entrada.

```
<order_by_clause> ::= ORDER BY <order_by_expression> [, <order_by_expression> [...] ]
```

```
<order_by_expression> ::=
```

```
<column_name> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
| <column_position> [ <collate_clause> ] [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
```

```
<collate_clause> ::= COLLATE <collation_name>
```

<clausula colateral> especifica la intercalación que se va a utilizar para ordenar los valores en los resultados. <collate_cláusula> solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un sinónimo de NVARCHAR). <collation_name> es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

Descripción

Los valores NULL se tratan como cadenas vacías.

El orden predeterminado es ASC NULLS FIRST. Si se especifica DESC, la expresión ORDER BY se convierte en DESC NULLS LAST.

Ejemplo

El siguiente ejemplo crea la tabla r1 y la rellena con datos.

```
CREATE ROW TABLE r1(a INT, str NVARCHAR(20), grp INT);

INSERT INTO r1 VALUES (3,'str2',0);
INSERT INTO r1 VALUES (0,'str1',0);
INSERT INTO r1 VALUES (NULL,'NULL',0);
INSERT INTO r1 VALUES (5,'str3',0);
INSERT INTO r1 VALUES (3,'val3',1);
INSERT INTO r1 VALUES (6,'val6',1);
INSERT INTO r1 VALUES (NULL,'NULL',1);
INSERT INTO r1 VALUES (1,'val1',1);
```

Ejecute la siguiente sentencia para devolver la cadena de concatenación de cada registro de la tabla r1 en orden ascendente.

```
SELECT grp, STRING_AGG(str, ',' ORDER BY a)AS agg FROM R1 GROUP BY grp;
```

La sentencia anterior devuelve los siguientes resultados.

GRP	AGG
0	NULL,str1,str2,str3
1	NULL,val1,val3,val6

Ejecute la siguiente sentencia para devolver la cadena de concatenación de cada registro de la tabla r1 en orden descendente.

```
SELECT grp, STRING_AGG(str, ',' ORDER BY a DESC) AS agg FROM r1 GROUP BY grp;
```

La sentencia anterior devuelve los siguientes resultados.

GRP	AGG
0	str3,str2,str1,NULL
1	val6,val3,val1,NULL

Información relacionada

[Agregar funciones](#)

[Vista del sistema COLLATIONS](#)

Función STRTOBIN (cadena)

Convierte todos los caracteres de una cadena en una codificación binaria mediante la página de códigos especificada.

Sintaxis

```
STRTOBIN(<string>, <codepage>)
```

Descripción

Convierte todos los caracteres de la cadena *<string>* en una codificación binaria mediante la *<codepage>* definida.

Ejemplo

Este ejemplo convierte todos los caracteres de la cadena indicada en codificación binaria UTF - 16BE y devuelve el valor 0041006E0074:

```
SELECT STRTOBIN ('Ant', 'UTF-16BE') "strtobin" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función SUBARRAY (matriz)

Devuelve un subconjunto de valores de la matriz especificada empezando por la posición de inicio especificada.

Sintaxis

```
SUBARRAY(<array_value_expression>, <start_position> , <length>)
```

Descripción

<array_value_expression>

Especifica la matriz para la que la función devuelve el subconjunto de valores.

<start_position>

Especifica dónde empieza el subconjunto de valores en la matriz.

<length>

Determina el número de valores.

Devuelve un conjunto de valores de *<array_value_expression>* empezando por *<start_position>* dentro de la cadena.

Si *<start_position>* es inferior o igual a 0, se considera 1.

Si *<length>* es menor que 0, o es mayor que la parte restante de *<array_value_expression>*, SUBARRAY devuelve la parte restante de *<start_position>*. Si *<length>* es exactamente 0, la función devuelve una matriz vacía.

Ejemplo

El siguiente ejemplo devuelve subconjuntos de la matriz especificada:

```
CREATE COLUMN TABLE ARRAY_TEST (IDX INT, VAL INT ARRAY);
INSERT INTO ARRAY_TEST VALUES (1, ARRAY(1, 2, 3));
INSERT INTO ARRAY_TEST VALUES (2, ARRAY(10, 20, 30, 40));
```

El siguiente ejemplo genera una submatriz que empieza por 1 y tiene una longitud de 2:

```
SELECT SUBARRAY(VAL, 1, 2) "subarray" FROM ARRAY_TEST;
```

La sentencia anterior devuelve los siguientes resultados:

Submatriz
1, 2
10, 20
1, 2
10, 20

El siguiente ejemplo genera una submatriz que empieza por 1 y con una longitud de 10:

```
SELECT SUBARRAY(VAL, 1, 10) "subarray" FROM ARRAY_TEST;
```

La sentencia anterior devuelve los siguientes resultados:

Submatriz
1, 2, 3
10, 20, 30, 40
1, 2, 3
10, 20, 30, 40
1, 2, 3
10, 20, 30, 40

Información relacionada

[Expresiones](#)

SUBSTR_AFTER Función (cadena)

Devuelve una subcadena de una cadena especificada que sigue la primera ocurrencia del patrón especificado.

Sintaxis

SUBSTR_AFTER(<string>, <pattern>)

Elementos de sintaxis

<string>

Especifica la cadena que contiene la subcadena.

<pattern>

Especifica la ocurrencia dentro de la cadena que marca el lugar inmediatamente antes de la subcadena devuelta.

Descripción

Devuelve una subcadena de la cadena especificada que sigue la primera ocurrencia del patrón especificado.

- Si <string> no contiene la subcadena <pattern>, se devuelve una cadena vacía.
- Si <pattern> es una cadena vacía, se devuelve <string>.
- Si <string> o <pattern> es NULL, entonces se devuelve NULL.

Ejemplo

Este ejemplo devuelve Amigo, la parte de la cadena dada que está a la derecha de la primera ocurrencia de Mi:

```
SELECT SUBSTR_AFTER ( 'Hello My Friend', 'My ' ) "substr after" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)